JOHANNES KEPLER
UNIVERSITY LINZ

Author
Hannes Sochor
51821001

Submission
LIT Secure and Correct Systems Lab

Supervisor and First Evaluator
Prof. Dr. **Robert Wille**

Second Evaluator
Prof. Dr. sc. **Mathias Payer**

Assistant Thesis Supervisor
Dr. **Flavio Ferrarotti**

August 2025

Fuzzing-based Grammar Inference for Context-free Languages

Doctoral Thesis

to obtain the academic degree of

Doktor der technischen Wissenschaften

in the Doctoral Program

Technische Wissenschaften

JOHANNES KEPLER UNIVERSITY LINZ
Altenbergerstraße 69
4040 Linz, Austria
www.jku.at
DVR 0093696

Kurzfassung

Diese Dissertation befasst sich mit dem Problem der Grammatikinferenz (Grammar Inference) für kontextfreie Sprachen. Der Hauptbeitrag dieser Arbeit liegt in der Entwicklung und umfassenden Evaluierung von Methoden, welche die Fähigkeiten moderner grammatikbasierter Fuzzer (grammar-based fuzzer) einsetzen, um eine kontextfreie Sprache von einem Programm abzuleiten. Der grammatikbasierte Fuzzer wird verwendet, um eine große Menge positiver und negativer Beispiele zu generieren, welche mithilfe des Programms als Orakel klassifiziert werden. Dieser zweistufige Prozess (i) extrahiert eine Ausgangsgrammatik aus dem Quellcode, welche anschließend verwendet wird, um (ii) eine kontextfreie Grammatik abzuleiten. In Summe werden drei verschiedene Methoden vorgestellt, wobei alle einen fundamental anderen Ansatz verfolgen. Die erste Methode versucht sich durch schrittweises Verbessern der Ausgangsgrammatik an die Zielgrammatik anzunähern. Dabei werden in jedem Schritt neue, vielversprechende Grammatiken als Kandidaten gesucht und verbessert, bis eine gute Annäherung an die Zielgrammatik gefunden, oder alle Kandidaten verworfen wurden. Die zweite Methode baut auf einer Ausgangsgrammatik auf, welche auf Kosten geringer Präzision garantiert alle Produktionsregeln der Zielgrammatik enthält. Danach werden schrittweise überflüssige Regeln identifiziert und entfernt, bis eine gute Annäherung an die Zielgrammatik gefunden wurde. Die dritte Methode basiert auf einer unvollständigen Ausgangsgrammatik, in welcher mithilfe eines Lernalgorithmus fehlende Produktionsregeln identifiziert und hinzugefügt werden. Jede Methode wurde hinsichtlich ihrer Komplexität und praktischen Anwendbarkeit evaluiert. Die Ergebnisse der Evaluierung zeigen, dass die vorgestellten Methoden im Vergleich zu verwandten Arbeiten, mithilfe von weniger komplexen Programmanalysemethoden und ohne zusätzlicher Information in Form von Beispielen aus der Zielsprache, in angemessener Zeit gute Ergebnisse liefern.

Abstract

This thesis addresses the problem of grammar inference for context-free languages. The main contribution of this thesis lies in the proposal, development, and extensive evaluation of novel methods that utilize the capabilities of modern grammar-based fuzzers to infer a context-free grammar from some program. We utilize the fuzzer to generate a large amount of positive and negative examples, using the program as an oracle during grammar inference. In particular, we propose a two-step process that (i) extracts a seed grammar from source code that is then used to (ii) infer a context-free grammar. In total, we present three different methods, each following a fundamentally different approach. The first method aims to approximate the target grammar by iteratively improving the seed grammar. In each step, it searches for new promising candidates and tries to further improve them until either a good approximation is found or the candidate is deemed unfit and thus discarded. The second method implements a top-down approach that starts with a seed grammar that is guaranteed to contain each production rule of the target grammar at the cost of very low precision. The method proceeds to identify and remove incorrect rules until the resulting grammar fits the target language well. The third method starts with a seed grammar with perfect precision but low recall and applies a learning algorithm to automatically identify and add missing parts in incomplete production rules. We evaluated each method in terms of complexity and practical applicability. Our results show that our methods remove the dependence on any given seed input, use less resource intensive program analysis methods, all while outperforming related work in terms of precision and recall of the inferred grammars.

Acknowledgements

First and foremost, I would like to express my deepest gratitude to my first supervisor, Prof. Dr. Robert Wille, whose expertise and guidance have been essential to initially shape this thesis into what it is now. Whenever I needed a sparring partner to discuss some ideas or to decide on a publication strategy, Robert was always there to support me in the best way possible.

I am also immensely grateful to Dr. Flavio Ferrarotti for his detailed and constructive feedback, which greatly enriched the quality of this work. Without our hour-long discussions on the topics of learnability and language theory, this thesis would not have been possible. In addition, his unwavering support in formalizing the (somewhat confusing) ideas coming out of my brain down to a presentable state in the form of scientific papers has been a substantial, irreplaceable help.

I would also like to extend my heartfelt thanks to my wife Christina and both of my children, Johanna and Sophie, for their unwavering emotional support and patience throughout the course of this doctoral journey. Nothing motivates more to keep pushing forward than the constant belief in you from someone you love. Nothing cheers you up more after failing to find a bug for days, or after getting a paper rejected, than spending a good, joyful time with your little ones to forget about it all for the moment being.

Lastly, I would like to acknowledge the financial support provided by the Software Competence Center Hagenberg. The research reported in this Ph.D. thesis has been funded by the Federal Ministry for Innovation, Mobility and Infrastructure (BMIMI), the Federal Ministry for Economy, Energy and Tourism (BMWET), and the State of Upper Austria in the frame of the COMET Module Dependable Production Environments with Software Security (DEPS) (FFG grant no. 888338) and the SCCH competence center INTEGRATE (FFG grant no. 892418) within the COMET - Competence Centers for Excellent Technologies Programme managed by Austrian Research Promotion Agency FFG.

Contents

1 Introduction **1**
1.1 Objectives 4
1.2 Contributions 5
1.3 Publications 7
1.4 Outline 8

2 Preliminaries **10**
2.1 Language Theory 10
2.1.1 Notation 10
2.1.2 Chomsky Hierarchy 12
2.2 Grammar-based Fuzzing 13
2.3 Equivalence of context-free grammars 16
2.3.1 Precision and Recall 16
2.4 Running Example 17

3 Overview **20**

4 Experiment Setup **23**

5 Fuzzing-based Grammar Inference by Approximation **31**
5.1 Core Grammar 31
5.2 Approximation Algorithm 32
5.2.1 Need for solution classes 33
5.2.2 Forking the search process 35
5.2.3 Need for Instrumentation 35
5.2.4 Example 36
5.2.5 Complexity Analysis 38
5.3 Core Grammar Extraction 39
5.3.1 Learning Terminals and Tokens 39
5.3.2 AST analysis based core grammar extraction 41
5.3.3 CFG analysis based core grammar extraction 43
5.4 Evaluation & Results 46

6 Fuzzing-based Grammar Inference by Refinement **50**
6.1 Expanded Core Grammar 51
6.2 Refinement Algorithm 51
6.2.1 Complexity Analysis 56
6.3 From Core Grammar to Expanded Core Grammar 59
6.4 Evaluation & Results 59

7 Fuzzing-based Grammar Inference by Learning **66**
7.1 Grammar Fractions 67
7.2 Learning Algorithm 67
7.2.1 Learning regular languages 68
7.2.2 Adapted Learning algorithm 69
7.2.3 Complexity Analysis 76
7.2.4 Angluin-style learning of context-free languages 77
7.3 Grammar Fraction Extraction 80
7.3.1 Dynamic Symbolic Execution based Grammar Fraction Extraction 81
7.3.2 From Expanded Core Grammar to Grammar Fraction 82
7.3.3 Complexity Analysis 84
7.4 Evaluation & Results 85

8 Conclusion and Outlook **96**
8.1 Related Work 96
8.2 Discussion 100
8.2.1 Limitations 104
8.3 Conclusion 106
8.4 Epilogue 108

List of Figures

1.1 Overview of the standard Grammar Inference Process. 2
1.2 Overview of the Fuzzing-based Grammar Inference Process. 3

3.1 General fuzzing-based grammar inference process 21

4.1 Overview on test strategy 25

5.1 Core Grammar with Conflicting Place Holders Resolutions 34
5.2 Derivation trees produced during Grammar Inference by Approximation 37
5.3 Candidate Grammars produced by Grammar Inference by Approximation 38
5.4 AST of method `Expr` 42
5.5 Control Flow Graph Refinement to Core Grammar of Method *Factor* . . . 44
5.6 Control Flow Graph Refinement of Method *Expr* 45
5.7 Resulting core grammar from control flow analysis. 46
5.8 Evolution of recall during Grammar Inference by Learning 48
5.9 Visualisation of all forked branches while learning JSON using Grammar Inference by Approximation 49

6.1 Parse Trees produced during Grammar Inference by Refinement 55

7.1 Parse Tree Evolution of Query 1 74
7.2 Parse Tree Evolution of Query 2 75
7.3 Parse Tree Evolution of Query 3 75
7.4 From the parse trees to a grammar fraction 83
7.5 Json Learning process 91
7.6 URL Learning process 95

List of Tables

5.1 Experiment Results for Grammar Inference by Approximation. 47

6.1 Experiment Results for Grammar Inference by Refinement 61

7.1 Experiment Results for Grammar Inference by Learning (CoCo/R) 86
7.2 Experiment Results for Grammar Inference by Learning (ANTLR) 86

8.1 Experiment Results in comparison to Related Work. 101

Listings

2.1 Target Grammar G_p 17
2.2 Java code of ExprParser 19

4.1 MathExpr language grammar 26
4.2 Mail language grammar 26
4.3 Hello language grammar 27
4.4 AdvMathExpr language grammar 27
4.5 Json language grammar 28
4.6 URL language grammar 30

7.1 Example of a Learned Grammar 76
7.2 ExprParser Extracted Grammar Fraction. 86
7.3 ExprParser Grammar inferred by the learning based approach. 86
7.4 Mail Extracted Grammar Fraction. 87
7.5 MailParser Grammar inferred by the learning based approach. 87
7.6 HelloParser Extracted Grammar Fraction. 87
7.7 HelloParser Grammar inferred by the learning based approach. 88
7.8 AdvExprParser Grammar Fraction. 88
7.9 AdvExprParser Grammar inferred by the learning based approach. 89
7.10 JsonParser Extracted Grammar Fraction. 90
7.11 JsonParser Grammar inferred by the learning based approach. 92
7.12 URLParser Extracted Grammar Fraction 93
7.13 URLParser Grammar inferred by the learning based approach. 94

Acronyms

AST Abstract Syntax Tree

CFG Control Flow Graph

CNF Chomsky Normal Form

CYK CockeYoungerKasami Algorithm

DFA Deterministic Finite Automaton

DSE Dynamic Symbolic Execution

EQ Equivalence-Query

LLM Large Language Model

MBT Model-Based Testing

MQ Membership-Query

NFA Nondeterministic Finite Automaton

OMG Object Management Group

PAC Probably Approximately Correct Learning Algorithm

VPL Visibly PushDown Language

List of Symbols

Language Theory

G A grammar defined by (N, Σ, P, S).

$\mathcal{L}(G)$ The language produced by G.

N Set of non-terminal symbols.

Σ Set of terminal symbols, the alphabet.

P Set of production rules.

S Start symbol.

Global Variables

p Program (Source Code) that is analyzed.

G_p Grammar that produces the input language of p.

G'_p Grammar that approximates the input language of p.

$CFG(p.a)$ Control Flow Graph of procedure a in program p.

G_{core} Core grammar as defined in Section 5.1.

G_{exp} Expanded core grammar as defined in Section 6.1.

G_{frac} Grammar fraction as defined in Section 7.1.

1 Introduction

Testing software for correct and safe program behavior is an essential part of software development. To ensure these properties, various methods have been developed. One way to test whether a program adheres to certain constraints is to define a model that describes the expected program behavior. Making use of such a model enables us to automatically generate new test cases from the model and check whether our program adheres to the expected behavior. This approach is called Model-Based Testing (MBT). Defining the structure of the input a program expects, i.e. the input language of the program, in the form of a formal grammar is one way MBT can be realized in practice. In this case, the grammar is used to produce new inputs that are then fed to the program. Ideally, such a grammar is given. If not, we need to define an appropriate grammar. Manually creating such a grammar can be very time-consuming, so methods that automate this process are essential. In the literature, the task of automatically inferring grammars from examples or source code is known as *Grammar Inference*. A general definition of grammar inference is provided in [1].

Usually, grammar inference is done using positive and, in some cases, negative examples of the target language. These examples are either given or can be artificially crafted by means of symbolic execution or other forms of program analysis, like static source code analysis. Figure 1.1 illustrates the standard grammar inference process, where a grammar is generalized based on some given positive and negative examples, optionally supported by methods for program analysis. The problem of inferring regular languages has been studied extensively and effective methods are available [2, 3]. In contrast, inferring a grammar for context-free languages is not trivial. It has been established early on that context-free languages cannot be learned in the limit from only positive samples according to Gold's Theorem [4]. In this context, learning some language in the limit means that there exists an algorithm that is capable of generating the correct language in all cases after given an infinite number of examples that belong to the language itself.

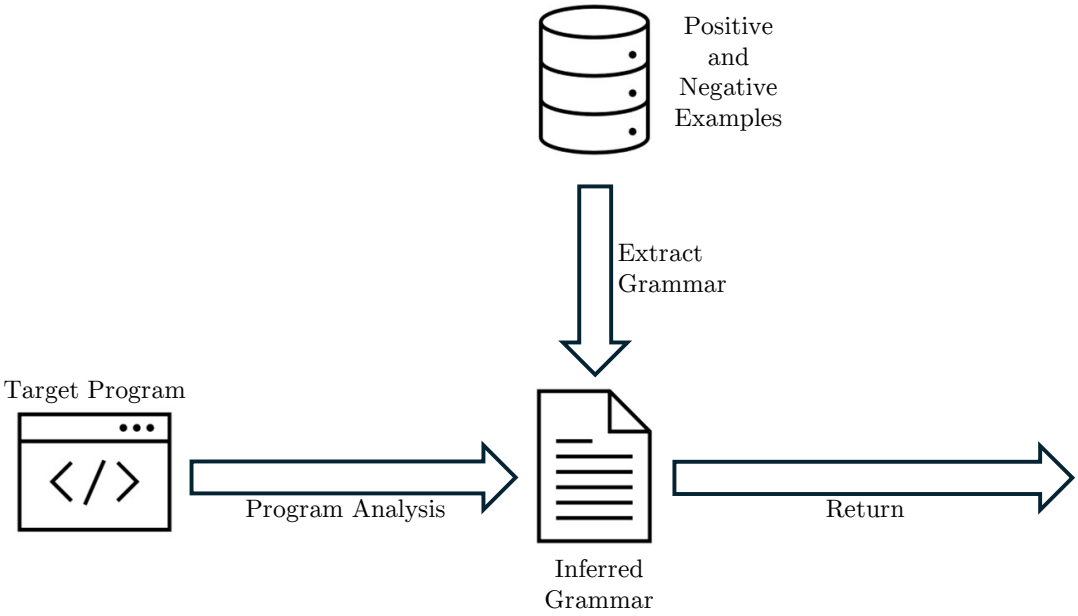


Figure 1.1: Overview of the standard Grammar Inference Process.

This means that while learning a context-free language, we have to provide negative counterexamples that are then used to correct the inferred grammar. In the literature, this problem is typically addressed by introducing a hypothetical teacher capable of supplying such counterexamples [2]. These counterexamples are then used to correct the current state of the grammar learned so far. In practice, the challenge lies in implementing this teacher. One way to implement such a teacher is by randomly sampling a large number of words. This thesis approaches the problem from a novel perspective by applying fuzzing techniques that produce such a set of examples. Using the program as an oracle, these examples can be classified into positive and negative examples.

Fuzzing has developed significantly in the last 30 years [5] from simply generating random input to the application of more sophisticated methods such as mutational generation of new inputs to be able to test the target more efficiently. In our setting, we want to

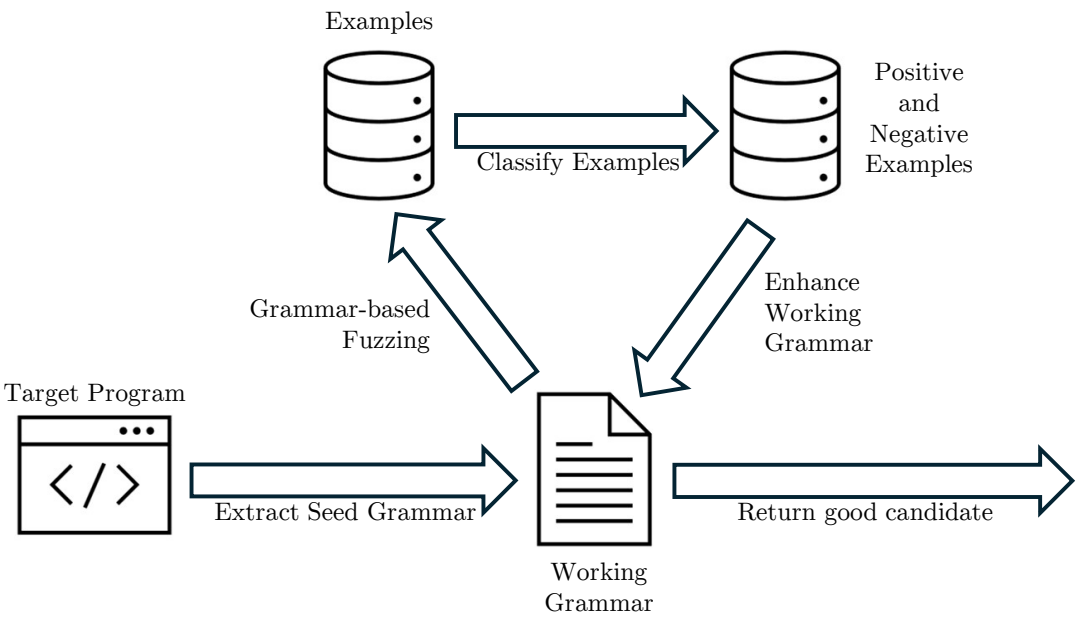


Figure 1.2: Overview of the Fuzzing-based Grammar Inference Process.

generate a large number of examples from a given context-free grammar. In fuzzing[1], the usage of grammars is primarily implemented through grammar-based fuzzing tools such as LangFuzz [7], Nautilus [8], Skyfire [9] and EvoGFuzz [10]. These tools use a context-free grammar to guide input generation, ensuring that the inputs produced conform to the expected language of the target program. This enables the fuzzer to explore deeper program logic by generating well-formed inputs. To uncover issues beyond the defined input language, various mutation strategies can be employed to efficiently generate a large number of edge cases [11].

To be able to apply grammar-based fuzzing to generate a new set of examples, we have to provide an initial seed grammar. This seed grammar is then used by the grammar-based fuzzer to generate the first batch of words which are then used to improve the seed

[1]There exists a large number of different approaches to fuzzing that are not covered here like SAGE [6] that uses symbolic execution and constraint solving to identify new program paths instead of mutations.

grammar and so on. We call this iterative approach to grammar inference *Fuzzing-Based Grammar Inference for Context-Free Languages*, as it relies on the usage of a grammar-based fuzzer. Figure 1.2 illustrates this process: First, we extract the initial seed grammar. This seed grammar is used to produce a new set of words, which are classified into positive and negative examples via membership queries. A membership query executes the target program with a given word as input. Based on the program's response and/or behavior, we determine whether the input belongs to the set of valid inputs. We then use this newly constructed set of positive and negative examples to improve the seed grammar. This process is repeated until a predefined stop condition is met, at which point the resulting grammar is returned. These conditions may include high accuracy of the working grammar or that no more counterexamples can be found in the given context. In itself, this should be a self-improvement process as the more accurate our inferred grammar gets, the more targeted we are able to generate new examples that are then used to further improve our grammar. However, we know that grammar inference of context-free languages is not possible based purely on positive examples according to Gold's Theorem [4] as discussed above. Through fuzzing, we are able to provide negative examples as well, but due to their randomness, those are rarely good counterexamples that are suitable to correct the inferred grammar. For this reason, we leverage program analysis methods to further support grammar inference. Program analysis can support the grammar inference process by providing additional information on how a given input is consumed by the program, eventually revealing structural patterns that hint at the shape of the underlying grammar.

1.1 Objectives

The objective of our research is to identify practical methods that can be applied to realize *Fuzzing-based Grammar Inference*. This includes methods for seed grammar extraction, as well as novel methods for *Grammar Inference* that rely on an initial seed grammar. To achieve this goal, we have defined the following main objectives:

O1: Develop methods to extract seed grammars. Given the source code of a program, we want to extract an initial seed grammar that is capable of producing words with a higher probability of being accepted by the program than we would be able to do by randomly generating new words. To achieve that goal, we want to consider different

methods used in source code analysis, beginning from simple code instrumentation to symbolic execution, among others.

O2: Develop methods that infer a context-free grammar for the input language of a program from its source code and seed grammar. These methods should take some given seed grammar and then improve it based on membership queries and program analysis. The aim is to consider related work on grammar inference and apply and/or adapt those methods to our setting, developing novel methods for grammar inference. We want to ensure the applicability of the developed methods to real world applications (*O2.1*). Furthermore, the inferred grammar should be of high quality (*O2.2*). This means that it produces a language that is equal or very close to the input language of the target program.

O3: Design and implement a reusable evaluation framework to assess the performance of the proposed methods. The evaluated performance within the evaluation framework should aim to verify wether *O1* and *O2* have been achieved. This includes the applicability of the tested methods to various languages, including real-world examples. Furthermore, the quality of the inferred grammars should be measurable in terms of precision and recall (see Section 2.3.1).

1.2 Contributions

This thesis follows the structure of our published and peer-reviewed scientific papers this thesis is based upon. Each of the three core chapters is based on one or two of these publications, contributing to the overall research goal of enabling practical fuzzing-based grammar inference. In the following, we present the contributions made in each chapter individually.

- **Experiment Setup:** Chapter 4 presents the evaluation framework that was designed and implemented to test arbitrary grammar inference methods on a given context-free language. The framework takes a context-free grammar as input, automatically generates one or more parsers that recognize the language defined by that grammar, applies the grammar inference method to the generated parsers, and finally evaluates the inferred grammar in terms of precision and recall with respect to the original

grammar. This approach was first introduced in our earlier work [12] and has since been adopted in all subsequent publications [13, 14, 15].

- **Fuzzing-based Grammar Inference by Refinement:** Our first publication on grammar inference by refinement [12] presented in Chapter 6 introduces the idea of using a *seed grammar*, rather than seed inputs, distinguishing it from related work like [16, 17]. However, note that the idea of using seed grammars is not new and has already been explored by [18, 19, 20], among others. Based on this concept, we developed a refinement-based grammar inference algorithm that infers a grammar with near-perfect precision and recall using only *black-box access* to the target program. This algorithm uses a *grammar-based fuzzer to identify issues* within words up to a certain length, enabling it to remove incorrect rules from the seed grammar accordingly.

- **Fuzzing-based Grammar Inference by Learning:** Chapter 7 presents our algorithm for grammar inference by learning, first introduced in [13]. This approach uses an Angluin-style learning algorithm to infer a context-free grammar from the source code of a program. The key contributions include the *extension and enhancement of Angluins ideas* by integrating program analysis, as well as the *introduction of a fuzzing-based teacher* that supplies counterexamples. Similar to our previous work[12], this method builds upon an initially extracted seed grammar. Our improved seed grammar extraction algorithm [13, 14] uses a significantly *lower number of seed inputs* than related work [21, 17], while returning grammars with *higher recall and precision*.

- **Fuzzing-based Grammar Inference by Approximation:** Chapter 5 presents our algorithm for grammar inference by approximation first published in [15]. This approach, again using an initially extracted seed grammar, automatically identifies and resolves small issues within the seed grammar. To resolve these issues, a *grammar-based fuzzer is used to explore and evaluate candidate modifications* through rapid input generation and feedback. The process is repeated until no further issues remain. The main contributions of this work include an improved method for seed grammar extraction that, using control-flow information, extracts the seed grammar without requiring any seed input. As a result, our method *removes the dependence on seed inputs*, while still achieving *higher precision and recall* compared to related work [21, 17].

- **Other:** The grammar-based fuzzer developed for this thesis has been applied to find weaknesses in the OpenPLC Compiler[2]. We have found one memory corruption and an endless loop in the integrated MatIEC project[3] while compiling structured text [22] code to c-code. At the time of publication, all issues have already been fixed. Furthermore, the grammar-based fuzzer has been directly integrated into the build pipeline of one of our industry partners to automatically test newly developed *IEC 61131-3* [22] code for potential weaknesses.

1.3 Publications

All contributions presented in this thesis have been published in peer-reviewed scientific conferences and journals. The following is a list of all relevant publications in chronological order that have contributed to this thesis:

- Hannes Sochor and Flavio Ferrarotti. "A Refinement Based Algorithm forăLearning Program Input Grammars". In: *From Data to Models and Back*. Ed. by Juliana Bowles, Giovanna Broccia, and Roberto Pellungrini. Cham: Springer International Publishing, 2022, pp. 138–156. ISBN: 978-3-031-16011-0

- Hannes Sochor, Flavio Ferrarotti, and Daniela Kaufmann. "Fuzzing-Based Grammar Inference". In: *Model and Data Engineering*. Ed. by Philippe Fournier-Viger, Ahmed Hassan, and Ladjel Bellatreche. Cham: Springer Nature Switzerland, 2023, pp. 72–86. ISBN: 978-3-031-21595-7
  Best Paper Award

- Hannes Sochor, Flavio Ferrarotti, and Daniela Kaufmann. "Fuzzing-based grammar learning from a minimal set of seed inputs". In: *J. Comput. Lang.* 78 (2024), p. 101252. URL: `https://doi.org/10.1016/j.cola.2023.101252`

- Hannes Sochor, Flavio Ferrarotti, and Robert Wille. "GrammarForge: Learning Program Input Grammars forăFuzz Testing". In: *Software Engineering and Formal Methods*. Ed. by Alexandre Madeira and Alexander Knapp. Cham: Springer Nature Switzerland, 2025, pp. 272–289. ISBN: 978-3-031-77382-2

[2] `https://github.com/thiagoralves/OpenPLC_v3`
[3] `https://github.com/nucleron/matiec`

In addition to scientific publications, the work done for this thesis has also produced a significant number of prototypes as well as evaluation artifacts. As some of this work depends on closed-source software from the Software Competence Center Hagenberg, we were unable to publish everything. However, the evaluation artifacts used for all publications and our prototype presented in [14] are available online: `https://github.com/software-competence-center-hagenberg/fbgl`.

1.4 Outline

The remainder of this thesis is structured as follows. Chapter 2 provides the necessary background knowledge needed to follow the subsequent chapters of the thesis. This includes a brief overview of general language theory and the Chomsky hierarchy, as well as fixing the basic notation used throughout the thesis. An introduction to grammar-based fuzzing follows, focusing on the question of how to efficiently generate words from a given context-free grammar. To evaluate the results of our proposed algorithms for grammar inference, we discuss the usage of recall and precision metrics to measure the equivalence of two context-free grammars. The chapter concludes with the introduction of a simple example of a language, together with its corresponding target grammar, that is used throughout the thesis to illustrate the different grammar inference methods.

We outline the complete process of Fuzzing-Based Grammar Inference in Chapter 3, starting with seed grammar extraction and concluding with the inference of a context-free grammar. We briefly present and discuss our proposed methods for Grammar Inference by Approximation, Refinement and Learning. Next, we define the experiment setup used to properly evaluate different methods for Fuzzing-based Grammar Inference in Chapter 4.

Chapters 5, 6 and 7 present the main contributions of this thesis: Fuzzing-based Grammar Inference by Approximation, Refinement and Learning, respectively. Grammar Inference by Approximation takes an incomplete seed grammar and approximates the target language by systematically exploring new paths using a grammar-based fuzzer. Grammar Inference by Refinement takes a seed grammar that produces a superset of the target language and removes rules until a good approximation is achieved. Again, a grammar-based fuzzer is used to produce words up to a certain length, which are then used to

identify and remove incorrect rules from the grammar. Grammar Inference by Learning makes use of a learning algorithm to approximate the target language. In this scenario, a grammar-based fuzzer is used to provide counterexamples to the learning algorithm. Each chapter is structured as follows: First, we formally define the seed grammar. This is followed by a detailed description of the respective grammar inference algorithm and the methods used for seed grammar extraction. Each chapter concludes with an in-depth evaluation of the proposed approach.

Finally, Chapter 8 concludes the thesis by (i) discussing related work, (ii) summarizing experiment results and comparing them with findings from the literature, and (iii) addressing the limitations of the proposed methods. Finally, we present our conclusions and some ideas for future work in the form of open questions.

2 Preliminaries

This chapter outlines several foundational concepts that are essential for understanding the remainder of this thesis. Specifically, Section 2.1 provides a brief introduction to basic language theory. Building on this, Section 2.2 introduces grammar-based fuzzing, demonstrating how large sets of words can be efficiently generated from a given context-free grammar. Section 2.3 addresses the challenge of reasoning about the equivalence of context-free languages. This poses an undecidable problem in theory, yet is relevant in practice through approximate methods. The chapter concludes with the introduction of a simple context-free grammar and its corresponding parser in Section 2.4 that serves as a running example throughout the thesis.

2.1 Language Theory

This section introduces the basic concepts of formal languages needed to follow the thesis. We define the general notation for formal grammars first, followed by a brief introduction into the well-known Chomsky Hierarchy. Understanding the properties of different types of formal languages, as defined by the Chomsky Hierarchy, is vital to understand why our approach works for regular and context-free languages, but not for more complex types.

2.1.1 Notation

We assume that the reader is familiar with basic concepts of language theory. An excellent reference for that is the classical book by Hopcroft and Ullman [23].

Let Σ be an *alphabet*, that is, a finite set of symbols. A finite sequence of symbols taken from Σ is called a *word* or *string* over Σ. The free monoid of Σ, that is, the set of all (finite)

strings over Σ plus the empty string λ, is denoted as Σ^* and is known as the *Kleene star* of Σ. If $v, w \in \Sigma^*$, then $vw \in \Sigma^*$ is the *concatenation* of v and w and $|vw| = |v| + |w|$ is its length. If $u = vw$, then v is a *prefix* of u and w is a *suffix*. A *language* is any subset of Σ^*.

A *grammar* is formally defined as a 4-tuple $G = (N, \Sigma, P, S)$, where N and Σ are finite disjoint sets of *nonterminal* and *terminal symbols*, respectively, $S \in N$ is the *start symbol* and P is a finite set of *production rules*, each of the form:

$$\alpha \to \gamma$$

for $\alpha \in (\Sigma \cup N)^+$ and $\gamma \in (\Sigma \cup N)^*$.

We say G *derives* (or equivalently *produces*) a string y from a string x in one step, denoted $x \Rightarrow y$, iff there are $u, v, p, q \in (\Sigma \cup N)^*$ such that $x = upv$, $p \Rightarrow q \in P$ and $y = uqv$. We write $x \Rightarrow^* y$ if y can be derived in zero or more steps from x, i.e., $\Rightarrow^*$ denotes the reflexive and transitive closure of the relation $\Rightarrow$.

The language of G, denoted as $\mathcal{L}(G)$, is the set of all strings in Σ^* that can be derived in a finite number of steps from the start symbol S. In symbols,

$$\mathcal{L}(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$$

In this work, Σ always denotes the "input" alphabet (e.g., the set of ASCII characters) of a given executable (binary) program p. The set of *valid inputs of* p is defined as the subset of Σ^* formed by all well formed inputs for p. In symbols:

$$\mathit{validInputs}(p) = \{w \in \Sigma^* \mid w \text{ is a well formed input for } p\}$$

The definition of a well-formed input for a given program p depends on the application at hand. In our setting, we only need to assume that it is possible to determine whether a given input string w is well formed or not for a program p by simply running p with input w. We define that a p *consumes* a word $w \in \Sigma^*$ when it reads and processes part or all of an input string according to its grammar, thereby advancing its input position. A prefix u of w, with $u \in \Sigma^*$ and $w = uv$, is consumed by p if there exists a production $A \Rightarrow^* u$ in G_p, and when applied to w, p processes u and leaves v unconsumed.

2.1.2 Chomsky Hierarchy

The Chomsky hierarchy, as defined by Noam Chomsky, establishes a classification of formal languages into four types. Each type, that is type 0 to type 3, is defined by the structure and limitations of its production rules as well as which type of automata is capable of accepting it. This builds a strict hierarchy of formal languages. A good introduction to this topic is given by Linz [24]. Let us briefly define the four types of the Chomsky Hierarchy in the remainder of this section, beginning with the most complex, type 0 or recursively enumerable languages.

Recursively enumerable languages or type 0 languages are generated by unrestricted grammars accepted by Turing machines. Each production rule has the form

$$\alpha \to \gamma$$

for $\alpha \in (\Sigma \cup N)^+$, $\gamma \in (\Sigma \cup N)^*$ and α being non-empty.

Context-sensitive languages or type 1 languages are generated by context-sensitive grammars and are accepted by a restricted class of Turing machines, namely linear bounded automata. In general, the production rules of context-sensitive grammars are each of the form

$$\alpha A \beta \to \alpha \gamma \beta$$

for $A \in N$ and $\alpha, \beta \in (\Sigma \cup N)^*$ and $\gamma \in (\Sigma \cup N)^+$. This means that the production rule $A \to \gamma$ can only be applied in the "context" of α and β, hence being context-sensitive. In addition, we define $|\alpha\gamma\beta| \geq |\alpha A \beta|$.

Context-free languages or type 2 languages are generated by context-free grammars. A context-free grammar is accepted by a pushdown automata. Context-free languages are defined by limiting the production rules of a grammar to the form

$$A \to \gamma$$

for $A \in N$ and $\gamma \in (\Sigma \cup N)^*$.

In this thesis, we assume that $validInputs(p)$ is a context-free language as this class constitutes the majority of all languages used in computer science in practice, including most programming languages and generic data formats, for example JSON or URL. As such, our main goal is to learn a context-free language. Consequently, we assume that there exists a context-free grammar G_p such that $\mathcal{L}(G_p) = validInputs(p)$.

Regular languages or type 3 languages are generated by regular grammars. A regular language is accepted by finite-state automata. We define a grammar to be right-linear if all production rules are of the form

$$A \rightarrow xB$$

for $A, B \in N$ and $x \in \Sigma^*$. A left-linear grammar has production rules of the form

$$A \rightarrow Bx.$$

A regular grammar that produces a regular language is either right-linear or left-linear.

In addition, a regular language can be defined in the form of a regular expression. Let λ, that is the empty symbol, and $a \in \Sigma$ be primitive regular expressions. Then we define that if r_1 and r_2 are regular expressions, so are $r_1 + r_2, r_1 \cdot r_2, r_1^*$ and (r_1). Those stand for the union, concatenation, star closure and parentheses, respectively. Some string is a regular expression if, and only if, it can be derived from the primitive regular expressions by a finite number of applications of these rules. For example, the regular expression $(a + (b \cdot c))^*$ stands for the star closure of $\{a\} \cup \{bc\}$ that defines the language $\{\lambda, a, bc, aa, abc, bca, bcbc, aaa, aabc, ...\}$.

2.2 Grammar-based Fuzzing

Given some context-free grammar, a grammar-based fuzzer generates a large number of words produced by this grammar in the shortest time possible. Additionally, we want to be able to efficiently generate so-called corner-cases that are not in the target language but are quite close.

As discussed in the previous section, we can derive some word y in zero or more steps from the start symbol S, formally $S \Rightarrow^* y$, by applying the production rules of the grammar. However, here we want to give some additional details on the inner workings of the grammar-based fuzzer that has been implemented for our prototypes. Given a grammar in EBNF [25][1] or COCO/R[2] format, we read rule by rule and split each into elements. Each element is either a terminal symbol, a non-terminal symbol, a sequence of elements, or a set of elements (called an alternative). This enables us to represent complex structures, including nested *and* as well as *or* relations as sequences and alternatives, respectively. Note that this structure consists of a list of rules $nt \rightarrow e$ where $nt \in N$ and e is some new element formed by some potentially nested structure of sub-elements.

Next, we determine the cost c of each element in the grammar, where the cost of an element is defined by the smallest number of steps needed to resolve it to a sequence of terminal symbols. The assignment of costs enables us, in addition to its inherent value, to determine whether certain elements of the grammar are unreachable (=no cost can be assigned) or if the grammar is *generating*. A *generating* grammar produces at least one word consisting only of terminal symbols[23]. In this context, we extend the definition of *generating* grammars to elements, that is, an element of the grammar is *generating* if a word consisting only of terminals can be derived in a finite number of steps by applying the production rules of the grammar. We assign costs according to the following steps:

1. Assign $c = 1$ to each element that is a terminal.
2. Find an element that has no cost assigned but all its children are *generating*; if non can be found return.
3. If the element is a sequence, its cost equals the sum of costs assigned to its children.
4. If the element is an alternative, its cost equals the lowest cost assigned to one of its children.
5. If the element is a nonterminal nt, its cost equals the cost assigned to the element e on the right-hand side of the associated production rule $nt \rightarrow e$ in G.
6. Go to (2)

[1] `https://www.iso.org/standard/26153.html`
[2] `https://ssw.jku.at/Research/Projects/Coco/`

7. If a non-terminal has no cost assigned, remove it's associated production rule.

Finally, we remove all remaining *non-generating* elements from G, that is, all elements that have no cost assigned after the procedure described above.

Whenever we want to produce a new random word, we start by defining a string $w = S$ for start symbol S of the given grammar G and follow the procedure below:

1. Set $i = 0$ and choose some $n \in \mathbb{N}$.
2. Randomly select a non-terminal symbol $nt \in w$.
3. Take the rule $nt \rightarrow e$ in G and replace nt with e in w.
   a) If e is an alternative and $i \leq n$, randomly choose an element of the set.
   b) If e is an alternative and $i > n$, select the element with the smallest cost c from the set.
4. If w contains any non-terminal symbol, go to step (2).
5. Return w.

Choosing the parameter n enables us to force the generation algorithm to produce a *finite* word. This is necessary for certain grammars. Take a look at the production rule given below:

$$\mathtt{S} \rightarrow \mathtt{S}x\mathtt{S} \mid y$$

Each step where we randomly choose either $\mathtt{S}x\mathtt{S}$ or y leaves us with a 50% chance to introduce two new non-terminals with the chance to only remove one non-terminal. We can clearly see that there is a high probability that if we randomly apply the rules of such a grammar the produced word will get larger and larger and the chances of each non-terminal getting finally resolved will converge to zero. However, forcing the algorithm to always select y after a fixed number of random steps will finally lead to a result. In addition, by choosing a smaller or larger n we can guide our procedure to produce smaller or larger words on average, respectively.

Finally, we are able to produce a corner case w' using a simple mutational approach. First, we randomly produce a word w following the procedure described above. Then, we randomly apply one or more of the mutations below at random indexes i in w:

- Randomly select some $t \in \Sigma$ and insert it into w at index i.
- Remove some $t \in w$ at index i.
- Replace some $t \in w$ at index i with a randomly selected $t' \in \Sigma$.

2.3 Equivalence of context-free grammars

In this work we discuss novel algorithms that take as input a program p and infer a grammar G_p such that $\mathcal{L}(G_p)$ approximates $\mathit{validInputs}(p)$. We say "approximates" since it is *not* decidable in our setting (see [4]) if $\mathcal{L}(G_p) = \mathit{validInputs}(p)$. To evaluate how well $\mathcal{L}(G_p)$ approximates $\mathit{validInputs}(p)$, we measure the precision and recall of $\mathcal{L}(G_p)$ w.r.t. $\mathit{validInputs}(p)$ as in [18], among others.

2.3.1 Precision and Recall

In our setting, we first fix a procedure to calculate the probability distribution of a language, starting from its corresponding grammar. Following [18] we use random sampling of strings. Let $G = (N, \Sigma, P, S)$ be a context-free grammar. As a first step, G is converted to a *probabilistic context-free grammar* by assigning a *discrete distribution* $\mathcal{D}_A$ to each nonterminal $A \in N$. As usual, $\mathcal{D}_A$ is of size $|P_A|$, where P_A is the subset of productions in P of the form $A \to \alpha$. Here, we assume that $\mathcal{D}_A$ is *uniform*. We can then *randomly sample a string x from the language* $\mathcal{L}(G, A) = \{w_i \in \Sigma \mid A \Rightarrow^* w_i\}$, denoted $x \sim \mathcal{P}_{\mathcal{L}(G,A)}$, as follows:

- Using $\mathcal{D}_A$ to randomly select a production $A \to A_1 \cdots A_k \in P_A$.
- For $i = 1, \ldots, k$, recursively sample $x_i \sim \mathcal{P}_{\mathcal{L}(G,A_i)}$ if $A_i \in N$; otherwise let $x_i = A_i$.
- Return $x = x_1 \cdots x_k$.

```
 parse → expr
  expr → term | term+term | term−term
  term → factor | factor∗factor | factor/factor
factor → 1 | 2 | 3 | (expr)
```

Listing 2.1: Target Grammar G_p

The *probability distribution* $\mathcal{P}_{\mathcal{L}(G)}$ *of the language* $\mathcal{L}(G)$ is simply defined as the probability $\mathcal{P}_{\mathcal{L}(G,S)}$ induced by sampling strings in the probabilistic version of G defined above.

We can now measure the quality of a learned (or inferred) language $\mathcal{L}'$ with respect to the target language $\mathcal{L}$ in terms of precision and recall.

- The *precision* of $\mathcal{L}'$ w.r.t. $\mathcal{L}$, denoted *precision*$(\mathcal{L}', \mathcal{L})$, is defined as the probability that a randomly sampled string $w \sim \mathcal{P}_{\mathcal{L}'}$ belongs to $\mathcal{L}$. In symbols, $\Pr_{w \sim \mathcal{P}_{\mathcal{L}'}}[w \in \mathcal{L}]$.
- Conversely, the *recall* of $\mathcal{L}'$ w.r.t. $\mathcal{L}$, denoted *recall*$(\mathcal{L}', \mathcal{L})$, is defined as $\Pr_{w \sim \mathcal{P}_{\mathcal{L}}}[w \in \mathcal{L}']$.

We say that $\mathcal{L}'$ is a good approximation to $\mathcal{L}$ if it has both, high precision and high recall. Note that, a language $\mathcal{L}' = \{w\}$, where $w \in \mathcal{L}$, has perfect precision, but most likely has also very low recall. On the other hand, $\mathcal{L}' = \Sigma^*$ has perfect recall, but probably low precision.

2.4 Running Example

Our running example will be used throughout the remaining chapters to illustrate the the different methods proposed in this thesis for grammar inference. All our prototypes have been implemented in java and python. Although our methods are language independent, all the programs used to evaluate our methods are written in Java as well. The code given in Listing 2.2 shows a parser that accepts basic mathematical expressions such as sums, multiplications, etc. This includes nested structures that make use of brackets. Note that the program uses a recursive descending parsing approach that reads some input from left to right with a look-ahead of one to determine if the input is accepted or not. The language that is accepted as well-formed by this program is the one corresponding to the grammar in Listing 2.1. Note that the naming of the parser functions aligns with

the non-terminal symbols in the grammar, which allows us to easily find parts in the program that implement certain rules of the given grammar. In the remaining chapters, the program given in Listing 2.2 is denoted as program p. The grammar in Listing 2.1 is denoted as G_p. Consequently, $\mathcal{L}(G_p)$ denotes the language produced by G_p, which is equivalent to $validInputs(p)$.

```java
public class ExprParser {
  public static int parse(char[] in) {
    int pos = expr(0, in);
    if (pos < in.length) {
      syntaxError("End of input", pos);
      pos = -1;}
    return pos;
  }
  public static int expr(int pos, char[] in) {
    pos = term(pos, in);
    if (pos == -1) return pos;
    if (pos == in.length) return pos;
    if (in[pos] == '+') {pos++; pos = term(pos, in);}
    else if (in[pos] == '-') {pos++; pos = term(pos, in);}
    return pos;
  }
  public static int term(int pos, char[] in) {
    pos = factor(pos, in);
    if (pos == -1) return pos;
    if (pos == in.length) return pos;
    if (in[pos] == '*') {pos++; pos = factor(pos, in);}
    else if (in[pos] == '/') {pos++; pos = factor(pos, in);}
    return pos;
  }
  public static int factor(int pos, char[] in) {
    if (pos == in.length) return -1;
    if (in[pos] == '1') return pos + 1;
    if (in[pos] == '2') return pos + 1;
    if (in[pos] == '3') return pos + 1;
    if (in[pos] == '(') {
      pos++; pos = expr(pos, in);
      if (pos == -1) return pos;
      if (pos == in.length) return -1;
      if (in[pos] == ')') {pos++; return pos;}
      else {syntaxError("invalid factor ", pos);}}
    return -1;
  }
  public static void syntaxError(String msg, int pos) {
    final boolean log = false;
    if (log) {
      System.out.print("Syntax error ");
      System.out.print(msg);
      System.out.print(" col: ");
      System.out.println(pos);}}}
```

Listing 2.2: Java code of ExprParser

3 Overview

Before we provide in-depth details on the different methods for *Fuzzing-based Grammar Inference*, let us first provide an overview. This chapter serves as an orientation for the rest of the thesis. The whole process of *Fuzzing-based Grammar Inference* is illustrated in Figure 3.1. It provides an overview of the entire thesis and refers to the publications in which the methods were proposed first.

The goal is to infer a grammar that produces a language that approximates the input language of a given program p as closely as possible. More precise, precision and recall of the inferred grammar with respect to the target language should be maximized. To achieve this goal, we apply a two step process as illustrated in Figure 3.1: First, we apply a method for *Seed Grammar Extraction*. Second, using the extracted seed grammar, we infer the grammar by applying one of the algorithms described for *Grammar Inference* in Chapters 5 to 7. Note that each algorithm for fuzzing-based grammar inference requires a different kind of seed grammar. These connections are illustrated in Figure 3.1. The exact properties of the seed grammar depend on the fuzzing-based grammar inference algorithm.

More precisely, let $G_p = (N, \Sigma, P, S)$ be a grammar that produces the input language of some program p, that is, $\mathcal{L}(G_p) = validInputs(p)$. The goal is to infer some grammar G'_p such that $\mathcal{L}(G'_p)$ approximates $\mathcal{L}(G_p)$ as closely as possible, measured in precision and recall. Given the source code of p, we extract some seed grammar $G_{seed} = (N_{seed}, \Sigma_{seed}, P_{seed}, S_{seed})$ from it. G_{seed} has to have different properties specified in detail in each chapter where it is used. During seed grammar extraction, we apply procedures that extract N_{seed} as well as Σ_{seed} from the source code of p. If $N_{seed} \subseteq N$ or $\Sigma_{seed} \subseteq \Sigma$, we will not be able to infer a grammar with perfect recall. On the other hand, $N_{seed} \supseteq N$ or $\Sigma_{seed} \supseteq \Sigma$ will decrease runtime performance. In the remainder of the thesis, we assume that $N_{seed} = N$ and $\Sigma_{seed} = \Sigma$ in all cases.

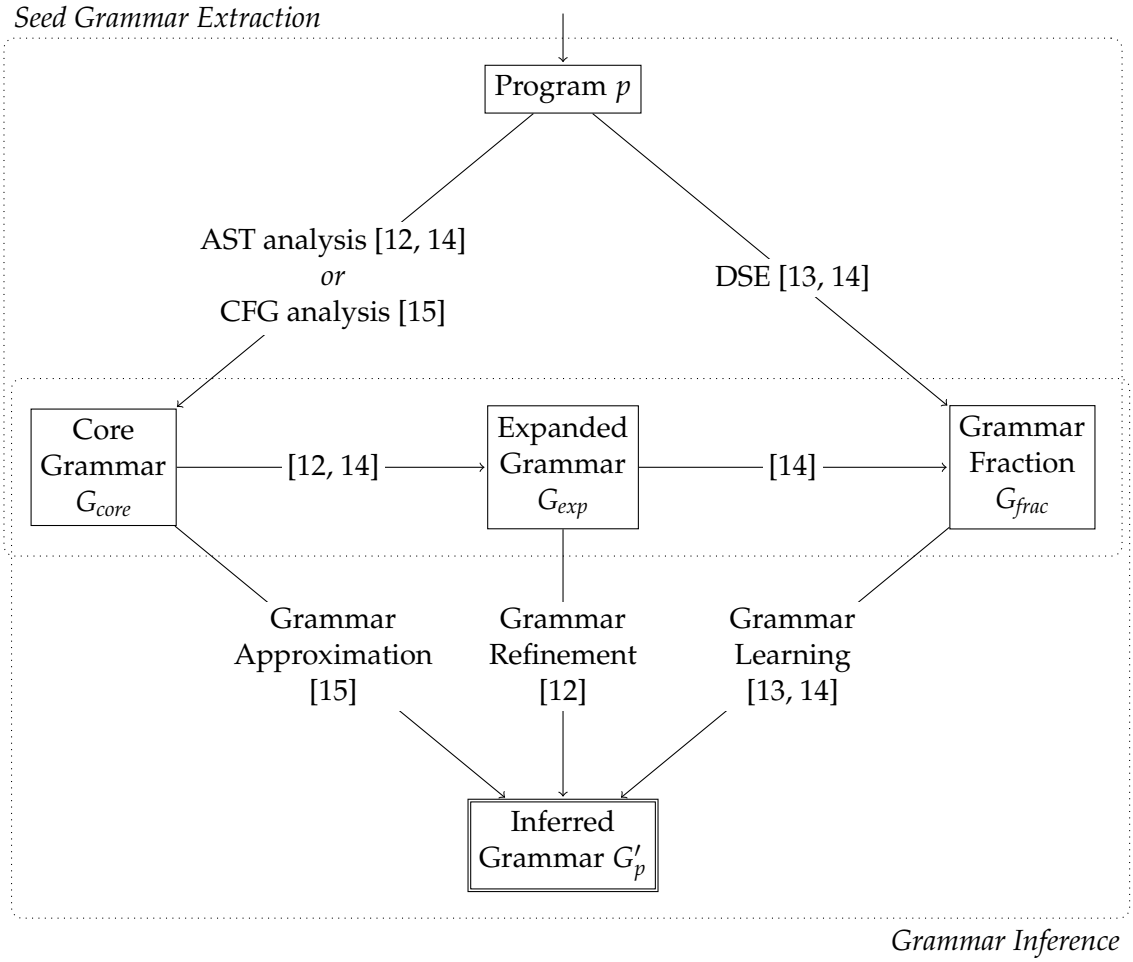


Figure 3.1: General fuzzing-based grammar inference process. This process consists out of two steps: Seed Grammar Extraction and Grammar Inference. Seed Grammar Extraction results in one of three different types of seed grammars, each used in one of the three methods for grammar inference.

To meet our main objective of inferring a grammar G'_p from the source code of p, we apply one of the methods for *Fuzzing-based Grammar Inference* presented in detail in Chapters 5 to 7. These take G_{seed} and infer G'_p using the source code of p. Note that all our proposed methods are based on the ability to randomly generate new words from a given grammar. For that purpose, we use a grammar-based fuzzer that takes any context-free grammar G as input and generates new words $w \in G$ as well as some mutations $w' \notin G$, thus *Fuzzing-based Grammar Inference*.

As illustrated in Figure 3.1, our contributions include three novel methods for *Fuzzing-based Grammar Inference*. This means that there exist three different types of seed grammars, each designed to work with one method for grammar inference. Let us briefly describe these connections, as illustrated in Figure 3.1, to conclude this chapter.

- **Fuzzing-based Grammar Inference by Approximation** uses a seed **Core Grammar** G_{core}, as described in Chapter 5. The properties of G_{core} are defined in Section 5.1. Section 5.2 describes the algorithm used to approximate the rules of G_p, returning some G'_p. Finally, Section 5.3.2 and Section 5.3.3 provide methods to extract a core grammar G_{core} from the source code of p by Abstract Syntax Tree (AST) analysis and Control Flow Graph (CFG) analysis, respectively.

- **Fuzzing-based Grammar Inference by Refinement** uses a seed **Expanded Core Grammar** G_{exp}, as described in Chapter 6. The properties of G_{exp} are defined in Section 6.1. Section 6.2 describes the algorithm used to refine the rules of G_{exp} to approximate G_p, resulting in G'_p. Finally, Section 6.3 describes how a G_{core} can be expanded to a G_{exp}.

- **Fuzzing-based Grammar Inference by Learning** uses a seed **Grammar Fraction** G_{frac}, as described in Chapter 7. The properties of G_{frac} are defined in Section 7.1. Section 7.2 describes a practical implementation of an angluin-style learning algorithm that deterministically approximates the language produced by G_p, returning some G'_p. Finally, Section 7.3.1 provides a method that extracts a grammar fraction G_{frac} from the source code of p using dynamic symbolic execution (DSE). Alternatively, we can produce a grammar fraction G_{frac} from any G_{exp} as described in Section 7.3.2.

4 Experiment Setup

This chapter presents the experiment setup we have used throughout our work for evaluation. Each of the proposed methods has been thoroughly tested in their corresponding papers [13, 12, 15, 14]. Please note that the evaluation results described later are taken from our previous work on grammar inference by approximation [15], grammar inference by refinement [12] as well as grammar inference by learning [13, 14]. As the experiment setup used is identical between all previous work, we are able to describe the experiment setup at this point for all the methods presented later. In addition, this enables us to directly compare the test results. Whenever there are some variations in the tested languages among our previous work, we will highlight them accordingly.

We want to show how well our methods are able to approximate a fixed set of test languages in terms of precision and recall (see Section 2.3.1). Individual languages are described in detail in Section 4. In addition, our aim is to demonstrate the practical applicability of our methods, even in scenarios where they exhibit theoretically exponential complexity. To this end, we pursue two main objectives: (1) to empirically evaluate the runtime performance of our algorithms on demanding grammar inference tasks executed on standard hardware, and (2) to quantify the computational effort by tracking relevant metrics such as the number of inference steps, candidate grammar evaluations, or internal algorithmic iterations performed during each experiment. Note that this evaluation is done purely on an empirical basis; however, we will discuss the theoretical complexity of the proposed methods in each chapter after describing the algorithms for both seed grammar extraction and grammar inference.

The advantage of our experiment setup lies in its reusability. With it we can test arbitrary methods for grammar inference for arbitrary languages. Thus, we have defined the following requirements that our experiment setup does fulfill:

1. The results contain both precision and recall of the inferred grammar with regard to the given target grammar, as well as the total number of membership-queries. Here, a membership-query equals exactly one execution of the program p, given some word w to determine if $w \in \mathcal{L}(G_p)$.

2. The experiment setup works as a framework that can handle different types of methods for grammar inference by providing an interface for membership-queries to the inference algorithm.

3. The experiment setup is capable of handling any given context-free language; in other words, the experiment setup is capable of generating an instrumented parser from any given context-free language.

An overview of our experiment setup can be found in Figure 4.1. This setup fulfills all the requirements defined above. Requirement (1) is fulfilled by an in-built component of our grammar-based fuzzer. First, given two context-free grammars, the fuzzer is capable of calculating precision and recall by random sampling, as described in Section 2.3.1. In addition, we count the total number of membership-queries performed by counting the total number of times the program p has been executed with some word w during one test run. To be able to fulfill Requirement (2), we use open-source parser generator tools to automatically generate a program that accepts the given grammar. This enables us to use a full circle in our experiments, where we can be sure that the program under test actually accepts the language we want to learn under the assumption that the tools used work correctly. To ensure that our whole setup and developed methods are not specifically tailored to the parser generator tool used, we have integrated both COCO/R as well as ANTLR in our experiment setup. Optionally, any other parser generator tool could be integrated as well. Finally, we have implemented our proposed methods in a way that they use the grammar-based fuzzer as an interface to execute our target program to fulfill requirement (3). Finally, we have configured the whole setup in a way that every component is directly called by the previous in order to enable full automation, and thus to fulfill Requirement (4).

In summary, each experiment starts with a context-free grammar and proceeds to generate a parser using parser generator tools. Next, a defined method for grammar inference uses the provided grammar-based fuzzer to infer some grammar from the generated parser. Finally, we calculate the precision and recall of the inferred grammar and return the results.

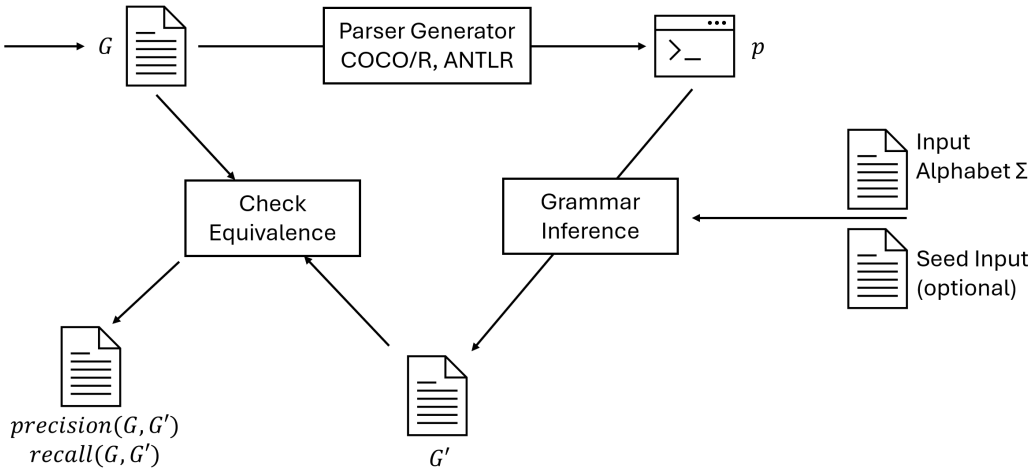


Figure 4.1: Overview on test strategy. We start from a given grammar G, generate a program p and apply a method for grammar inference to p to infer some approximated grammar G'. Finally, we check equivalence of (G, G') with respect to precision and recall.

Please note that if needed, seed inputs are directly provided to the grammar inference method. The input alphabet given to the grammar inference method is automatically taken from the given target grammar by the test framework.

Languages used for Evaluation

This section provides information on the languages used in our experiments. As described above, the grammar that produces each of the languages below is used to automatically generate a recursive-descending parser that accepts exactly this language. Next, we apply our grammar inference methods to approximate the original grammar. Finally, we calculate precision as well as recall of our inferred grammar. The remainder of this section describes each language, what it defines, and why we have included it in our evaluation. In addition, a grammar that produces this language (i.e. the target grammar) is given.

- **MathExpr:** The *MathExpr* language defines a small subset of mathematical expressions. It serves as the standard textbook example in the field of grammar inference, including this thesis as a running example. Although a very simple language, we have decided to include it nonetheless. Part of the language are addition, subtraction, multiplication, as well as division of single-digit numbers. Those numbers are

```
MathExpr → Term [("+"|"-") Term].
    Term → Factor [("*"|"/") Factor].
  Factor → "1" | "2" | "3" | "("MathExpr")".
```

Listing 4.1: MathExpr language grammar

```
  Mail → String "@" String "." Tag.
String → Char { Char }.
   Tag → Char Char [Char].
  Char → "a" .. "z".
```

Listing 4.2: Mail language grammar

limited to one, two, and three. Larger numbers are not considered. However, the *MathExpr* language includes some nested structures that can be built using brackets. Due to this, the *MathExpr* language contains an infinite number of words. Listing 4.1 shows the grammar that produces the *MathExpr* language.

- **Mail:** The *Mail* languages defines the core structure of mail addresses. This includes a prefix as well as a domain separated by "@". While the *Mail* language, in principle, is a regular language, it contains some interesting structures as given by the grammar in Listing 4.2. As we can see, the `Tag` part of this grammar forms a subset of the words produced by the `String` rule. Such subset structures have to be considered while learning the general language. We have to use instrumentation to dynamically determine if the parsers follow the one- or the other rule to correctly determine to which sub-element of the grammar produces some part of a word.

- **Hello:** The *Hello* language is a simple example taken from related work that can reveal the limitations of some methods. It includes all uppercase and lowercase variations of the word "hello". The *Hello* language is a finite, regular language. The grammar used to produce the *Hello* language is given in Listing 4.3. This may seem quite easy to learn at first glance; however, due to a large number of possible combinations we can easily run into some problems regarding algorithms with exponential complexity. The *Hello* language is used in our evaluation to reveal such problems early on. For example, the expanded grammar G_{exp} inferred by the method

```
Hello → ("H" | "h") ("E" | "e") ("L" | "l")
        ("L" | "l") ("O" | "o").
```

Listing 4.3: Hello language grammar

```
AdvMathExpr → Expr.
       Expr → Term { ("+"|"-") Term }.
       Term → Factor { ("*"|"/") Factor}.
     Factor → Num | "(" Expr ")".
        Num → Digit { Digit }.
      Digit → "1" | "2" | "3".
```

Listing 4.4: AdvMathExpr language grammar

described in Section 6.3 has a total of 59049 (=9^5) rules, which is substantially larger than the number of rules in the expanded grammars of other languages.

- **AdvMathExpr:** This language defines a more complex version of the previously described *MathExpr* language. In particular, the *AdvMathExpr* language allows for numbers of unbounded length. This serves to highlight a small step in the direction of more complex languages. We use the *AdvMathExpr* language in our evaluation to show the impact of adding a simple rule to a grammar. As the rest of the language is identical with the *MathExpr* language, such a direct comparison is feasible. The grammar used to produce the *AdvMathExpr* language is given in Listing 4.4.

- **Json:** The *JSON* language[1] used here follows the official language definition. The exact grammar we used in our experiments is given in Listing 4.5. *JSON* language is a widely known and extensively used data format, with multiple variations depending on the specific variant in use. What differentiates this language from the previous, apart from a significantly larger number of production rules, lies mainly in the usage of the empty symbol ϵ in some of its production rules. Usually, this introduces additional challenges both in parsing and in learning the language that we address in our evaluation.

[1] https://www.json.org/json-en.html

```
      Json → Element.
   Element → Ws Value Ws.
     Value → Object|Array|String|Number
             |"true"|"false"|"null".

    Array  → "[" Ws [Value Ws [ "," Elements ]] "]".
  Elements → Element [ "," Elements ].

    Object → "{" Ws [String Ws ":" Element["," Members]] "}".
   Members → Member ["," Members].
    Member → Ws String Ws ":" Element.

    String → "'" Characters "'".
Characters → ϵ|Character Characters.
 Character → "a"|"b"|"c".
        Ws → " ".

    Number → Integer Fraction Exponent.
  Fraction → ϵ|"." Digits.
  Exponent → ϵ|"E" Sign Digits|"e" Sign Digits.
      Sign → ϵ|"+"|"-".

   Integer → ["-"] ("0"|Onenine [Digits]).
    Digits → Digit [Digits].
     Digit → "0"|Onenine.
   Onenine → "1"|"2"|"3".
```

Listing 4.5: Json language grammar

- **URL:** The *URL* language used here defines a subset of the original URL[2] language. Our version includes HTTP-, FTP-, TELNET- as well as GOPHER-addresses. The exact grammar used in our evaluation is given in Listing 4.6. The *URL* language splits into different paths early on, which can pose as an additional challenge for the inference algorithm. For example, while providing a seed word for HTTP-addresses

[2] https://www.ietf.org/rfc/rfc1738.txt

enables us to infer a grammar that perfectly produces HTTP-addresses, such a grammar will not be able to produce any valid FTP-address. To solve this challenge, we either have to provide several seed inputs or apply one of our proposed methods for fuzzing-based grammar inference that works independently of any given seed input, like the ones described in Sections 5.2 and 6.2.

```
          URL → HTTPaddress | FTPaddress | TELNETaddress
                | GOPHERaddress.
  HTTPaddress → "h" "t" "t" "p" ":" "/" "/" Hostport
                [ "/" path ] [ "?" search ].
   FTPaddress → "f" "t" "p" ":" "/" "/" Login "/" path.
TELNETaddress → "t" "e" "l" "n" "e" "t" ":" "/" "/" Login.
GOPHERaddress → "g" "o" "p" "h" "e" "r" ":" "/" "/" Hostport
                ["/" Gtype [ Selector ] ] [ "?" search ].

        Login → user [ ":" password ] "@" Hostport.
     Hostport → Host [ ":" Port ].
         Host → Hostname | Hostnumber.
     Hostname → IAlpha [ "." Hostname ].
   Hostnumber → Digits "." Digits "." Digits "." Digits.
         Port → Digits.

     Selector → Path.
         Path → XAlphas [ "/" Path ].
       Search → XAlphas [ "+" Search ].
        Gtype → XAlpha.

         User → XAlphas.
     Password → XAlphas.

       XAlpha → Alpha | Digit | Safe | Extra.
      XAlphas → XAlpha [ XAlphas ].
       IAlpha → Alpha [ XAlphas ].
        Alpha → "a" | "b" | "c" | "A" | "B" | "C".

        Digit → "0" | "1" | "2" | "3".
         Safe → "$" | "-" | "_" | "&".
        Extra → "!" | "*" | "(" | ")" | ";" | ",".
       Digits → Digit [ Digits ].
```

Listing 4.6: URL language grammar

5 Fuzzing-based Grammar Inference by Approximation

In this chapter, we will discuss the overall process of *Grammar Inference by Approximation*. The content presented in this chapter is based on our previously published, peer-reviewed work on *GrammarForge*, as detailed in [15]. The goal of this process is to infer a grammar G'_p for a given program p such that $\mathcal{L}(G'_p)$ approximates as close as possible (in terms of precision and recall) $validInputs(p)$. We denote as G_p the actual target grammar, that is, the grammar that defines the input language of p. We start by defining the type of seed grammar on which the grammar inference part is based. We call this seed grammar a core grammar G_{core} because it basically describes what the target grammar we want to infer will look like on a fundamental level. This means that we only gain information on how the production rules of such a grammar are structured, without any details on which terminal symbols are used where.

The remainder of this chapter is structured as follows. We begin by formally defining the ideal core grammar G_{core}. Based on this definition, we then explain how our approximation-based algorithm can be applied, starting from a core grammar G_{core}, to infer a grammar G'_p that closely approximates the target language $\mathcal{L}(G_p)$. Before introducing core grammar extraction, we first describe our approach for identifying and learning tokens and terminals from source code. We will end the chapter with an evaluation of this approach.

5.1 Core Grammar

Let us first give the definition for the ideal core grammar. Not that this definition assumes that G_p is known, which is **not** the case in practice. The actual core grammar extracted during seed grammar extraction aims to approximate this definition as closely as possible.

A core grammar uses the production rules of G_p to produce a language that contains placeholder terminals instead of the original terminals. We define $G_{core} = (N, \Sigma_{core}, P_{core}, S)$ with $\Sigma_{core} = \{\bigcirc_0, ..., \bigcirc_n\}$ for $n = |\Sigma|$ and $P_{core} = \{a \rightarrow \alpha' : a \rightarrow \alpha \wedge \alpha' = \mathcal{F}(\alpha)\}$ where $\mathcal{F}(\alpha)$ replaces every terminal $t_i \in \Sigma$ in α with its corresponding placeholder terminal $\bigcirc_i \in \Sigma_{core}$. Note that $\mathcal{L}(G_{core}) \subseteq \{\bigcirc_0, ..., \bigcirc_n\}^*$.

We have identified two ways in which we can extract such a core grammar from the source code of p: As described in Section 5.3.2 and Section 5.3.3 we can either use AST analysis or control flow analysis, respectively. Once we obtain a core grammar, we can exploit it to infer G'_p by replacing placeholders by appropriate terminals from Σ. We propose in Section 5.2 an algorithm for this precise task.

5.2 Approximation Algorithm

This section explains how a core grammar can be used to approximate the language accepted by a program p. Before presenting our grammar inference algorithm based on approximation, we begin with a brief recap of the properties of a core grammar G_{core}. Section 5.1 defines a core grammar as

$$G_{core} = (N, \Sigma_{core}, P_{core}, S)$$

where the set of non-terminals N and the start symbol S are identical to those in G_p. The terminal alphabet Σ_{core} consists of placeholder symbols $\bigcirc_i$, and the production rules P_{core} mirror those of P, replacing concrete terminals with placeholders. As discussed above, extracting the exact core grammar is generally infeasible in practice. Instead, the actual core grammar only approximates these idealized properties.

We need to transform the obtained core grammar G_{core} into a proper grammar G'_p for the input language of the program p. The aim is to replace every placeholder $\bigcirc_i$ in G_{core} with corresponding terminal symbols t_j taken from the alphabet Σ of p, such that the language $\mathcal{L}(G'_p)$ of the resulting grammar G'_p approximates $validInputs(p)$ with good precision and recall.

In general terms, the idea is to repeatedly apply the following steps until all place holders in P_{core} have been replaced by terminals in Σ. The resulting grammar is denoted as G'_p.

1. Generate a $w \in \mathcal{L}(G_{core})$ using a random walk through the productions of G_{core}.
2. Collect in a set T_w all the different place holders for terminals that appear in w.
3. For each $\bigcirc_i \in T_w$, choose a terminal $t_i \in \Sigma$.
4. Replace every place holder $\bigcirc_i$ that appears in w by its corresponding terminal t_i (as selected in the previous step), obtaining $w' \in \Sigma^*$.
5. If $w' \notin validInputs(p)$, return to step (3) above, choosing a different combination of terminal symbols.
6. For every rule in P_{core}, replace every place holder $\bigcirc_i \in T_w$ by its corresponding terminal t_i

Of course, we cannot directly apply this broad strategy to learn G'_p, as it would require in the worst case to examine $\mathcal{O}(|\Sigma|^{|\Sigma_{struc}|})$ possible combinations. In order to make this approach tractable, we refine step (1) above as follows:

1.1 Fix a threshold x, and set $n = 0$ and $w = \epsilon$.

1.2 Generate a word $u \in \mathcal{L}(G_{core})$ using a random walk through the productions of G_{core}.

1.3 If $|T_u| < |T_w|$ or $T_w = \emptyset$, then assign u to w.

1.4 If $|T_w| > 2$ and $n < x$, then add 1 to n and go to (1.2).

This modification means that we start by resolving a minimal number of place holders and then incrementally solve the remaining ones. If the iteration threshold is chosen right (in our experiments x was set to 1000), then the possible combinations of terminals that one needs to examine are of order quadratic on $|\Sigma|$.

5.2.1 Need for solution classes

In the strategy described above, the different choices for the place holders in step (3) frequently lead to the extraction of different sets of derivation rules from the grammar structure G_{core}. Although individually each of these sets of rules defines grammars that can only derive words that belong to $validInputs(p)$, it is not necessarily the case that all of them are good candidates for the inference of the target language. Let us illustrate this

$\texttt{S} \rightarrow \texttt{A}$
$\texttt{A} \rightarrow \texttt{B} \mid (\ \texttt{B}\ )$
$\texttt{B} \rightarrow 1 \mid 2$

(a) Target Grammar

$\texttt{S} \rightarrow \bigcirc_0\ \texttt{A}$
$\texttt{A} \rightarrow \bigcirc_4\ \texttt{B} \mid \bigcirc_1\ \texttt{B}\ \bigcirc_2$
$\texttt{B} \rightarrow \bigcirc_3$

(b) Core Grammar Structure

Figure 5.1: Core Grammar with Conflicting Place Holders Resolutions

point with a simple example. Assume that the target grammar, that is, the one used by the parser, is the one in Figure 5.1a. Let the core grammar obtained from the parser be as in Figure 5.1b.

Assume that we want to find a solution for the word $\bigcirc_0\ \bigcirc_1\ \bigcirc_3 \bigcirc_2$ produced by the core grammar. Clearly, we could choose to replace the place holders in two different ways, where both replacements give grammar rules that produce the word "(1)" belonging to the language of the target grammar.

1. $\bigcirc_0 = (, \bigcirc_1 = \epsilon, \bigcirc_3 = 1, \bigcirc_2 =)$
2. $\bigcirc_0 = \epsilon, \bigcirc_1 = (, \bigcirc_3 = 1, \bigcirc_2 =)$

Moreover, these two alternative sets of rules will give us grammar with prefect precision 1. However, following (i), we would get a refined core grammar with the following rules: $\texttt{S} \rightarrow (\ \texttt{A}$, $\texttt{A} \rightarrow \bigcirc_4\ \texttt{B} \mid \texttt{B}\ )$, and $\texttt{B} \rightarrow 1$. This core grammar can now produce "$(\bigcirc_4 1$", which cannot be refined to a proper grammar since any replacement of $\bigcirc_4 1$ leads to a word that is not in the target language. Therefore, in principle, we would need to explore all these possible alternative resolutions for the place holders via a trial-and-error approach. However, this can quickly lead to an explosion of cases and make the problem unfeasible in practice.

In order to mitigate this explosion of cases, we follow a heuristic approach consisting of grouping the partial grammars obtained by different potential resolutions of a list of place holders into classes formed by those that have similar properties. Then we only take a representative grammar for each resulting class, effectively reducing the number of cases that need to be explored into a tractable size. Two different grammars G_1 and G_2 (or core grammars depending on whether there are remaining place holders in the rules or not) obtained after replacing a given list of place holders by different terminals

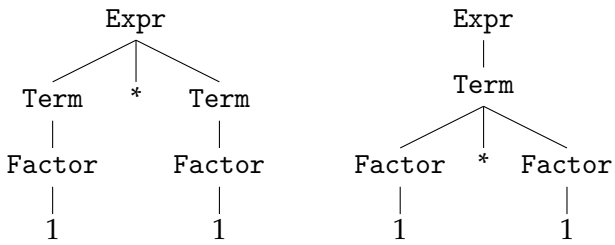
are considered to belong to the same class if and only if the precision of their union is 1. That is, the grammar obtained by taking the union of G_1 and G_2 only produces words that are recognized by the parser.

5.2.2 Forking the search process

The previous process gives a series of grammar classes after each iteration in which the new placeholders are resolved. Let us say $C_0, \ldots, C_n$. From each of these classes C_i we choose (arbitrarily) a representative grammar G_i. We insert these grammars into a queue, where the grammar structures are ordered according to the number of place holders in their respective rules, the least number of place holders being the first in this queue. At this point, we can use parallel processes (forks) to resolve the place holders in every remaining grammar structure. Each parallel process will apply the same procedure as explained before, but now to a different grammar G_i taken from the queue. The forked process with input G_i will either (a) discard G_i, (b) produce one or more classes $C_{i0}, \ldots, C_{im}$ of candidate grammar structures, or (c) produce one or more final candidate grammars without place holders in their rules. The first case arises when a substitution of (some) remaining place holders results in a grammar that produces words that are not recognized by the parser. If case (a) is not verified, then there are one or more possible substitutions of place holders, each leading to grammars that only produce words accepted by the parser (case (c)), or to grammar structures that have this same property (case (b)). If case (b) is verified, then a representative grammar G_{ij} is chosen from each class C_{ij} and inserted (in the order determined by the number of placeholders in the rules) into the queue. The process continues until a case c has either recall 1 or the queue is emptied. At this point, the candidate grammar with the highest recall is selected as a result.

5.2.3 Need for Instrumentation

We apply instrumentation to check whether the derivation trees (produced by successive refined grammars) of words accepted by p match the actual paths the program takes in its call tree while parsing these inputs. Assume that we have the word "1*1". We can produce this by taking two potentially ambiguous paths while expanding our core grammar structure:



If we do not check which of these derivation trees actually matches the call tree of p and take both as valid, then we would end up with the following productions.

$$\begin{aligned}
\texttt{Expr} &\rightarrow \texttt{Term}\ [\ (+|-|*)\ \texttt{Term}\ ] \\
\texttt{Term} &\rightarrow \texttt{Factor}\ [\ (*|/)\ \texttt{Factor}\ ] \\
\texttt{Factor} &\rightarrow 1 \mid 2 \mid 3 \mid (\ \texttt{Expr}\ )
\end{aligned}$$

Thus, we could produce words such as "1*1*1" which do not belong to the input language of p. As the search process will be forked when encountering such ambiguous paths, we will eventually discard the faulty branch. However, using instrumentation enables us to discard such incorrect branches early on, which significantly decreases the total number of paths that need to be explored.

5.2.4 Example

Let G_{core} be the core grammar extracted from the ExprParser example in Section 5.3.3. Assume that the random walk from step (1) described in the beginning produces the word: $w = \bigcirc_5 \bigcirc_{12} \bigcirc_4 \bigcirc_1 \bigcirc_{12} \bigcirc_5 \bigcirc_{12} \bigcirc_3 \bigcirc_{12} \bigcirc_5 \bigcirc_{12} \bigcirc_0$.

We thus obtain $T_u = \{\bigcirc_0 \bigcirc_1, \bigcirc_3, \bigcirc_4 \bigcirc_5 \bigcirc_{12}\}$. Since $|T_u| > 2$, we proceed to generate the next word. We repeat this until we find that the shortest word we are able to produce is $w = \bigcirc_5 \bigcirc_{12} \bigcirc_4 \bigcirc_2 \bigcirc_0$ (recall the refined steps (1.1–1.4) of step (1) explained in the third paragraph of Section 5.2). Via random testing of different substitutions of terminal symbols for the place holders, the algorithm discovers that many of the different alternative substitutions result in derivation rules that produce words accepted by the parser. Figure 5.2 shows the syntax tree of w as well as the resulting parse trees obtained

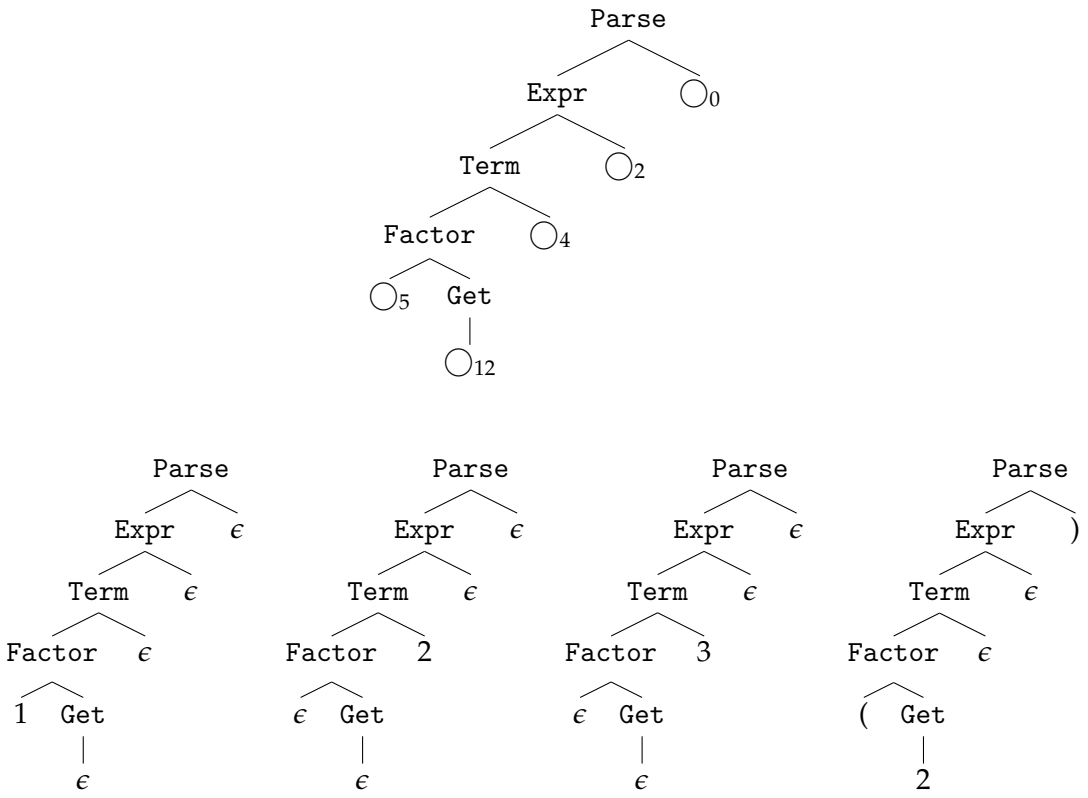


Figure 5.2: Derivation tree of word "$\bigcirc_5 \bigcirc_{12} \bigcirc_4 \bigcirc_2 \bigcirc_0$" using G_{core} and corresponding derivation trees of words "1", "2", "3" and "(2)" accepted by the parser (and obtained from the grammars resulting from substituting place holders in G_{core}).

by substitution of place holders by symbols that result in different words $w_0 =$ "1", $w_1 =$ "2", $w_2 =$ "3" and $w_3 =$ "(2)" accepted by the parser.

Applying the instrumentation strategy described above, our algorithm discards w_3, since its derivation tree does not coincide with the call tree resulting from an instrumented execution of the parser. As the trees for w_0, w_1 and w_2 pass this step, we now apply the strategy to divide them into disjoint classes. Let G_{w_0}, G_{w_1} and G_{w_2} be the grammars corresponding to the substitution of place $\bigcirc_5 \bigcirc_{12} \bigcirc_4 \bigcirc_2 \bigcirc_0$ in the rules of G_{core} by the corresponding symbols in the derivation tress of w_0, w_1 and w_2 shown in Figure 5.2, respectively. We can easily see that G_{w_1} and G_{w_2} belong to a same class, while G_{w_0} belongs to a different class. Indeed, the union of G_{w_0} with G_{w_1} results in a grammar that can derive the word "12" which is not accepted by the parser. Likewise, the union of G_{w_0} with G_{w_2} can derive the word "13" which is again not accepted by the parser.

$$
\begin{aligned}
\texttt{Parse} &\rightarrow \texttt{Expr}\ \epsilon \\
\texttt{Expr} &\rightarrow \texttt{Term}\ (\ \bigcirc_1\ \texttt{Get}\ \texttt{Term} \mid \epsilon\ ) \\
\texttt{Term} &\rightarrow \texttt{Factor}\ (\ \bigcirc_3\ \texttt{Get}\ \texttt{Factor} \mid \epsilon\ ) \\
\texttt{Factor} &\rightarrow 1\ \texttt{Get} \mid (\ \bigcirc_6\ \texttt{Get} \mid (\ \bigcirc_7\ \texttt{Get} \mid (\ \bigcirc_8\ (\ \bigcirc_9 \mid \bigcirc_{10}\ \texttt{Get}\ \texttt{Expr}\ \bigcirc_{11}\ )\ )\ )\ ) \\
\texttt{Get} &\rightarrow \epsilon \mid \bigcirc_{13}\ \texttt{Get}
\end{aligned}
$$

$$
\begin{aligned}
\texttt{Parse} &\rightarrow \texttt{Expr}\ \epsilon \\
\texttt{Expr} &\rightarrow \texttt{Term}\ (\ \bigcirc_1\ \texttt{Get}\ \texttt{Term} \mid \epsilon\ ) \\
\texttt{Term} &\rightarrow \texttt{Factor}\ (\ \bigcirc_3\ \texttt{Get}\ \texttt{Factor} \mid 2\ ) \\
\texttt{Factor} &\rightarrow \epsilon\ \texttt{Get} \mid (\ \bigcirc_6\ \texttt{Get} \mid (\ \bigcirc_7\ \texttt{Get} \mid (\ \bigcirc_8\ (\ \bigcirc_9 \mid \bigcirc_{10}\ \texttt{Get}\ \texttt{Expr}\ \bigcirc_{11}\ )\ )\ )\ ) \\
\texttt{Get} &\rightarrow \epsilon \mid \bigcirc_{13}\ \texttt{Get}
\end{aligned}
$$

Figure 5.3: Rules of the intermediate grammar structures G_{w_0} and G_{w_1}, respectively.

As a result, the algorithm produces a fork in the search process and explores further independent resolutions of the remaining place holders in the grammars G_{w_0} and G_{w_1} (assuming w.l.o.g. that they are the chosen representative grammars of their respective classes). The rules of these two grammar structures are shown in Figure 5.3. After a number of further iterations, our algorithm will be able to refine G_{w_0} into a correct grammar for the target language. It will also find that further refinement of G_{w_1} does not lead to good candidate grammars.

5.2.5 Complexity Analysis

We assume that using a recursive-descending parsing implementation, the program p is able to determine if some input u of length n is well-formed or not in $\mathcal{O}(n)$. Finding a short word $u \in \mathcal{L}(G_{core})$ as described in Step 1 adds constant time, since we limit the steps taken to a fixed threshold. To find all possible solutions for replacing placeholder symbols in T_u we have to examine $|\Sigma|^{|T_u|}$ words at worst. Determining if such a possible solution is accepted by p can only be done by executing p given the possible solution. This leaves us with a runtime complexity of $\mathcal{O}(n \cdot |\Sigma|^m)$ where m describes the largest set T_u observed. In our experiments, m remained below five. Finally, at worst, each possible solution leads to a branch that has to be fully observed. Assuming that we need x steps to resolve all placeholders, this leaves us with a total runtime complexity of $\mathcal{O}\Big((n \cdot |\Sigma|^m)^x\Big)$.

Note that in practice many branches are discarded early on because of instrumentation. In addition, the remaining solutions are summarized into solution classes, which reduces the total number of branches further. Furthermore, all branches, except the correct one, will be determined to be incorrect and thus will be discarded before having to completely explore them. In this analysis, the possibility of executing branches in parallel has not been considered.

5.3 Core Grammar Extraction

The objective of core grammar extraction is to infer a grammar G_{core} that aims to approximate the definition given in Section 5.1 as closely as possible. We have proposed two different methods based on the analysis of an Abstract Syntax Tree (AST) and Control Flow Graph (CFG) in [12, 14] and [15], respectively. In both cases, we extract information about function calls from the source code of p. We assume that each function corresponds to a production rule in the core grammar G_{core}. For each function, we add a new production rule with the functions identifier as the left-hand side, and the identifiers of functions it calls as non-terminals on the right-hand side. A heuristic is applied to determine whether terminals may be consumed between these non-terminals. Using function identifiers as non-terminal symbols improves the readability of the resulting grammar for human users, though this benefit depends on the parser employing meaningful function names.

5.3.1 Learning Terminals and Tokens

If the input alphabet is not known, we assume that we have good methods to identify terminals as well as tokens used by the target parser. Here we will discuss some methods that can be applied to achieve this. Terminal symbols are the basic, indivisible symbols of the grammar, often corresponding to individual characters or literals, while tokens are higher-level abstractions that group terminals into meaningful categories, such as identifiers, keywords, or operators. First, we identify a set of terminal symbols. Next, we summarize the terminals found into tokens.

Learning Terminal symbols:

There exist several methods that can be applied to learn the terminal symbols, or in other words, the input alphabet of some program p. This input alphabet is equal to the set of terminals Σ of G_p.

We assume that we have access to the source code of the target program. This enables us to apply strong heuristics to approximate the input alphabet. For example, we are able to search for all character or string literals found in the source code and add all identified symbols to Σ. Doing so will most likely result in a superset of the actual input alphabet. This does not pose a problem for our proposed algorithms. However, missing some characters will be problematic. For example, some terminal symbols may be used indirectly. An integer that is interpreted as a *char* at runtime will not show up as a character literal and, as such, will be missing in Σ. To solve this problem, we could try to apply more advanced static analysis that is capable of handling such corner-cases correctly. Another method would be to grossly overestimate the input alphabet and simply put every *ascii character* into it. However, if we use a superset of Σ, then this leads to an increase in runtime of our proposed algorithms. On the other hand, using a subset of Σ will decrease runtime at the cost of not being able to correctly infer G_p.

Identification of potential token positions:

To decrease runtime of our proposed method, it makes sense to summarize terminal symbols into tokens whenever possible. To do so, we follow a naive approach. Given a set of known words $S \subseteq \mathcal{L}(G_p)$ as well as some input alphabet Σ, we are able to produce a set of valid tokens for $\mathcal{L}(G_p)$ from a black-box. We start by extracting potential tokens from each word $s \in S$. For every $a \in \Sigma$ that is a substring of s we define a potential set of tokens $T_{as} = \{a\}$ for the terminal a of the word s. Next, we produce a new set of words by replacing a with every other $a' \in \Sigma$. We proceed to execute p given those new inputs. If one is accepted by the parser, we add a' to the potential token set T_{as}. This operation results in a set of potential tokens for G_p. This set is further refined by checking whether one token set T_{as} is a subset of another token set$T_{a's'}$. If so, we remove $T_{a's'}$, adding $T_{a''s''} = T_{a's'} \setminus T_{as}$ instead. Now, we are left with a set of potential tokens that are interchangeable in the context of the given set of known words S.

Let us take a look at the running example given in Section 2.4. Given the source code defined in Listing 2.2, alphabet $\Sigma = \{1, 2, 3, +, -, /, *, (,)\}$, and $S = \{1 + 1/1, 1 + 1, (1)\}$, we now proceed to extract the set of potential tokens. This leaves us with a new alphabet $\Sigma' = \{[1, 2, 3], [+, -], [/, *], (,)\}$ and effectively reduced the size of our input alphabet from nine to five terminal variables. A decreased total number of terminals in our alphabet leads to a significant improvement in terms of runtime of our proposed algorithms for seed grammar extraction and grammar inference.

5.3.2 AST analysis based core grammar extraction

The following analysis is based on the extraction of an AST model $T(s_p)$ from the source code of p. For this purpose, we use the eKnows tool [26]. Note that any other tool that can extract a similar AST can be used instead. That is, our algorithm does not depend on the eKnows tool. We extract a core grammar G_{core} from a program p by following two basic steps:

1. Extract the basic structure of G_p from the AST model $T(s_p)$.
2. Heuristically identify potential token positions.

We start by identifying the set of all functions F present in $T(s_p)$. For every $f \in F$, we define a non-terminal for G_{core}. Next, we identify which functions are called by each f within its scope, sequentially. Based on this information, we infer the initial rules of G_{core} following the heuristics:

- For every function $f \in F$, add a production rule $f \rightarrow \epsilon$.
- For every distinct control flow in $f \in F$ that calls some other functions $g, ..., h$ sequentially, add a production rule $f \rightarrow g, ..., h$.

Next, we need to determine where the terminal symbols are consumed in the parser and add placeholders accordingly to the initial production rules. In contrast to identifying function calls within the AST model $T(s_p)$, determining where terminals can be consumed is not trivial due to the various ways in which this could be done.

To make sure we do not miss potentially consumed terminals, we simply assume that any element in the AST that is not a function call potentially consumes a terminal. As such,

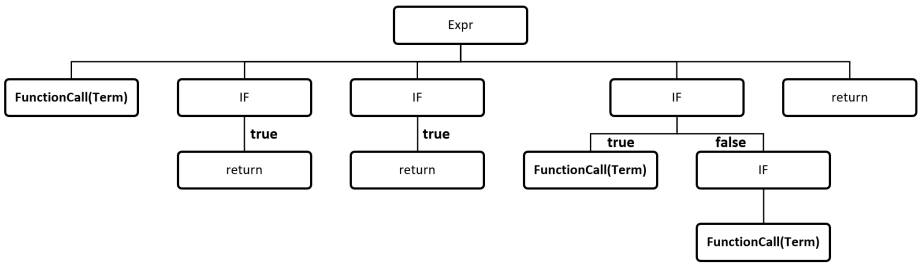


Figure 5.4: AST of method `Expr` in Java code in Listing 2.2

we add a placeholder before and after every non-terminal in the production rule. The only case where we can confidentially determine that no terminal is present is when there is no element in the AST. For example, if two functions are called directly after each other, we are sure that no terminal has been consumed in between. The same logic applies to the beginning and end of each rule if a function call is the first or last code statement of the function, respectively.

Let us take a look at our running example defined in Section 2.4. We start by extracting the AST model $T(s_p)$. Figure 5.4 shows a simplified version of the AST model for the function `Expr`. Based on the AST for function `Expr` we proceed to infer the following rules:

$$\texttt{expr} \rightarrow \texttt{term}$$

$$\texttt{expr} \rightarrow \texttt{term}\ \texttt{term}$$

Next, we infer potential positions for consumed terminals and add new placeholders $\bigcirc_i$. As the first statement in the AST model of function `Expr` is a call to function `Term`, we are sure that no terminal is consumed there. Contrary, due to some elements between the first and second calls to `Term`, we assume that a terminal potentially is consumed there and add a new placeholder $\bigcirc_i$ in between. Finally, as the optional second call to `Term` is the last statement, we do not need to add anything here. This results in the following rule for the function `Expr` for G_{core}:

$$\texttt{expr} \rightarrow \texttt{term} \bigcirc_0 \mid \texttt{term} \bigcirc_1 \texttt{term}$$

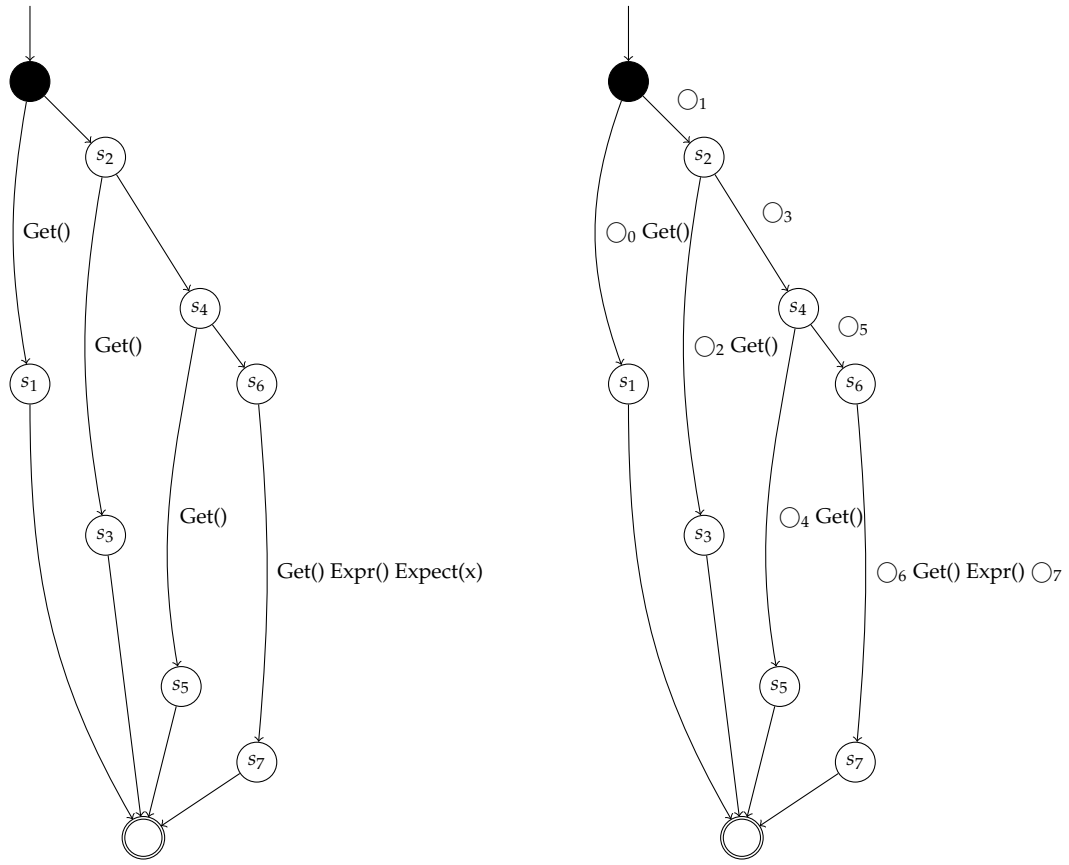
We proceed to apply the process given for function `Expr` to all other functions present in $T(s_p)$ in our running example. This gives us the following G_{core} extracted from the AST model of the program p defined in Listing 2.2:

$$\begin{aligned}
\texttt{parse} &\rightarrow \texttt{expr} \bigcirc_0 \\
\texttt{expr} &\rightarrow \texttt{term} \bigcirc_1 \mid \texttt{term} \bigcirc_2 \texttt{term} \\
\texttt{term} &\rightarrow \texttt{factor} \bigcirc_3 \mid \texttt{factor} \bigcirc_4 \texttt{factor} \\
\texttt{factor} &\rightarrow \bigcirc_5 \texttt{expr} \bigcirc_6 \mid \bigcirc_7
\end{aligned}$$

5.3.3 CFG analysis based core grammar extraction

In the following, we discuss how we can use the CFG of the source code of p to extract a suitable core grammar. Introduced by Allen [27] in 1970, the usage of CFGs in the field of program analysis is very well known and researched. A CFG represents all the potential paths that can be taken while traversing a procedure of a program when executed. Each sequential piece of code without jumps or jump targets is represented by a node in a digraph. Each jump is represented by two outgoing lines from a node that lead to the next, different jump targets.

We first discussed the idea of utilizing a CFG to extract a suitable core grammar in [14]. The process was further refined in [15]. In both cases, the eKnows tool [26] was used to extract a CFG from a given program p. As a first step, we extract a $CFG(p.a)$ for each procedure a, that is, the method or function of p. $CFG(p.a)$ contains some nodes S_i as well as edges between them. Next, we assign each jump to another procedure f with optional parameters $\bar{x}$ outside the context of a to the corresponding edge of $CFG(p.a)$ as a label $l_i = f(\bar{x})$ for each edge between two nodes (S_i, S_j). If there exist more than one jump or call to another procedure, we assign them in a sequential order. Taking the



(a) Control Flow of CoCo/R parser

(b) Refined Control Flow

$$\texttt{Factor} \rightarrow \bigcirc_0\ \texttt{Get} \mid \bigcirc_1\ (\bigcirc_2\ \texttt{Get} \mid \bigcirc_3\ (\bigcirc_4\ \texttt{Get} \mid \bigcirc_5\ (\bigcirc_6\ \texttt{Get Expr}\ \bigcirc_7)))$$

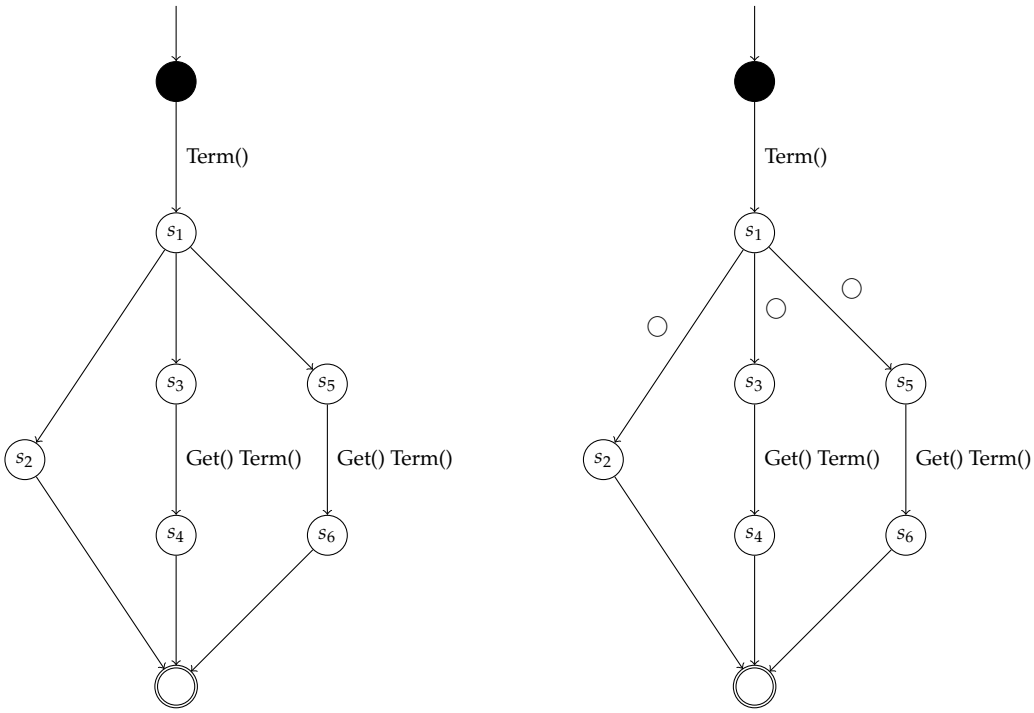
(c) Grammar Rule from Refined Control Flow

Figure 5.5: Control Flow Graph Refinement to Core Grammar of Method *Factor*

running example described in Section 2.4, we show the result of such an operation for the procedure `Factor` in Figure 5.5a and in Figure 5.6a for the procedure `Expr`.

Now we need to refine the extracted $CFG(p.a)$ to get a core grammar G_{core}. For this, we first refine the $CFG(p.a)$ by adding new labels $\bigcirc_i$ wherever we might expect terminal symbols to be consumed during execution. Thus, we follow the following heuristic:

1. Identify each node S_j with two or more outgoing edges. As the split in the control flow potentially occurs due to a consumed terminal, we add a $\bigcirc_i$ to the label of each outgoing edge at the beginning.



(a) Control Flow of CoCo/R parser

(b) Refined Control Flow

$$\texttt{Expr} \rightarrow \texttt{Term}\ (\ \bigcirc_1\ \texttt{Get Term} \mid \bigcirc_2\ )$$

(c) Grammar Rule from Refined Control Flow

Figure 5.6: Control Flow Graph Refinement of Method *Expr*

2. Identify each label $f(\bar{x})$ and replace it with a $\bigcirc_i$. This simplification assumes that each procedure that receives some parameter $\bar{x}$ potentially checks if a given terminal is present at this point.

The result of applying the heuristics described above for procedures `Factor` and `Expr` are given in Figure 5.5b and Figure 5.6b, respectively.

Finally, let $G_{core} = (N_{core}, \Sigma_{core}, P_{core}, S_{core})$. The set of non-terminals N_{core} is the set of all procedures present in p, that is, we treat each function in p as a non-terminal in G_{core}. Consequently, we define the start symbol S_{core} as the main procedure of p. Next, the set of terminal symbols Σ_{core} is formed by all placeholder symbols $\bigcirc_i$ present in all refined $CFG(p.a)$. Finally, we extract the set of production rules P_{core} from the refined $CFG(p.a)$. For every path in each $CFG(p.a)$ from the root to the leaf, we add a rule $a \rightarrow l_0 \ldots l_n$

$$
\begin{aligned}
\texttt{Parse} &\rightarrow \texttt{Expr}\ \bigcirc_0 \\
\texttt{Expr} &\rightarrow \texttt{Term}\ (\ \bigcirc_1\ \texttt{Get}\ \texttt{Term} \mid \bigcirc_2\ ) \\
\texttt{Term} &\rightarrow \texttt{Factor}\ (\ \bigcirc_3\ \texttt{Get}\ \texttt{Factor} \mid \bigcirc_4\ ) \\
\texttt{Factor} &\rightarrow \bigcirc_5\ \texttt{Get} \mid (\ \bigcirc_6\ \texttt{Get} \mid (\ \bigcirc_7\ \texttt{Get} \mid (\ \bigcirc_8\ (\ \bigcirc_9 \mid \bigcirc_{10}\ \texttt{Get}\ \texttt{Expr}\ \bigcirc_{11}\ )\ )\ )\ ) \\
\texttt{Get} &\rightarrow \bigcirc_{12} \mid \bigcirc_{13}\ \texttt{Get}
\end{aligned}
$$

Figure 5.7: Resulting core grammar from control flow analysis.

where $l_0 \ldots l_n$ is the concatenation of all labels along the path. Furthermore, two or more consecutive $\bigcirc_i \ldots \bigcirc_j$ are summarized in a single $\bigcirc_i$ as we assume that such a sequence can be represented by a single token. The production rules built from procedures `Factor` and `Expr` are given in Figure 5.5c and Figure 5.6c, respectively. Finally, in the case of our running example, this results in the core grammar G_{core} given in Figure 5.7.

5.4 Evaluation & Results

Our method for grammar inference by approximation makes use of a core grammar structure (see Section 5.1) to approximate the target language. Each step adds some new terminals to the core grammar until it fully recognizes the target language. During the experiments, we calculate precision and recall by random sampling of 1 000 words for each instance. The graph in Figure 5.8 show the obtained values of recall obtained for various target grammars. Note that we do not need to show precision as this is always one. Since our method starts from a core grammar with recall $= 0$ and precision $= 1$, recall is increased periodically during the learning process. Intermediate grammar structures which bring precision below 1 are directly discarded as possible solutions. The curves in Figure 5.8 correspond to the evolving (class representative) grammars in the branch that led to the best solution in each case. Recall that the learning process can produce many different forks depending on the possible grammar classes determined by the algorithm.

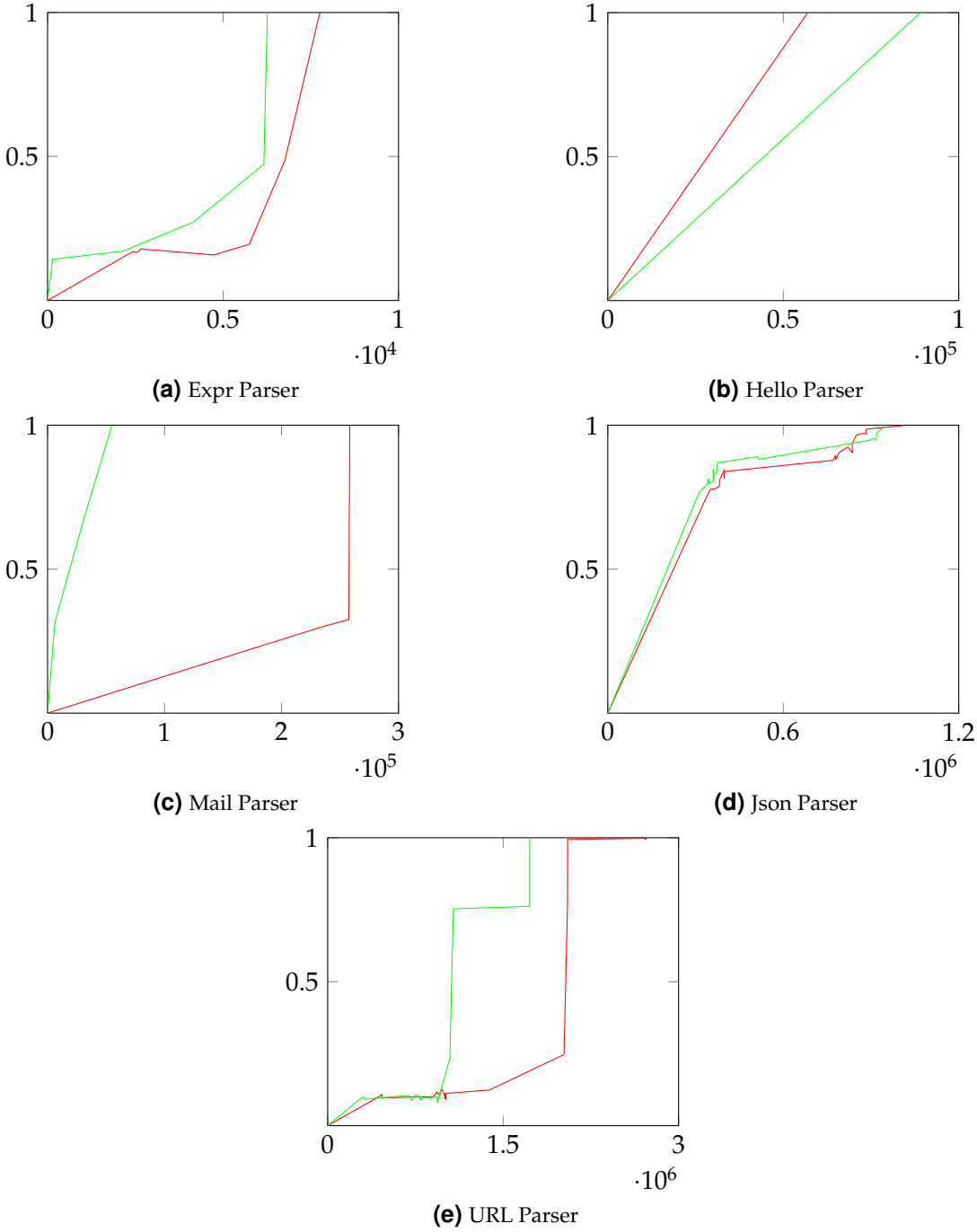
Figure 5.9 provides a visualization in the form of a tree of the different forks produced by the algorithm while learning the grammar of the JSON language. The root node of the tree represents the grammar resulting from the core grammar extraction described in Section 5.3.3. Each subsequent node depicts a grammar (structure) resulting from the

Target	Precision	Recall	Queries (COCO/R)	Queries (ANTLR)
ExprParser	1.0	1.0	7 760	6 263
MailParser	1.0	1.0	258 354	55 019
HelloParser	1.0	1.0	56 979	89 098
JsonParser	1.0	1.0	1 011 157	940 456
URLParser	1.0	1.0	2 719 994	1 741 088

Table 5.1: Experiment results showing precision and recall for tested languages.

search process described in Section 5.2. Yellow leaves denote branches that have been discarded due to low precision (that is, where no suitable substitution for the remaining place holders was found). The green leaf denotes the grammar learned by the algorithm, which in this case has both recall and precision equal to 1.

In our experiments we have demonstrated that *Grammar Inference by Approximation* works in practice and provides a significant improvement by removing the dependence on a set of seed inputs. Our experiments show that we have been able to remove the dependence on prior knowledge about the target language while achieving both perfect recall and precision with the inferred grammars, including real-world examples like JSON and URL.



(a) Expr Parser **(b)** Hello Parser **(c)** Mail Parser **(d)** Json Parser **(e)** URL Parser

Figure 5.8: Evolution of recall along the learning process using the learning based method (Red = CoCo/R, Green = ANTLR). The x-axis shows the amount of queries while the y-axis shows recall.

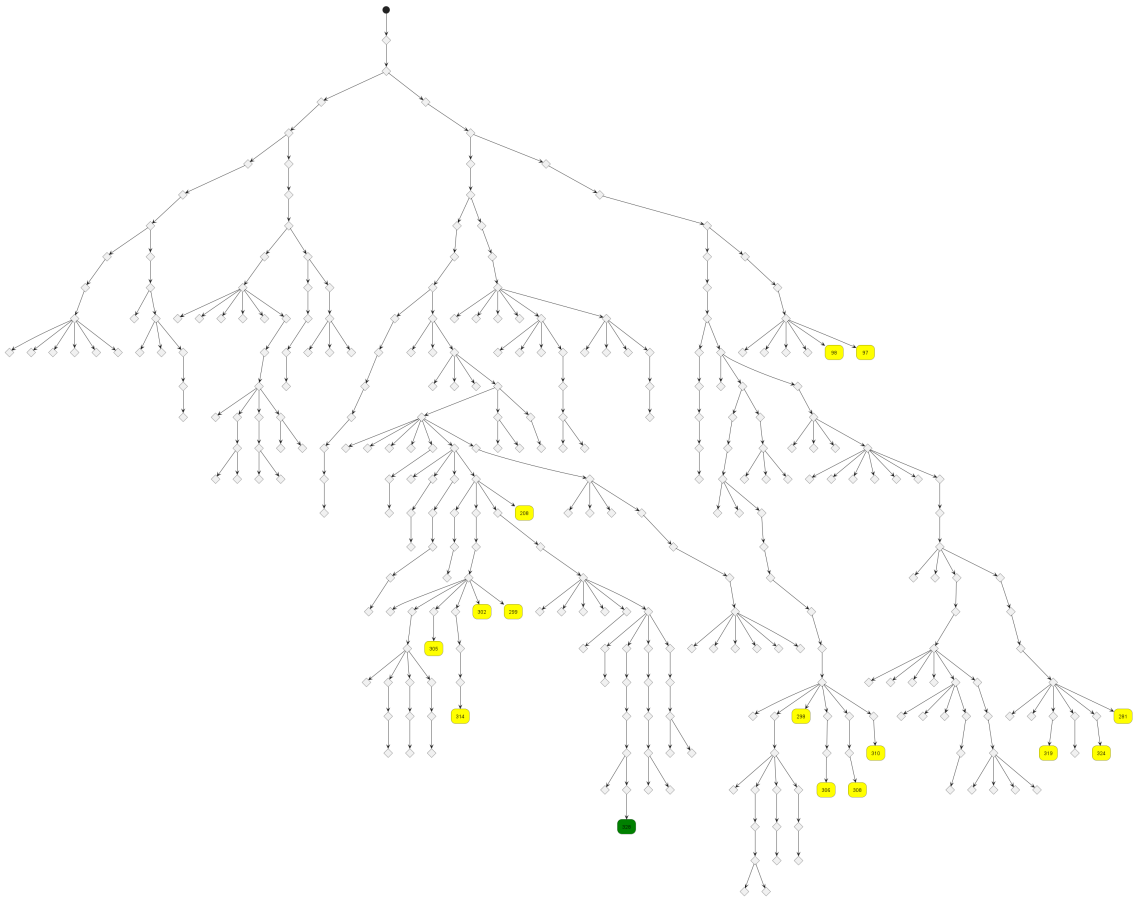


Figure 5.9: Visualisation of all forked branches while learning JSON from a CoCo/R generated parser using the grammar approximation method. Each node depicts a new grammar that has been identified during the learning process where the root node is the grammar returned from the CFG analysis.

6 Fuzzing-based Grammar Inference by Refinement

In this chapter, we discuss the overall process of *Grammar Inference by Refinement*. The content presented in this chapter is based on our previously published, peer-reviewed work on *Grammar Refinement*, as detailed in [12]. The goal of this process is to infer some grammar G'_p that produces a language $\mathcal{L}(G'_p)$. $\mathcal{L}(G'_p)$ should approximate the input language $\mathcal{L}(G_p)$ of some program p as closely as possible. As usual, we define $G_p = (N, \Sigma, P, S)$ for the set of nonterminals N, the input alphabet Σ, the set of production rules P, and the start symbol S. We will start by defining the type of seed grammar on which the grammar inference part is based. We call this seed grammar an expanded core grammar G_{exp} because it extends a given core grammar to, in addition, provide us with information on which terminals are used in the production rules. However, we still lack information on which terminal is used at which exact position. However, such an expanded core grammar most likely includes all production rules present in the target grammar by design, among others, as we will demonstrate later in Section 6.3. Finally, we discuss how we can apply our refinement-based algorithm, using an expanded core grammar G_{exp} as a starting point, to infer a grammar G'_p that produces a good approximation to the target language $\mathcal{L}(G_p)$ as closely as possible. This refinement algorithm takes the expanded core grammar and iteratively identifies and removes incorrect rules. In each step, the correctness of the grammar's rules is evaluated based on the set of words generated up to an incrementally increasing maximum word length. The remainder of this chapter is structured as follows. We begin by formally defining the ideal expanded core grammar G_{exp}. Building on this definition, we describe how our refinement-based algorithm uses a G_{exp} to infer a grammar G'_p that closely approximates the target language $\mathcal{L}(G_p)$. We then describe how an expanded core grammar can be constructed from a core grammar. The chapter concludes with an evaluation of our approach.

6.1 Expanded Core Grammar

An expanded core grammar produces a language that is, under the assumption that the core grammar it has been built from is correct, a superset of $\mathcal{L}(G_p)$. Formally, we define $G_{exp} = (N, \Sigma, P_{exp}, S)$ for $P_{exp} \supseteq P$. As described in Section 6.3, we can build such an expanded core grammar given the original core grammar in addition to the input alphabet Σ. We can then use such an expanded core grammar to, as described in Section 6.2, apply a refinement-based algorithm to approximate $\mathcal{L}(G_p)$.

6.2 Refinement Algorithm

This section describes how we can use an expanded core grammar to approximate the language accepted by some program p. Before describing our algorithm on grammar inference by refinement, let us start with a brief summary of the properties of an expanded core grammar G_{exp}. Section 6.1 defines an expanded core grammar as

$$G_{exp} = (N, \Sigma, P_{exp}, S)$$

for $P_{exp} \supseteq P$ where the set of non-terminals N, the input alphabet Σ, and the start symbol S are equal to the one in G_p. The production rules P_{exp} are a superset of P. Note that we are not able to determine wether the definition given above holds in practice as G_p is unknown. However, it gives us a good intuition on how and why our refinement algorithm works and we should keep in mind that they pose an ideal case.

The central grammar refinement task in our approach is given in Algorithm 1. We illustrate how the algorithm works with a simple example. Note that this example differs significantly from our running example as the number of rules of the grammar given in Section 2.4 in Chomsky Normal Form (CNF) would be too long to consider here. As an alternative example, we will use a much simpler grammar at this point. Let us assume an executable program p with valid inputs $validInputs(p) = \mathcal{L}(G_p)$, where G_p is the context-free grammar with the start symbol S, the set of terminal and non-terminal symbols $\Sigma = \{+, *\}$ and $N = \{S\}$, respectively, and production rules:

$$S \rightarrow +* \mid +S*$$

Let us further assume that the input grammar G in Algorithm 1 has, respectively, the same sets Σ and N of terminal and non-terminal symbols as G_p, and that it has also S as its start symbol. The set of seed productions of G, that is, the set of productions that we want to refine with our algorithm to increase the precision of G, is formed by the following rules:

$$S \to + \mid * \mid ++ \mid +* \mid *+ \mid ** \mid +S \mid *S \mid S+ \mid S* \mid +S+ \mid +S* \mid *S+ \mid *S*$$

The first step in the algorithm (line 1) transforms the grammar G into an equivalent Chomsky normal form grammar $G' = (N', \Sigma, P', S)$, where $N' = \{S, N_+, N_*, A_1, A_2, A_3, A_4\}$ and P' is formed by the following productions:

$$\begin{aligned}
S &\to + \mid * \mid N_+N_+ \mid N_+N_* \mid N_*N_+ \mid N_*N_* \\
S &\to N_+S \mid N_*S \mid SN_+ \mid SN_* \mid N_+A_1 \mid N_+A_2 \mid N_*A_1 \mid N_*A_2 \\
A_1 &\to SN_* \\
A_2 &\to SN_+ \\
N_+ &\to + \\
N_* &\to *
\end{aligned}$$

Lines 2–10 of the algorithm simply initialize the required variables. Note that *countEmpty* + 1 is the minimum word length among all words in $\mathcal{L}(G')$. This means that in our example we need to start by inspecting words of *length* = *countEmpty* + 1 = 1. Further, we assign to *refinedProductions* the set of production in G', and initialize the sets of *safeProductions* and *seenProductions*. In *safeProductions* we initially include those productions in G' of the form $A \to t$, where A is a non terminal symbol added by the transformation to Chomsky normal form and $t \in \Sigma$. These rules are then safe from elimination during the whole

refinement process, unless they become unreachable from S. Thus, after executing these steps, we get the following values:

$$\begin{aligned} \mathit{length} &= 1 \\ \mathit{refinedProductions} &= P' \\ \mathit{safeProductions} &= \{N_+ \to +, N_* \to *\} \\ \mathit{seenProductions} &= \emptyset \end{aligned}$$

At this point, the algorithm is ready to start the refinement process, i.e., to step-by-step eliminate rules from the grammar that are responsible for producing words that do not belong to the input language of p. This has to be done as careful as possible, trying not to adversely affect the recovery rate of the candidate grammar. We have found out during the experiment reported in the paper, that a good rule of thumb is to only eliminate rules used in the corresponding parsing tree up to level denoted by the parameter l in the algorithm. Notice that to set the parameter l we use the fact that the minimum possible depth of the parse tree of a word of length n (if the grammar in Chomsky normal form) is $\lceil \log_2 n \rceil + 1$. A more aggressive approach to refinement, i.e., an approach that increases the value of l faster, has the advantage of reducing the computation time considerably. However, as witnessed by our experiments, if the approach is too aggressive in this regard, then it considerably reduces the recall rate of the obtained grammar. The while-loop in Line 11 defines the termination condition, which is met when there is no further rule among *refinedProductions* that can be disposed of. If the termination condition has not been meet, then the algorithm continues by repeating the procedure defined in Lines 12–28 until no production can be further deleted from the set of refined productions by examination of parse trees of words of the current length.

This repeat-until loop accounts for the fact that the seed grammar will most probably be ambiguous, i.e., there will be words in its language which can be derived using different parse trees. Since the number of parse trees grows too fast, we cannot look at all parse trees for each inspected word. Instead, we just take the first parse tree returned by the parsing algorithm (in our case the CockeYoungerKasami (CYK) algorithm). The intuitive idea is that by repeating the process for the same length of word after each refinement of the grammar, we can still examine the relevant parse tree that were missed in the previous round.

Following our example, at this point line 13 assigns to the variable *words*, all words of length 1 that can be produced by G' (that is, $+$ and $*$). Clearly, after the for loop in Lines 15–22 is executed, the variable *deleteCandidates* contains the set $\{S \to +, S \to *\}$ of productions. Since none of these productions is among those in *safeProductions*, both are deleted from *refinedProductions* and the set of productions of grammar G' is updated accordingly (Lines 25–27). Since the updated grammar G' does no longer produces any word of length 1, no refinement takes place in the next iteration of the repeat-until loop. Then the value of *length* is incremented by 1 and the process repeats itself, with the following outcomes for each subsequent step:

- For *length* $= 2$,
  - G' produces the words: $+*$, $*+$, $++$ and $**$.
  - Since $*+, ++, ** \notin$ *validInputs*(p) and $l = 1$, *deleteCandidates* takes the value $\{S \to N_*N_+, S \to N_+N_+, S \to N_*N_*\}$.
  - Since $+* \in$ *validInputs*(p), $S \to N_+N_*$ is added to *safeProductions*.
  - Thus *refinedProductions* reduces to

$$S \to N_+N_* \mid N_+S \mid N_*S \mid SN_+ \mid SN_* \mid N_+A_1 \mid N_+A_2 \mid N_*A_1 \mid N_*A_2$$
$$A_1 \to SN_* \qquad A_2 \to SN_+ \qquad N_+ \to + \qquad N_* \to *$$

- For *length* $= 3$,
  - G' produces the words: $++*$, $*+*$, $+*+$ and $+**$.
  - Since none of the words of length 3 produced by G' is in *validInputs*(p) and $l = 1$, then *deleteCandidates* takes the value $\{S \to N_+S, S \to N_*S, S \to SN_+, S \to SN_*\}$ and *safeProductions* remains unchanged.
  - At the end of this step, *refinedProductions* further reduces to

$$S \to N_+N_* \mid N_+A_1 \mid N_+A_2 \mid N_*A_1 \mid N_*A_2$$
$$A_1 \to SN_* \qquad A_2 \to SN_+ \qquad N_+ \to + \qquad N_* \to *$$

- For *length* $= 4$,

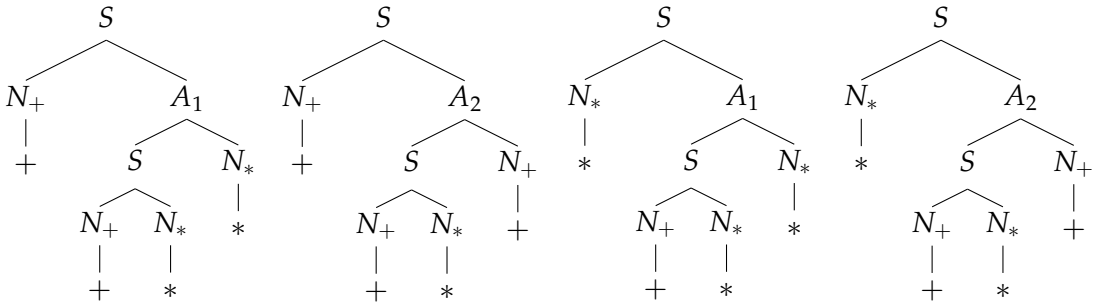


Figure 6.1: Parse Trees

- G' produces the words: $++**$, $++*+$, $*+**$ and $*+*+$ with the corresponding parse trees shown in Figure 6.1.
- Since $++*+, *+*+, *+** \notin validInputs(p)$ and $l = 1$, *deleteCandidates* takes the value $\{S \to N_+A_2, S \to N_*A_1, S \to N_*A_2\}$.
- Since $++** \in validInputs(p)$, $S \to N_+A_1$ is added to *safeProductions*.
- Thus, the content of *refinedProductions* further reduces to (notice that $A_2 \to SN_+$ is no longer reachable and, therefore, also eliminated)

$$S \to N_+N_* \mid N_+A_1 \qquad A_1 \to SN_* \qquad N_+ \to + \qquad N_* \to *$$

- For $length = 5$, the value of l is set to 2 and nothing else changes since the grammar obtained in the previous step does not produce words of length 5.
- Finally, for $length = 5$, the grammar only produces words that belong to $validInputs(p)$ and since $l = 2$, the rule $A_1 \to SN_*$ is added to *safeProductions*. By now, all productions in *refinedProductions* belong to *safeProductions* and thus the termination condition is met.

Notice that the language of G' returned by the algorithm corresponds exactly to $validInputs(p)$. Of course, given the heuristic nature of our algorithm, this satisfactory result cannot be guaranteed. Even when we are certain that the language recognized by the seed grammar G includes the input language of p, an exact solution would need to take into account all possible derivation trees for each word generated by the grammar, which would make the

approach unfeasible. A non-ambiguous seed grammar would solve this problem, but this cannot be realistically ensured for the targeted use cases of our algorithm.

6.2.1 Complexity Analysis

Since G' is in Chomsky normal form (see Line 1 in Algorithm 1), we see that every parsing tree of G' is a binary tree. Therefore, the main while loop from Lines 11–34 in Algorithm 1 will be repeated at most $p = 2^m$ times, where $m = |N'|$ is the number of non-terminals in the initial G'. Notice that by the pumping lemma for context-free languages, every word z in $\mathcal{L}(G)$ such that $|z| \geq p$ can be written as $z = uvwxy$ with the following conditions:

- $|vwx| \leq p$
- $vx \neq \lambda$
- For $i \geq 0$, the word uv^iwx^iy is also in $\mathcal{L}(G)$.

Therefore, when *length* in the algorithm reaches the value p, then $l = length$ and all remaining productions are added to *seenProductions*. Thus, each production in the initial G' is at this point no longer in *refinedProductions* or is in *safeProductions*, and the termination condition is met.

Our algorithm examines in the first step all words of length 1 produced by the grammar, then all words of length 2 produced by the refined grammar in the previous step, and so on. As seen before, this can go on up to words of length p. If there are $|\Sigma| = n$ terminals in the grammar G', then the number of different words that we will have to examine is bounded by $\mathcal{O}(n^p)$, i.e., $\mathcal{O}(n^{2^m})$. This complexity bound makes our algorithm intractable in theory. In practice, however, the values for the exponent p are in our experience usually somewhere between 3 and 8, since it corresponds to the nesting depth of the production rules in the grammar. The higher this value however, the more important it is to keep the number of symbols in Σ in check, using tokens to represent sets of symbols. For example, replacing symbols $0, 1, \ldots, 9$ in Σ by a token d.

As shown in our evaluation section, another point that allows the algorithm to perform reasonably well in practice is the fact that each time we eliminate a production rule from the grammar, the number of words that need to be checked in the next round reduces considerably.

Regarding the production of words ordered by length for a given grammar $G = (N, \Sigma, P, S)$ in Chomsky normal form, we notice that one can do that quite efficiently. First, we fix a total order $<$ of $N \cup \Sigma$, where the symbols in N precede those in Σ. Then using a priority queue Q defined in terms of $<$, the words belonging to $\mathcal{L}(G)$ can be generated without repetitions and ordered by length then lexicographical as follows:

1. Add S to Q.
2. Remove the first element w from Q (that is, the one with higher priority).
3. If w contains only terminals, output w.
4. If w contains a non-terminal, for each production α for the first non-terminal in w, append the results of expanding α to Q.
5. Repeat step 2–4 until either Q is empty, or all the required words have been produced.

Finally, we note that for each word w generated by the previous procedure, we can in parallel keep an associated parsing tree. Thus, it is relatively inexpensive in terms of computational time to produce the necessary parsing tree. In our prototype implementation of the algorithm, we use instead a less efficient approach consisting on applying the well-known CYK parsing algorithm for retrieving the parsing tree for each examined word. Even this less efficient approach still works reasonably well in practice, as shown by our experiments.

Algorithm 1: Grammar Refinement Algorithm

Input : Expanded grammar G_{exp} and program p.
Output: Refined grammar G'_p in Chomsky normal form.
Ensure : $precision(\mathcal{L}(G'_p), validInputs(p)) \geq precision(\mathcal{L}(G_{exp}), validInputs(p))$.

```
1  G'_p ← G_exp.to_normal_form()
2  countEmpty ← 0
3  length ← 1
4  while G'_p.getWords(length) = ∅ do
5      countEmpty ← countEmpty + 1
6      length ← length + 1
7  refinedProductions ← G'_p.productions()
8  safeProductions ← {A → t ∈ refinedProductions | A ∉ G_exp.non_terminals()}
9  seenProductions ← ∅
10 l ← 1
11 while (refinedProductions ∩ safeProductions) ≠ refinedProductions do
12     repeat
13         words ← G'_p.getWords(length)
14         deleteCandidates ← ∅
15         for w ∈ words do
16             parseTree ← G'_p.getParseTree(w)
17             seenProductions ← seenProductions ∪ parseTree.getProductions()
18             evalProductions ← parseTree.getProductionsUpToLevel(l)
19             if w ∈ validInputs(p)
20                 safeProductions ← safeProuctions ∪ evalProductions
21             else
22                 deleteCandidates ← deleteCandidates ∪ evalProductions
23         deleteProductions ← deleteCandidates \ safeProductions
24         if deleteProductions ≠ ∅
25             refinedProductions ← refinedProductions \ deleteProductions
26             refinedProductions ← reachable(refinedProductions)
27             G'_p.update(refinedProductions)
28     until deleteProductions = ∅;
29     if refinedProductions ⊆ seenProductions
30         l ← length
31     else
32         if length − countEmpty > 1 ∧ isInteger(log_2(length − countEmpty))
33             l ← log_2(length − countEmpty)
34     length ← length + 1
```

6.3 From Core Grammar to Expanded Core Grammar

Provided a core grammar structure G_{core} as well as the input alphabet Σ, constructing an extended core grammar G_{exp} as defined in Section 6.1 is trivial. We infer the production rules P_{exp} by replacing every $\bigcirc_i$ in every production rule $r \in P_{core}$ with the given input alphabet Σ and adding the resulting production rule p' to P_{exp}. Given that P_{core} is complete in the sense of its definition, we ensure that the property defined for extended core grammars in Section 6.1, namely that $P_{exp} \supseteq P$, is valid.

Let us take a look at the core grammar extracted by means of CFG analysis in Figure 5.7 and alphabet $\Sigma = \{\epsilon, 1, 2, 3, (,), +, -, *, /\}$. For reasons of simplicity, we will only consider rule `Expr`:

$$\texttt{Expr} \rightarrow \texttt{Term}\ (\ \bigcirc_1\ \texttt{Get Term} \mid \bigcirc_2\ )$$

Following the process described above, we replace every $\bigcirc_i$ in this production rule with Σ. This leaves us with the following production rule that will be added to P_{exp}:

$$\begin{aligned}\texttt{Expr} \rightarrow &\texttt{Term}\ \Big(\ (\epsilon \mid 1 \mid 2 \mid 3 \mid (\mid) \mid + \mid - \mid * \mid /)\ \texttt{Get Term} \\ &\mid (\epsilon \mid 1 \mid 2 \mid 3 \mid (\mid) \mid + \mid - \mid * \mid /)\ \Big)\end{aligned}$$

We can clearly see that the language $\mathcal{L}(G_{exp})$ produced by G_{exp} using this production rule is a superset of $\mathcal{L}(G_p)$. Consequently, refinement of this rule by removing terminals is a good way to approximate what the target production rule looks like. Section 6.2 describes how that could be done in practice.

6.4 Evaluation & Results

Our method for grammar inference by refinement takes as input an expanded grammar structure (see Section 6.1) that theoretically contains every rule that is needed. Now, each step determines which rules are removed in the context of a incrementally increasing

depth of the grammar, in other words each step considers words up to a certain length to determine which rules can be safely removed.

We conducted a series of tests to evaluate the performance in terms of precision and recall of Algorithm 1 for grammar learning through refinement. Precision and recall were measured as described in Section 2.3.1, random sampling $10,000$ words from each grammar. The results of those experiments are summarized in Table 6.1. Each column represents a different input language, which have been described in detail in Section 4. The first line with label "Seed G." shows the initial precision of each of the seed grammars w.r.t. its corresponding target language. Each subsequent line labeled as "Word length X" shows the evolution of the precision value after the algorithm has refined the grammar based on the examination of words of length X. The second-to-last line shows the recall of each of the refined grammars at the end of the process. At the beginning of the process, all seed grammars have perfect recall 1. The final line is self-explanatory and shows the running time for each of the experiments.

First, we should note that the running times could be improved substantially, simply making use of the fact that the examination of words of a given length can clearly be done in parallel. Furthermore, these running times correspond to a prototype implementation of the algorithm in Python, using an educational library for formal language manipulation (Pyformlang)[28], which is simple to use but not the most efficient for the task. The experiments were run in a Linux installation in a modest virtual machine with 16GB of RAM and an Intel(R) Core(TM) i7-8665U CPU at 1.90GHz. In any case, even the worst-case running time shown in the table is still somehow acceptable for the envisaged applications of our algorithm.

The column labeled "$\mathcal{L}(G_p)$" shows the results corresponding to the execution of the algorithm with $validInputs(p) = \mathcal{L}(G_p)$, where G_p is the simple grammar used as an example in Section 6.2. The seed grammar provided as input for this experiment corresponds to the seed grammar G, also given as an example in Section 6.2. With this simple example, it only takes our implementation 0.4s to return a grammar with perfect precision and recall.

Table 6.1: Precision improvement over time, final recall and running time of the refinement based method.

	$\mathcal{L}(G_p)$	MathExpr	Mail	JSON
Seed G.	0.0702	0.129	0.0025	0.3841
Word length 1	0.1517	0.3751	0.0025	0.3841
Word length 2	0.5904	0.5817	0.0025	0.3841
Word length 3	0.6401	0.584	0.0025	0.3841
Word length 4	1.0	0.5919	0.0025	0.3841
Word length 5		1.0	0.0025	0.3841
Word length 6			0.0025	0.3841
Word length 7			0.0025	0.3841
Word length 8			0.0032	0.3841
Word length 9			0.0107	0.3841
Word length 10			0.1808	0.3841
Word length 11			0.1808	0.9
Word length 12			0.1808	0.9
Word length 13			0.1808	0.9
Word length 14			0.59	0.9148
Word length 15			0.59	0.964
Word length 16			1.0	0.964
Word length 17				1.0
Final Recall	1.0	1.0	1.0	0.9265
Running time	0.4s	2.7s	34.7s	30m 18.8s

The column labeled as *MathExpr* show the results corresponding to the execution of the algorithm with $validInputs(p) = \mathcal{L}(G)$, where G is the following grammar for mathematical expressions:

$$S \rightarrow S + S \mid S - S \mid S * S \mid S/S \mid (S) \mid d$$

Please note that this grammar slightly differs from the *MathExpr* grammar given in Section 4 in a way that the dot before dash rule is not considered here. The seed grammar

given as input was formed by all rules obtained from the expansion (as described in detail in Section 6.3) of the following core grammar rules:

$$
\begin{aligned}
S &\rightarrow \bigcirc\, T \,\bigcirc\, T \,\bigcirc \mid \bigcirc\, T \,\bigcirc \\
T &\rightarrow \bigcirc\, F \,\bigcirc\, F \,\bigcirc \mid \bigcirc\, F \,\bigcirc \\
F &\rightarrow \bigcirc\, S \,\bigcirc \mid \bigcirc
\end{aligned}
$$

The algorithm returned a grammar with perfect recall and precision in less than 3 seconds.

In the experiment labeled "Mail", we used $validInputs(p) = \mathcal{L}(G)$, where G is the grammar that recognizes valid e-mail addresses defined by the following productions:

$$
\begin{aligned}
S &\rightarrow Body\,@\,Head\,.\,Tag \\
Body &\rightarrow Char\,Chars \\
Head &\rightarrow Char\,Chars \\
Tag &\rightarrow Char\,Char \mid Char\,Char\,Char \\
Chars &\rightarrow Char\,Chars \mid \epsilon \\
Char &\rightarrow t \qquad\qquad \text{being } t \text{ a token for } \{a, b, \ldots, z\}.
\end{aligned}
$$

Note that the these rules differ from the *Mail* grammar given in Section 4. Nevertheless, the languages produced by both grammars are equivalent. The corresponding expanded grammar used in this experiment has the following core rules:

$$
\begin{aligned}
S &\rightarrow \bigcirc\, Body \,\bigcirc\, Head \,\bigcirc\, Tag \,\bigcirc \\
Body &\rightarrow Char\,Chars \\
Head &\rightarrow Char\,Chars \\
Tag &\rightarrow Char\,Char \mid Char\,Char\,Char \\
Chars &\rightarrow \bigcirc\, Char \,\bigcirc\, Chars \bigcirc \mid \bigcirc \\
Char &\rightarrow \bigcirc
\end{aligned}
$$

Since our approach does not work well when the empty symbol is part of the seed grammar, mostly due to the fact that it is based in inspecting increasing word lengths, we

treat ϵ in the seed grammar as a non-empty standard symbol. Thus, the resulting seed grammar contains rules such as $Chars \rightarrow \epsilon\ Char\ Chars$, where ϵ is simply a non-terminal symbol of length 1. After the learning process has been completed, we simply remove all ϵ symbols from the refined grammar except for those that appear in rules of the form $A \rightarrow \epsilon$. This trick works well in our experiments. The increase in running time with respect to the previous experiments is mainly due to the fact that the algorithm can only start to eliminate rules from the seed grammar after it has considered words of length at least 8. Notice that this seed grammar does not produce any words of length smaller than 8, and that the number of words that need to be examined grows exponentially with length, unless some rules are eliminated from the grammar.

Our final experiment concerns learning a grammar that can recognize well-formed JSON files. The set $validInputs(p)$ is defined as the language recognized by the JSON grammar with the following rules.

$$
\begin{aligned}
S &\rightarrow Element \\
Element &\rightarrow Ws\ Value\ Ws \\
Value &\rightarrow Object \mid String \mid Number \mid Array \mid false \mid true \mid null \\
Object &\rightarrow \{\ Ws\ \} \mid \{\ Members\ \} \\
String &\rightarrow \text{“}\ Characters\ \text{”} \\
Number &\rightarrow Digit \mid Digit\ Number \\
Digit &\rightarrow d && \text{being } d \text{ a token for } \{0, \ldots, 9\} \\
Array &\rightarrow [\ Elements\ ] \mid [\ Ws\ ] \\
Members &\rightarrow Member \mid Member\ ,\ Members \\
Member &\rightarrow Ws\ String\ Ws : Element \\
Elements &\rightarrow Element \mid Element\ ,\ Elements \\
Characters &\rightarrow Character\ Characters \mid \epsilon \\
Character &\rightarrow t && \text{being } t \text{ a token for } \{a, \ldots, z\} \\
Ws &\rightarrow _ \mid \epsilon
\end{aligned}
$$

In this case the seed grammar had the following core rules:

$$
\begin{aligned}
S &\rightarrow \mathit{Element} \\
\mathit{Element} &\rightarrow \mathit{Ws}\ \mathit{Value}\ \mathit{Ws} \\
\mathit{Value} &\rightarrow \bigcirc\ \mathit{Object}\ \bigcirc \mid \bigcirc\ \mathit{String}\ \bigcirc \\
\mathit{Value} &\rightarrow \mid \bigcirc\ \mathit{Number}\ \bigcirc \mid \bigcirc\ \mathit{Array}\ \bigcirc \mid \bigcirc \\
\mathit{Object} &\rightarrow \bigcirc\ \mathit{Ws}\ \bigcirc \mid \bigcirc\ \mathit{Members}\ \bigcirc \\
\mathit{String} &\rightarrow \bigcirc\ \mathit{Characters}\ \bigcirc \\
\mathit{Number} &\rightarrow \mathit{Digit} \mid \mathit{Digit}\ \mathit{Number} \\
\mathit{Digit} &\rightarrow d \\
\mathit{Array} &\rightarrow \bigcirc\ \mathit{Elements}\ \bigcirc \mid \bigcirc\ \mathit{Ws}\ \bigcirc \\
\mathit{Members} &\rightarrow \bigcirc\ \mathit{Member}\ \bigcirc \mid \mathit{Member}\ \bigcirc\ \mathit{Members} \\
\mathit{Member} &\rightarrow \mathit{Ws}\ \bigcirc\ \mathit{String}\ \bigcirc\ \mathit{Ws}\ \bigcirc\ \mathit{Element} \\
\mathit{Elements} &\rightarrow \mathit{Element} \mid \mathit{Element}\ ,\ \mathit{Elements} \\
\mathit{Character} &\rightarrow a \ldots z \\
\mathit{Characters} &\rightarrow \mathit{Character}\ \bigcirc\ \mathit{Characters} \mid \bigcirc \\
\mathit{Ws} &\rightarrow \bigcirc
\end{aligned}
$$

As before, ϵ is considered not only as the empty symbol, but also as a terminal symbol of length 1.

Table 6.1 shows the result of the execution of our algorithm. The seed grammar starts to produce words that allow the algorithm to increase the precision of the refined grammar after it has examined words of length 11. At that point, the grammar can produce 9432 words, which are used to eliminate most of the incorrect productions with *Value* on its left-hand side. Unfortunately, at this point, the algorithm incorrectly eliminates a production, which decreases the recall of the refined grammar from 1 to 0.9265. Note that this rule belongs to the Chomsky normal form version of the seed grammar described above. Given the high number of words examined of length 11 (after this step less than 5000 in total are produced by the grammar for lengths 12 to 17) and our inefficient prototype, the running time for learning these grammar increases considerably w.r.t. the previous experiments. However, the observed increase is still acceptable.

While the theoretical complexity of the algorithm is high in theory, we show through a running time analysis and also through empirical experiments that the algorithm can be successfully used in practice for learning program input grammars of reasonable large size such as JSON. The initial results obtained are indeed very encouraging, achieving in all evaluated cases a perfect precision as well as a perfect or close to perfect recall.

7 Fuzzing-based Grammar Inference by Learning

In this chapter, we discuss the overall process of *Grammar Inference by Learning*. The content presented in this chapter is based on our previously published, peer-reviewed work on *Fuzzing-based Grammar Learning*, as detailed in [13] and [14]. The goal of this process is to infer a grammar G'_p that defines a language $L(G'_p)$ closely approximating the input language of a given program p. As usual, we denote by $G_p = (N, \Sigma, P, S)$ the target grammar of our inference process, where N is the set of non-terminal symbols, Σ is the set of terminal symbols, P is the set of production rules, and $S \in N$ is the start symbol. We assume that $L(G_p) = \text{validInputs}(p)$, that is, the set of all valid inputs accepted by the program p. We start by defining the type of seed grammar on which the grammar inference part is based. We call this seed grammar a grammar fraction G_{frac} because it only describes a small subset of the actual language. Such grammar fractions are usually extracted by examining how a program p consumes some word and then building a grammar that closely resembles this process. This can be done in practice by using dynamic symbolic execution or based upon instrumentation and CFG analysis as discussed later in this chapter. The resulting grammar fraction usually only produces the word used to build it, although some other words may be produced in addition. We can then use such a grammar fraction and apply a variation of the well-known L* algorithm for learning regular languages, namely NL* [3], to approximate $\mathcal{L}(G_p)$. As NL* cannot learn a context-free language, we apply it to learn the right-hand side of production rules of our grammar fraction instead. As described in Section 7.2, we propose novel variants of membership- and equivalence queries used by NL*.

The remainder of this chapter is structured as follows. We begin by formally defining the ideal grammar fraction G_{frac}. Building on this definition, we describe how our learning-based algorithm uses a G_{frac} to infer a grammar G'_p that closely approximates the target

language $\mathcal{L}(G_p)$. We then describe how an grammar fraction can be constructed from an expanded core grammar or, built from scratch using dynamic symbiolic execution (DSE). We will end the chapter with an evaluation of this approach, and conclude with a brief discussion of a potential alternative, namely the direct application of Angluins ideas for learning context-free languages in this context, which, while not implemented or evaluated yet, offers an intriguing direction for future exploration.

7.1 Grammar Fractions

In general, a grammar fraction G_{frac} produces a language that is a subset of $\mathcal{L}(G_p)$. Formally, we define $G_{frac} = (N_{frac}, \Sigma_{frac}, P_{frac}, S)$ for $\Sigma_{frac} \subseteq \Sigma$ and $P_{frac} \subseteq P$. In addition, starting from S we can produce at least one word $w \in \mathcal{L}(G_p)$ by applying the production rules P_{frac}. As we will see in Section 7.2, we can only infer a correct grammar if $N = N_{frac}$. It is important to note that, in practice, this property is undecidable during grammar fraction extraction, since G_p is unknown. Moreover, each non-terminal in the grammar fraction must be *generating*. As defined in Section 2.2, a *generating* non-terminal is one from which a word consisting solely of terminals can be derived in a finite number of production steps.

Such grammar fractions can be derived in two ways: As described in Section 7.3.1, we may do so by performing dynamic symbolic execution based on a given seed input. Alternatively, given some seed input, we can refine a grammar fraction from an expanded core grammar as described in Section 7.3.2. In both cases, the resulting grammar fraction provides the foundation for the learning algorithm introduced in Section 7.2.

7.2 Learning Algorithm

This section describes how we can use a grammar fraction to approximate the language accepted by some program p. Before describing our learning algorithm, let us start with a short recap of the properties on a grammar fraction G_{frac}. Section 7.1 defines a grammar fraction as

$$G_{frac} = (N_{frac}, \Sigma_{frac}, P_{frac}, S)$$

for $\Sigma_{frac} \subseteq \Sigma$ and $P_{frac} \subseteq P$ where the set of non-terminals N as well as the start symbol S is equal to the one in G_p. The production rules P_{frac} and the alphabet Σ_{frac} are a subset of P and Σ, respectively. Our learning algorithm is based on the research of Angluin et al. [2]. Before we jump into describing our method, let us first dive into the idea of learning regular languages from queries and counterexamples.

7.2.1 Learning regular languages

It is well known that there are effective algorithms that can infer a Deterministic Finite Automaton (DFA), and hence also a regular grammar or regular expression, to recognize regular languages $\mathcal{L}$. However, these algorithms need a "teacher" that can answer *membership* and *equivalence queries* w.r.t. $\mathcal{L}$. The first and most well-known algorithm of this kind was introduced by Dana Angluin [2] and is known as L^*.

- The *membership query (MQ)* inquires as to whether a provided string is a part of the target language.
- The *equivalence query (EQ)* tests if the target language exactly matches the language that a given automaton (or grammar in our example) recognizes. Otherwise, the "teacher" should be able to offer a counterexample.

L^* uses two sets (initially empty) of words $U, V \subseteq \Sigma^*$ which are prefix- and suffix-closed, respectively. L^* proceeds by making membership queries for all words in $(U \cup U\Sigma)V$ and tracking the results in a table $\mathcal{T} = (T, U, V)$ where T maps each $w \in (U \cup U\Sigma)V$ to either $+$ or $-$, depending on whether w is accepted or not. To each $u \in U \cup U\Sigma$ corresponds a function row_u such that $row_u(v) = T(u, v)$. A table $\mathcal{T}$ is closed if, for all $u \in U$ and all $a \in \Sigma$, there is a $u' \in U$ such that row_{ua} and $row_{u'}$ coincide. $\mathcal{T}$ is consistent if, for all $u, u' \in U$ and all $a \in \Sigma$, it holds that row_{ua} and $row_{u'a}$ coincide whenever row_u and $row_{u'}$ also coincide.

If $\mathcal{T}$ is not closed, then L^* finds $u' \in U\Sigma$ such that $row_{u'}$ does not coincide with any of the row_u with $u \in U$, adds u' to U, and asks membership queries for the words $u'av$ for all $a \in \Sigma$ and $v \in V$, updating table $\mathcal{T}$. Similarly, if $\mathcal{T}$ is not consistent, that is, if there are $u, u' \in U$, $a \in \Sigma$ and $v \in V$ such that row_u and $row_{u'}$ coincide and $row_{ua}(v) \neq row_{u'a}(v)$, then L^* adds av to V, asks for membership queries $u''av$ for every $u'' \in U \cup U\Sigma$, and updates $\mathcal{T}$ accordingly.

Finally, with $\mathcal{T}$ both closed and consistent, L^* builds a DFA $\mathcal{H}$ with state set $Q = \{row_u \mid u \in U\}$, initial state $q_0 = row_\lambda$, transition function δ, where $\delta(row_u, a) = row_{ua}$ and set of final states $F = \{r \in Q \mid r(\lambda) = +\}$. If the equivalence test is successful, that is, language $\mathcal{L}(\mathcal{H})$ coincides with the target language, then learning L^* is done. Otherwise, L^* the counter example u given by the "teacher", as well as every prefix of u, are added to U, and the process is repeated until the correct automaton is learned.

For every regular language, there is a unique minimal DFA accepting it, which can be characterized thanks to Nerodes right congruence. This characterization is at the base of the L^* algorithm. Even though a minimal DFA is well-defined and unique, its size can become impractically large depending on the structure of the language [23]. To solve this problem, Bollig et al. proposed in [3] an alternative algorithm (NL^*) for learning Nondeterministic Finite Automaton (NFA) instead, which can be exponentially more succinct than DFAs. NL^* works similarly to L^*, except that not all rows of the table correspond to the states of the hypothesized automaton. This, however, leads to a much more involved algorithm with complex closeness and consistency and termination conditions. The interested reader should refer to [3] for specific details.

In this work, we leverage the approach NL^* as part of our strategy (i.e. as a subroutine) in our algorithm to infer context-free grammars from program input samples. The assumed setting in our work makes the task more challenging, since, as explained at the beginning of this section, we cannot perform strict equivalence query tests as used in L^* and NL^*. We can only approximate these tests. On the other hand, we can still ask membership queries, since it is equivalent to determining if a particular input to a program p belongs to $validInputs(p)$.

7.2.2 Adapted Learning algorithm

Now we can adapt the ideas described in Section 7.2.1 to a practical setting making use of a grammar fraction G_{frac}. The key challenge here lies in adapting NL*, an algorithm for learning regular language, to function in a context where we want to learn a context-free language instead.

The grammar fraction G_{frac} should ideally have perfect precision. In the case of the symbolic execution based method, this is by construction. Refining a grammar fraction

Algorithm 2: Fuzzing-based Grammar Inference Algorithm

Input : Seed inputs $I \subseteq$ validInputs(p), set of terminals Σ and program p
Output: Inferred grammar G'_p

```
1  G_frac := findSeedGrammar(I, p);
2  N := G_frac.getNonTerminals();
3  G'_p := G_frac.clone();
4  T := G_frac.parseTrees(I);
5  for A ∈ N do
6      c := null;
7      repeat
8          M := runNL*(p, A, Σ, N, c);
9          c := searchForCounterexample(M, p, A, G_frac, T, 1000, 10)    ▷ See Alg. 3;
10     until c = null;
11     α := M.toRegularExpression();
12     G'_p.add("A → α")
13 return G'_p
```

from an expanded core grammar may, in theory, result in G_{frac} recognizing strings that are neither in I nor in $validInputs(p)$. Nevertheless, in our experiments we have not encountered such a case, confirming that the proposed heuristics work well in practice.

Assuming then a prefect precision of the grammar fraction G_{frac}, we can concentrate on expanding the set of production rules of G_{frac} to increase its presumable low recall. For this task, we propose the grammar-based fuzzing approach described in Algorithm 2.

The grammar fraction extraction is performed by the function findSeedGrammar(I, p) (line 1 in Algorithm 2). This function can be implemented using one of the alternative methods proposed in Sections 7.3.1 and 7.3.2, respectively. Due to the way we construct G_{frac}, we ensure that $I \subseteq L(G_{frac})$. Furthermore, if we assume that p uses a recursive descent parsing technique where each procedure/function implements one of the nonterminals of a grammar G_p such that $L(G_p) = validInputs(p)$, then $L(G_{frac}) \subseteq validInputs(p)$. Moreover, for every nonterminal symbol A of G_{frac}, we have that $\mathcal{L}(G_{frac}, A) = \{w \in \Sigma \mid A \Rightarrow^* w\}$ is not empty, that is, for every non-terminal A of the grammar fraction there is at least one finite derivation that starts with A and produces a word in Σ^*.

The algorithm then tries to augment the set of productions in G_{frac}, i.e., in the grammar fraction, to better approximate $validInputs(p)$ (lines 4-11 in Algorithm 2). It proceeds

Algorithm 3: Adapted Equivalence Query

Function: searchForCounterexample($M, p, G_{frac}, A, \mathcal{T}, n, m$)
Input: Automaton M, program p, Grammar Fraction G_{frac}, NonTerminal A, Set $\mathcal{T}$ of parse Trees and maximum number n and m of trials and mutations per trial, respectively.
Output: Counterexample string if found. Otherwise, *null*.

```
1 G := M.toGrammar();
2 for (i := 0; i < n; i++) do
3     w := G.generateString();
4     if ¬membershipQuery(G_frac, w, A, T, p) return w        ▷ See Alg. 4;
5     for (j := 0; j < m; j++) do
6         w' := w.applyMutation();
7         if w' ∉ L(G) ∧ membershipQuery(G_frac, w', A, T, p) return w';
8         else w := w';
9 return null
```

by considering, for every non-terminal symbol $A \in N$, a new rule of the form $A \rightarrow \alpha$, where α is a regular expression such that $\mathcal{L}(\alpha) \subseteq (N \cup \Sigma)^*$. We search for a suitable α using the NL^* algorithm [3] coupled with the procedure described in Algorithms 4 and 3 to answer the necessary membership and equivalence queries, respectively. At the end of this process, we obtain α by translating the automaton returned by NL^* into an equivalent regular expression (line 11). There exist several well-known algorithms for converting finite automata into equivalent regular expressions, but most can be considered as reformulations of the same underlying algorithmic idea and yield (almost) the same regular expressions [29]. Any such algorithm would work well for our purpose here. See, e.g., the classical textbook algorithm based on Ardens lemma [30, 31].

Each time NL^* needs to answer a membership query for a string $w \in N \cup \Sigma$, it calls the procedure described in Algorithm 4 with the following parameters:

- Grammar fraction G_{frac}.
- String w.
- Non-terminal A.
- Set $\mathcal{T}$ of parse trees such that $T_u \in \mathcal{T}$ iff $u \in I$

Algorithm 4: Adapted Membership Query

Function: membershipQuery($G, w, A, \mathcal{T}, p$)
Input: Grammar $G = (N, \Sigma, P, S)$, $w \in (N \cup \Sigma)^*$, $A \in N \setminus \{S\}$, set $\mathcal{T}$ of parse trees, program p.
Output: *true* if $A \to w$ is deemed to be a good candidate for extending the productions of G. Otherwise *false*.

```
1  if ¬∃Tx(T ∈ 𝒯 ∧ x ∈ nodes(T) ∧ label(x) = A) return false;
2  s := deriveString(w, G)                ▷ Derives s ∈ Σ* from w using G ;
3  T_A := parseTree(A, w, s, G)           ▷ Using left-most derivation and A → w ;
4  for each T ∈ 𝒯 do
5      if ∃x(x ∈ T ∧ label(x) = A)
6          for each x ∈ nodes(T) with label(x) = A do
7              T.replaceSubTree(x, T_A)   ▷ Subtree rooted at node x;
8          input := T.toString();
9          if input ∉ validInputs(p) return false;
10         else
11             if p.parseTree(input) = T return true else return false;
```

- Program p.

The function *deriveString* in line 2 of this algorithm produces a string $s \in \Sigma^*$ starting from w by applying the rules of G with using the grammar-based fuzzing technique described in Section 2.2. More precisely, for each non-terminal $A_i \in w$ we select a rule $A_i \to \alpha$ from G. We continue by replacing A_i in w with α and repeat until w no longer contains non-terminals. We select the rule of G that is applied in each step so that it corresponds to the shortest path that leads from A_i to a string of terminals. This is implemented by assigning weights to each non-terminal in G. The weight of a non-terminal reflects how costly it is to derive a string of terminals from it, in the sense of how many additional derivations are needed. To calculate these weights, we start by assigning weight zero to all terminals. Next we continuously go over the rules of G and search for a rule $A_i \to \alpha$ where α is a string that already has weights assigned to all terminals and non-terminals that appear in it. We sum up the weights of the terminals and non-terminals of α. If A_i has no weight assigned yet or has a weight greater than the one calculated in the current step, then we assign the new weight to A_i. This process continues until all weights are assigned and no lower weight is found for any of the non-terminal of G. Note that the function *parseTree*(A, w, s, G) in line 3 returns the parse tree corresponding to the left-most

derivation $A \Rightarrow w \Rightarrow^* s$ of G. On the other hand, $p.parseTree(input)$ in line 10 returns the parse tree obtained by tracking the function calls in the run of p with *input*, using the procedure explained earlier. The remaining parts of this algorithm are self-explanatory.

Whenever NL^* needs to answer the equivalence query for an automaton M, we apply the heuristic described in Algorithm 3. Notice that we do not have in this context a properly defined regular language that NL^* needs to learn. Instead, we search for a counterexample string in $\mathcal{L}(M)$ that does not satisfy the (adapted) membership query expressed by Algorithm 4, or vice versa. The search for a counterexample is carried out until one is found or a maximum number of trials n has been reached. In each trial, the algorithm first derives a string $w \in (N \cup \Sigma)$ by applying grammar-based fuzzing with a grammar G equivalent to the automaton M (line 3). If w does not satisfy the (adapted) membership query, the algorithm returns w as a counterexample (line 4). Otherwise, it generates a mutation w' of w (line 6). If w' is not in the language recognized by G (or equivalently M) but satisfies the (adapted) membership query, then the algorithm returns w' as a counterexample (line 7). Otherwise, it tries different mutations of w up to a maximum number m. For the cases considered in the experiments reported in this paper, $1000 \leq n \leq 10000$ and $m = 10$ give us maximum precision and maximum recall, as well as good relative performance.

We conclude this section by illustrating the behavior of the proposed algorithm using the running example of a Java program p, which parses the language G_p defined by the grammar in Listing 2.1, as introduced in Section 2.4.

The objective is to expand the rules of the grammar fraction G_{frac} obtained in the previous section from the Java program p in Listing 2.2, so that its language becomes as close as possible to that of G_p. Let us start by seeing how the expansion process works for the rule $\texttt{factor} \rightarrow 1$. To expand the rule, we want to learn a rule $\texttt{factor} \rightarrow \alpha_{\texttt{factor}}$ such that $\alpha_{\texttt{factor}}$ approximates the language consumed by the corresponding method in p. First, we have to identify which symbols are used by $\alpha_{\texttt{factor}}$. We do this by using static analysis to identify which functions are called by `factor` and add the according non-terminals as well as terminals from Σ. In our case, the set of symbols is $\{\Sigma \cup \texttt{expr}\}$. Now we can apply NL^* to learn $\alpha_{\texttt{factor}}$. To do so, we have to answer both membership-queries as well as equivalence-queries. First, we will give some examples on how membership-queries are answered while learning the rules $\texttt{expr} \rightarrow \alpha_{\texttt{expr}}$ as well as $\texttt{factor} \rightarrow \alpha_{\texttt{factor}}$. Note that

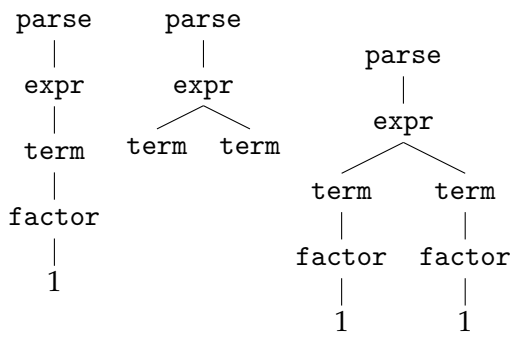


Figure 7.1: Parse Tree Evolution of Query 1

$\alpha_{\texttt{expr}}$ and $\alpha_{\texttt{factor}}$ are regular expressions, and therfore their respective languages $\mathcal{L}(\alpha_{\texttt{expr}})$ and $\mathcal{L}(\alpha_{\texttt{factor}})$ are regular.

1. $w \in \mathcal{L}(\alpha_{\texttt{expr}})$ for $w = \texttt{term term}$: We start by replacing the children of `expr` in T_u with w. Next we replace left non-terminals in the tree by applying the rules of G_{frac}. This process is illustrated in Figure 7.1. Finally, we transform the newly built parse tree to $w' = 11$ and execute p with w'. As $w' \notin validInputs(p)$, we return $w \notin \mathcal{L}(\alpha_{\texttt{expr}})$.

2. $w \in \mathcal{L}(\alpha_{\texttt{expr}})$ for $w =$ "$\texttt{term} + \texttt{term}$": Again, we start by replacing the children of `expr` in T_u with w. Next we replace left non-terminals in the tree by applying the rules of G_{frac} as illustrated in Figure 7.2. When we execute p with input $w' = 1 + 1$ we see that $w' \in validInputs(p)$. Now we check the call tree of the execution and see that it is equivalent to the parse tree of w', so we return $w \in \mathcal{L}(\alpha_{\texttt{expr}})$.

3. $w \in \mathcal{L}(\alpha_{\texttt{factor}})$ for $w =$ "`expr`": Again, we replace the children of `expr` in T_u with w and resolve left non-terminals with G_{frac}. This leads to the trees shown in Figure 7.3. Executing p with input $w' = 1$ shows that $w' \in validInputs(p)$. Note that the call tree of executing w' (see the most right tree in Figure 7.3) is not equivalent to the parse tree of w' so we return $w \notin \mathcal{L}(\alpha_{\texttt{factor}})$.

Finally, we provide an example of how we answer an equivalence-query while learning the rule $\texttt{expr} \rightarrow \alpha_{\texttt{expr}}$. We convert the given automaton M to a regular expression $\alpha = \texttt{term} \mid \texttt{term} + \texttt{term}$. Next, we build a new grammar G_α containing a single rule of the form $S \rightarrow \texttt{term} \mid \texttt{term} + \texttt{term}$ with S being the start symbol. We continue by searching for a counterexample generating new words using a grammar-based fuzzer

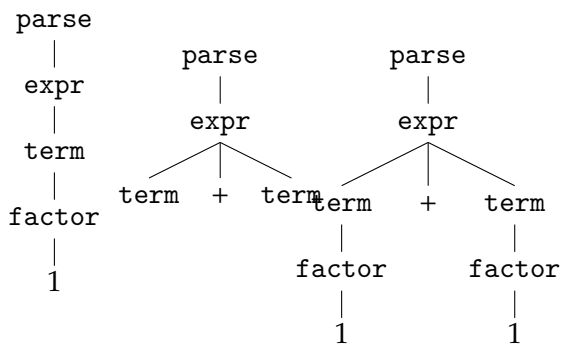


Figure 7.2: Parse Tree Evolution of Query 2

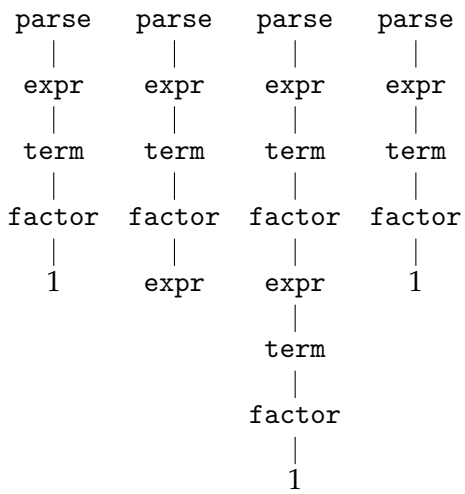


Figure 7.3: Parse Tree Evolution of Query 3

with G_α. Assume that we find a counterexample $c = \texttt{term} - \texttt{term}$ by applying a mutation to a generated word $\texttt{term} + \texttt{term}$. We return c and continue to answer membership-queries until the next equivalence-query is performed. In our case, the next automaton would translate to the regular expression $\alpha = \texttt{term} \mid \texttt{term}(-\texttt{term} \mid +\texttt{term} \mid \epsilon)$. As this is already the correct α, we will not be able to find another counterexample and stop fuzzing when the specified maximum number of attempts is reached. Finally, we add $\texttt{expr} \rightarrow \texttt{term} \mid \texttt{term}(-\texttt{term} \mid +\texttt{term} \mid \epsilon)$ to G'_p. We repeat the process described above for every non-terminal in G_{frac}. The resulting grammar G'_p is shown in Listing 7.1. This grammar is equivalent to G_p given in Figure 2.1.

```
 parse → expr
  expr → term | term ( −term | +term | ϵ)
  term → factor | factor ( /factor | ∗factor | ϵ)
factor → 1 | 2 | 3 | (expr)
```

Listing 7.1: Example of a Learned Grammar

7.2.3 Complexity Analysis

The core of our grammar learning method uses the NL^* algorithm proposed in [3] for learning NFA. See Line 7 in Algorithm 2. NL^* requires to resolve at most $\mathcal{O}(n^2)$ equivalence queries and $\mathcal{O}(m|\Sigma|n^3)$ membership queries, where n is the size of the minimal DFA for the given regular language, and m is the length of the longest counterexample returned by equivalence queries. Thus, the theoretical complexity of NL^* w.r.t. equivalence, as well as membership queries, is higher compared to L^*. Notice that L^* requires $\mathcal{O}(n)$ equivalence and $\mathcal{O}(m|\Sigma|n^2)$ membership queries. However, it is shown in [3] that fewer equivalence and membership queries are actually needed in practical scenarios. In fact, this has been confirmed by our experiments. Moreover, the NFAs resulting of applying the NL^* have, in general, significantly less states than the equivalent DFAs produced by the L^* algorithm, reducing the space requirements during computation and the number of rules in the corresponding grammar.

This analysis leaves us with two main questions regarding the computational complexity of implementing our grammar learning method, namely, what are the costs of the subroutines for resolving (the adapted) equivalence and membership queries described in Algorithms 3 and 4, respectively. The computational cost of the adapted membership query (Algorithms 4) can be established as follows. The condition in line 1 requires in the worst case to check every node of every tree in $\mathcal{T}$ and thus takes time bounded by $|\mathcal{T}| \cdot n$, where n is the number of nodes of the tree in $\mathcal{T}$ with the highest number of them. We already know from the previous section that the derivation/parsing required to complete the steps detailed in lines 2–3 takes (under our assumption of LL type grammars) time linear in the length of w. Except for the operations in Line 8, the tasks in lines 4–10 require (in the worst case) to examine each node of each tree in $\mathcal{T}$, plus linear time operations on the parse trees and linear time conversions of parse trees to strings. All of these operations take time $\mathcal{O}(|\mathcal{T}| \cdot n)$, where n is the number of nodes of the tree in $\mathcal{T}$ with the highest

number of them. Finally, checking whether the string *input* is in the set of valid inputs of p as required in Line 8, it can be assumed to take time linear in $|input|$ if p uses an LL type parser.

Regarding the computational cost of the adapted equivalence query, each time Algorithm 3 is called, it will need to transform a NFA into a grammar G and use a fuzzing technique to produce m mutations for n different string generated using G. Notice that there is a subtlety here as the non-terminals in the seed grammar are again treated as non-terminal in the grammar G, while they are considered terminals by the NFA. Nevertheless, the translation from the NFA M to the regular grammar G is done in the usual way, which takes time linear on the number of state transitions of the NFA. For the production of the mutations, each of the generated strings has length bounded by $\mathcal{O}(c \cdot e)$ where c is a constant parameter (which is set to 100 in our experiments) and e is the maximum number of non-terminals that appear on the right hand-side of any productions of G. To see this, note that the method *generateString*() of G works by generating a parse tree using the grammar G. The root of the tree is the start symbol. In each step from 1 to c, a leaf node labeled with a non-terminal is chosen randomly, let us say A, and then expanded with children nodes corresponding to a production (also chosen randomly) with A on its left hand side. Finally, each remaining leaf node that corresponds to a non-terminal is resolved by using the shortest derivation that leads to terminals. Regarding the time required for the m mutations of a given word w, this takes time $\mathcal{O}(|w|)$ as in each iteration we simply choose randomly a node of its parsing tree (built by the *generateString*() method) and randomly apply one mutation to that node (adding, removing, or changing it).

7.2.4 Angluin-style learning of context-free languages

A reader familiar with Angluins work might ask why we do not build upon the L^{cf} algorithm proposed in [2] that has been specifically designed to learn context-free languages, and instead choose to adapt her algorithm for regular languages to our setting. The purpose of this section is not to present a complete or practical solution, but rather to provide an intuition for how Angluins approach to learning context-free languages could, in principle, fit into the idea of *Fuzzing-based Grammar Inference*. We emphasize that this is a highly experimental exploration and has not been part of any published work. In the remainder of this section, we first give a brief introduction to the L^{cf} algorithm as

described in [2]. We then provide an intuition on how one might adapt this algorithm to our setting, followed by a short discussion on the potential impact of these adaptations on runtime performance.

Let us first introduce the L^{cf} algorithm. We have to assume a slightly more capable teacher as well as some deeper knowledge about the language itself, in contrast to L^*. The learner has to know the set of non-terminal symbols N as well as the start symbol S of G_p. The teacher has to be able to answer two types of queries:

- The membership-query $MQ(x, A)$ where x is a string of terminal symbols and A a non-terminal symbol. The teacher determines whether $A \xrightarrow{P^*_{G_p}} x$, or in other words, if the string x can be produced starting from A using the production rules of G_p.
- The equivalence-query $EQ(G')$ asks if G' is equivalent to G_p. If not, it returns some counterexample t that can be produced by G' but not by G_p.

We start with a grammar G' in Chomsky normal form. As the learner knows the set of terminals Σ as well as non-terminal symbols N, we fill the initial grammar with the set of all possible production rules. Note that this is only possible due to the limitation of G' to Chomsky normal form.

To learn a correct G', we continuously ask equivalence-queries until the answer is yes, or, in other words, until $\mathcal{L}(G') = \mathcal{L}(G_p)$. Whenever a counterexample t is returned, we diagnose the error in G' and remove it. Note that in this setting, a counterexample can only be a string generated by G' but not by G_p since the language of G_p is a subset of the language of G' by construction. We continue by constructing a parse tree of t following the production rules of G' starting with S. As G' is in Chomsky normal form, each node in the parse tree has two successor non-terminal symbols at most. Let us denote them by A and B. In addition, we denote the frontier derived from A and B as t_1 and t_2, respectively. Clearly, $t = t_1 \cdot t_2$.

Next, we ask $MQ(t_1, A)$, or in other words, we ask if t_1 can be built starting from the non-terminal A using the production rules of G_p. If the answer is *no*, we now continue to recursively diagnose the subtree rooted in A. If the answer is *yes*, we continue to ask $MQ(t_2, B)$. If the answer is *no*, we continue by recursively diagnosing the subtree rooted in B. However, if the answer to $MQ(t_2, B)$ is *yes* also, we now know that t_1 can be derived from A and that t_2 can be derived from B but t cannot be derived from S. This means that

the rule $S \rightarrow AB$ cannot be in G_p and so we remove it from G'. Recursive diagnosis of the subtrees works in the same way as described for the start symbol S, except by performing the same procedure for some A or B. Eventually, we can also find that some element A has a terminal a as a successor. In such a case, if $MQ(a, A)$ answers *no*, we remove the rule $A \rightarrow a$ from G'. t' Note that with each counterexample, one rule will be removed from G'. By removing only rules from G' that cannot be in G_p, each step preserves the property of G' that $\mathcal{L}(G') \supseteq \mathcal{L}(G_p)$ and each new counterexample is one that can be produced by G' but not by G_p. Eventually, all incorrect production rules are removed from G' such that G' is indeed equivalent to G_p.

Next, we want to briefly outline how we could apply L^{cf} to learn a G'_p in practice. Fundamentally, we have two main issues to solve:

1. We are unable to answer $MQ(x, A)$. To do this, we would need to identify the exact location of the parsing subroutine of non-terminal A within program p and execute this subroutine with string s.

2. L^{cf} assumes that the grammars are expressed in Chomsky Normal Form (CNF), a condition that cannot be guaranteed in practice. Consider a program p that implements G_p using a recursive-descent parsing strategy, where G_p is not written in CNF. Converting G_p into an equivalent grammar in CNF requires the introduction of additional non-terminals and altered production rules. While the resulting grammar generates the same language, its structure, specifically the set of non-terminals and production rules, no longer reflects the actual parsing logic implemented in p.

Issue (1) can be solved by the following approach which will be used in a similar way in our final method presented in Section 7.2.2. To answer $EQ(x, A)$, we need to be able to execute the target program p in a way that gives us the correct answer. However, directly executing p with x will not work. To avoid this, we could embed x in a known correct word $w \in validInputs(p)$. This word could be delivered using a grammar fraction G_{frac} (see Section 7.1). Now, we generate some w from G_{frac} along with its parse tree T_w. Finally, we exchange one node corresponding to A in T_w with x. As a result, we will be able to get a parse tree T'_w that we know is correct except for a potential issue having its root in the x we inserted. Clearly, if the frontier derived from T'_w is rejected when executing p with it, it is due to the inserted substring x and consequently we answer $MQ(x, A)$ with *no*. If the input is accepted, we answer $MQ(x, A)$ with *yes*.

Issue (2) could be solved by rewriting the algorithm for learning context-free languages to work not only in Chomsky normal form, but for any context-free grammar. Consequently, while diagnosing some counterexample t, we need to take into account that some non-terminal A may have one or more children $B_1 \cdots B_n$. As such, we define $t = t_1 \cdots t_n$. Now, we apply the same procedure as above: If one $MQ(t_n, B_n)$ is *no*, we continue to diagnose its parse tree rooted in B_n. If, and only if all membership queries have been answered with *yes*, we remove the rule $A \rightarrow B_1 \cdots B_n$. In practice, this would mean that we start from an expanded core grammar G_{exp} (see Section 6.1) as we know that the set of production rules is a subset of G_p. Now we remove rule after rule from G_{exp} until no counterexample is found.

In theory, we would have to perform at most $|P|$ equivalence queries, where $|P|$ is the total number of rules within our seed grammar[2]. However, some production rules in G_p could lead to an exponential explosion of rules that we would have to consider. For example, a production rule $S \rightarrow (H|h)(A|a)(L|l)(L|l)(O|o)$ would lead to a total of 59049 rules in our expanded core grammar. Almost all of them would have to be removed to approximate G_p. As described above, the removal of one rule is based on one provided counterexample. This, however, means that for each rule we want to remove, we first have to perform an equivalence query which is quite costly in regards to performance and finding a counterexample is not guaranteed. In theory, using the original version of the algorithm, the running time of L^{cf} is bounded by a polynomial in $|N|$ and $|\Sigma|$, and the maximum length of any counterexample provided. However, this polynomial running time can only be guaranteed when using a grammar in CNF [2]. The effects on running time of our proposed adaptions above have not been studied in detail and are thus unknown. Moreover, unlike the adapted L^* algorithm, the adapted version of L^{cf} does not support parallelization.

7.3 Grammar Fraction Extraction

The objective of grammar fraction extraction is to infer a grammar G_{frac} that aims to approximate the definition given in Section 7.1 as closely as possible. In particular, the grammar fraction should be capable of generating a subset of the target language. We have proposed two different methods based on Dynamic Symbolic Execution (DSE) as well as based on a given expanded core grammar. Extracting a grammar fraction using Dynamic

Symbolic Execution (DSE) involves analyzing the programs source code while executing it with a given seed input. During execution, the analysis determines which parts of the program consume which symbols of the input string. Based on this information, we construct a grammar that precisely reflects how the program processes the given input. Building a grammar fraction from an expanded core grammar removes each production rule from int that is not needed to generate a given seed input.

7.3.1 Dynamic Symbolic Execution based Grammar Fraction Extraction

Here we want to provide a method to extract a grammar fraction G_{frac} defined in Section 7.1 directly from the source code of p given some seed inputs I. The method consists of three main steps:

1. Execute p with some $w \in I$ and track all function calls. Add every corresponding function name to the initially empty set of non-terminals N_{frac} of G_{frac}.
2. For every execution run of p with some input $w \in I$, extract a parse tree T_w that follows how w has been consumed by p. The internal nodes of T_w are labeled with non-terminal symbols, while the leaves are labeled with terminal symbols whenever a terminal has been consumed by a function.
3. Add a production rule to G_{frac} for every internal node e of T_w for every $w \in I$. Each production rule has the form $e \rightarrow c_1, \ldots, c_n$ where $c_1, \ldots, c_n$ denotes the children of e in T_w from left to right.

Step (1) is used to initially determine the set of non-terminals used in G_{frac}. As mentioned previously, we might miss some non-terminals in this step that are needed to learn G_p if they are not used while consuming w. In such a case, we can extract grammar fractions from two or more words and combine them. Step (1) may be achieved by various means, e.g. code instrumentation, which tracks which functions are called during execution. However, Step (2) in addition needs to be able to track which terminal is consumed by which function during execution to be able to build the parse tree T_w. This brings about the need for a more complex analysis. In our case, we have used the dynamic symbolic execution framework of eknows [26]. In related work, dynamic tainting as in [21] or dynamic control flow as in [17] has been used for a similar purpose. However, those tools

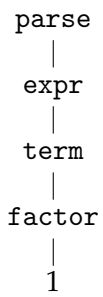
are mostly bound to specific programming languages. Step (3) finally describes how we build a grammar fraction G_{frac} from a set of parse trees T_w.

Let us take a look at the running example defined in Section 2.4 given the program p and the seed inputs $I = \{1, 2+2\}$. We start by executing p with inputs $w \in I$. By tracing the execution runs, we (1) determine the non-terminals, or, in other words, the functions called during both execution runs to be $N_s = \{\texttt{parse}, \texttt{expr}, \texttt{term}, \texttt{factor}\}$ as well as (2) build the corresponding parse trees. Figures 7.4a and 7.4c show the parse trees derived from inputs "1" and "2+2", respectively. We proceed to build the corresponding production rules given in Figures 7.4b and 7.4d from their respective parse trees. Note that Figure 7.4e shows the combined grammar fraction $G_{frac_{wu}} = (N_w \cup N_u, \Sigma_w \cup \Sigma_u, P_w \cup P_u, \texttt{parse})$ for both inputs w and u. Combining two grammar fractions as described above leads to a new grammar fraction that produces either the same or a larger subset of $\mathcal{L}(G_p)$. Note that combining enough grammar fractions from a wide array of different seed inputs eventually leads to a very good approximation of G_p. This strategy has been followed by [21] and [17] but is heavily based on a good set of seed inputs that are able to produce grammar fractions that, combined, produce a language equivalent to $\mathcal{L}(G_p)$.

7.3.2 From Expanded Core Grammar to Grammar Fraction

Given some expanded grammar $G_{exp} = (N, \Sigma, P_{exp}, S)$ for $P_{exp} \supseteq P$, we want to reduce it to a grammar fraction $G_{frac} = (N_{frac}, \Sigma_{frac}, P_{frac}, S)$ with $\Sigma_{frac} \subseteq \Sigma$ and $P_{frac} \subseteq P$ for some words I. Note that $P_{frac} \subseteq P \subseteq P_{exp}$. Consequently, the rules of the grammar fraction are a subset of the rules of the given expanded grammar. We remove each rule from P_{exp} that is *not* used in the production of words I. This leaves us with a grammar fraction P_{frac}.

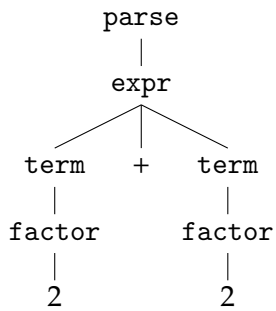
We eliminate the rules that are not necessary to derive given inputs I by iteratively selecting a rule $r \in P_{exp}$ and checking whether G_{exp} with rules $P_{exp} \setminus \{r\}$ is still capable of producing every word in I. If not, we add r to P_{frac}. We continue until all rules $r \in P_{exp}$ have been checked. Finally, we define Σ_{frac} to hold all terminal symbols present in the rules in P_{exp}. Additionally, the set of all non-terminal symbols present in the rules in P_{exp} should be equal to N. If this is not the case, the vital property of a grammar fraction defined in Section 7.1 is not valid. This indicates that the missing non-terminals (equivalent to parsing routines or functions in our program p) in fact are not covered by the set of seed inputs I.



(a) Parse tree of input "1"

$$\begin{aligned} \texttt{parse} &\rightarrow \texttt{expr} \\ \texttt{expr} &\rightarrow \texttt{term} \\ \texttt{term} &\rightarrow \texttt{factor} \\ \texttt{factor} &\rightarrow 1 \end{aligned}$$

(b) Grammar fraction of input "1"



(c) Parse tree of input "2+2"

$$\begin{aligned} \texttt{parse} &\rightarrow \texttt{expr} \\ \texttt{expr} &\rightarrow \texttt{term} + \texttt{term} \\ \texttt{term} &\rightarrow \texttt{factor} \\ \texttt{factor} &\rightarrow 2 \end{aligned}$$

(d) Grammar fraction of input "2+2"

$$\begin{aligned} \texttt{parse} &\rightarrow \texttt{expr} \\ \texttt{expr} &\rightarrow \texttt{term} \mid \texttt{term} + \texttt{term} \\ \texttt{term} &\rightarrow \texttt{factor} \\ \texttt{factor} &\rightarrow 1 \mid 2 \end{aligned}$$

(e) Combined Grammar Fraction of Inputs "1" and "2+2"

Figure 7.4: From the parse trees to a grammar fraction

Let us wrap up this section with a short example: Consider the expanded grammar at the end of Section 6.3 with the expanded rule for `Expr` and the input set $I = \{1\}$. The only rule needed to produce the word 1 is, in fact, `Expr` $\rightarrow$ `Term` ϵ. If we follow the steps described above we will get the following grammar, which is equivalent to the one

resulting from extracting a grammar fraction by means of dynamic symbolic execution given seed input 1:

```
 parse → expr
  expr → term
  term → factor
factor → 1
```

7.3.3 Complexity Analysis

We start with analyzing the runtime complexity of extracting a grammar fraction using dynamic symbolic execution of the source code given some set of inputs I. We assume that p uses a recursive descent parsing algorithm that is capable of deciding whether some input $u \in I$ of length n is accepted or not in time $\mathcal{O}(n)$. This assumption holds for the class of LL grammars. As symbolic execution of p only adds constant time to each step, we can safely assume that the steps necessary to construct the parse tree T_u of some input $u \in I$ with length n, that is, (1) tracking function calls and names, and (2) building the parse tree, can still be done in $\mathcal{O}(n)$. The size of T_u is proportional to the length n of the input u. To extract the rules of our grammar fraction from T_u, we need to visit each internal node of T_u exactly once, which requires constant time per node. Thus, the overall extraction runs in time linear in the number of internal nodes. Finally, we have to repeat that process for every $u \in I$ leaving us with a final runtime complexity of $\mathcal{O}(n \cdot |I|)$ where n is the length of the largest word in I.

Next, let us analyze the complexity of constructing a grammar fraction from an expanded grammar. We define I as a set of known words, and n to be the maximum length of some word $u \in I$. Furthermore, we need to consider the total number of rules present in G_{exp}. Here, we consider the rules of G_{exp} to be given in Chomsky normal form. First, we need to consider the complexity of deciding whether a given context-free grammar produces some $u \in I$. We assume that we use the CYK algorithm to decide this, although there exist more efficient algorithms considering a more limited setting. However, the CYK algorithm has a time complexity $\mathcal{O}(n^3)$ for words of length n. In total, we need to perform this operation for every $u \in I$ for every rule $r \in P_{exp}$, resulting in a total runtime complexity

of $\mathcal{O}(n^3 \cdot |I| \cdot |P_{exp}|)$ if we choose the CYK algorithm. However, note that the seed inputs provided are usually quite short.

7.4 Evaluation & Results

Our method for grammar inference based on learning takes as input a grammar fraction (see Section 7.1). In each step, one rule of the target grammar is learned, taking as input the provided grammar fraction. Note that to be able to extract a grammar fraction by the methods described in Section 7.3.1 as well as in Section 7.3.2, we need to provide some initial seed input.

We applied our grammar inference algorithm based on learning in the setup described in Section 4. If a specified maximum number of allowed membership queries has been reached without finding a counterexample, execution is stopped, and the current state of the inferred grammar G'_p is returned. Finally, we calculate $precision(L(G'_p), L(G_p))$ and $recall(L(G'_p), L(G_p))$ randomly sampling 1 000 words each.

Table 7.1 and Table 7.2 display the results, with the first column indicating the targeted parser. The second and third columns show the resulting precision and recall of our extracted grammar, followed by the total amount of unique MQ performed, the total number of EQ performed, and the time elapsed. In contrast to previous approaches that rely on a good number of representative seed inputs to perform appropriately, we have used sets I with no more than one seed input each except when learning the URL language. Our results in Tables 7.1 and 7.2 show that we are able to recover a grammar that perfectly matches the target grammar for the 1 000 random inputs examined. This shows that our approach is indeed robust to recover context-free grammars from recursive top-down parser.

The grammars used to generate the parsers for our experiments, along with the extracted seed grammars, are available in our prototype published on GitHub[1], enabling replication of our experiments. In the remainder of this section, we provide more details on the experimental evaluation of our method, including a small discussion of each target language.

[1] `https://github.com/software-competence-center-hagenberg/fbgl/`

Target	Precision	Recall	MQ	EQ	Time
ExprParser	1.0	1.0	15 361	7	7s
MailParser	1.0	1.0	28 227	8	11s
HelloParser	1.0	1.0	9 927	3	58s
AdvExprParser	1.0	1.0	24 100	8	9s
JsonParser	1.0	1.0	841 514	34	22m 40s
URLParser	1.0	1.0	1 270 165	40	40m 49s

Table 7.1: Precision and recall results for parsers generated with CoCo/R, evaluated using the learning-based approach.

Target	Precision	Recall	MQ	EQ	Time
ExprParser	1.0	1.0	18 952	9	9s
MailParser	1.0	1.0	41 884	11	26s
HelloParser	1.0	1.0	17 235	5	1m 56s
AdvExprParser	1.0	1.0	23 098	11	11s
JsonParser	1.0	1.0	1 415 345	54	32m 13s
URLParser	1.0	1.0	1 478 557	46	43m 30s

Table 7.2: Precision and recall results for parsers generated with ANTLR, evaluated using the learning-based approach.

```
ExprParser → Term
      Term → Factor
    Factor → "1"
```

Listing 7.2: ExprParser Extracted Grammar Fraction.

```
ExprParser → Term | Term ( "-" Term | "+" Term | ϵ).
      Term → Factor | Factor ( "/" Factor | "*" Factor | ϵ).
    Factor → "1" | "2" | "3" | "(" Expr ")".
```

Listing 7.3: ExprParser Grammar inferred by the learning based approach.

- **ExprParser:** The ExprParser is a rather small example and acted as our running example in previous sections. We only require the simple input word "1" to be able to learn a grammar that is equivalent to the target grammar implemented by both the COCO/R and ANTLR generated parsers.

```
MailParser → String "@" String "." Tag
    String → Char
       Tag → Char Char
      Char → "a"
```

Listing 7.4: Mail Extracted Grammar Fraction.

```
MailParser → String "@" String "." Tag
    String → Char String_q1.
 String_q1 → (Char String_q1 | ϵ).
       Tag → Char (Char | Char (Char | ϵ) ) .
      Char → ("z" | "y" | "x" | "w" | "v" | "u" |
              "t" | "s" | "r" | "q" | "p" | "o" |
              "n" | "m" | "l" | "k" | "j" | "i" |
              "h" | "g" | "f" | "e" | "d" | "c" |
              "b" | "a" ).
```

Listing 7.5: MailParser Grammar inferred by the learning based approach.

```
HelloParser → "h" "e" "l" "l" "o"
```

Listing 7.6: HelloParser Extracted Grammar Fraction.

- **MailParser:** The MailParser accepts very simple email addresses. Interestingly, we struggled to learn such simple languages in the beginning, as some rules produce a language that is a subset of another language produced by another rule. In this specific case, the rule that accepts country-tags produces a subset of the rule that produces the body of the email address. This led to some problems when directly applying NL*. This highlights the need for instrumentation to accurately track whether the syntax of a given word aligns with the execution path taken by the program. In this example, a country tag could be incorrectly consumed by the parsing subroutine intended for the email address body, leading to an incorrect response to a membership query. In our experiments, we use seed input "a@a.aa" to fully recover the grammar from both parsers generated by COCO/R as well as ANTLR.

```
HelloParser → ("h" ("e" ("l" ("l" ("o" | "O" ) | "L"
              ("o" | "O" ) ) | "L" ("l" ("o" | "O" )
              | "L" ("o" | "O" ) ) ) | "E" ("l" ("l"
              ("o" | "O" ) | "L" ("o" | "O" ) ) | "L"
              ("l" ("o" | "O" ) | "L" ("o" | "O" )
              ) ) ) | "H" ("e" ("l" ("l" ("o" | "O" )
              | "L" ("o" | "O" ) ) | "L" ("l" ("o" |
              "O" ) | "L" ("o" | "O" ) ) ) | "E" ("l"
              ("l" ("o" | "O" ) | "L" ("o" | "O" ) )
              | "L" ("l" ("o" | "O" )  | "L" ("o" |
              "O" ) ) ) ) ).
```

Listing 7.7: HelloParser Grammar inferred by the learning based approach.

```
AdvExprParser → Expr
         Expr → Term
         Term → Factor
       Factor → Num
          Num → Digit
        Digit → "1"
```

Listing 7.8: AdvExprParser Grammar Fraction.

- **HelloParser:** The HelloParser is a good example taken from related work to reveal the limitations of a proposed method. It simply accepts all uppercase and lowercase variations of the word "hello". Although it is fairly easy to get a good seed input by means of dynamic symbolic execution, it is hard to do this by refining a grammar constructed by an AST analysis. Initially, the input grammar for HelloParser had 59049 (=9^5) possibilities, which had to be refined in seed grammar extraction. We are still able to cover this application with our approach; however, we see an exponential behavior with increasing word length. In future work, we want to optimize our seed extraction approach to reduce the exponential runtime. All in all, we were still able to fully recover the grammar by using seed input "hello" from both parser generated by COCO/R as well as ANTLR.

- **AdvExprParser:** This language is similar to the ExprParser language. Numbers may have an arbitrary length and are not limited to one, two, and three. In addition, the

```
AdvExprP → Expr.
    Expr → Term Expr_q2.
 Expr_q2 → ("-" Term Expr_q2 | "+" Term Expr_q2 | ϵ).
    Term → Factor Term_q2.
 Term_q2 → ("/" Factor Term_q2 | "*" Factor Term_q2 | ϵ).
  Factor → (Num | "(" Expr ")" ).
     Num → Digit Num_q1.
  Num_q1 → (Digit Num_q1 | ϵ).
   Digit → ("3" | "2" | "1" ).
```

Listing 7.9: AdvExprParser Grammar inferred by the learning based approach.

AdvExprParser accepts a greater variety in combinations of mathematical expressions and in the use of brackets. Similar to the ExprParser, we only required the input word "1" to fully recover the target grammar from both parser generated by COCO/R as well as ANTLR.

- **JsonParser:** This parser accepts the json language[2]. We were able to fully recover the json language with the seed input " 'a' . 1e-0 , '' : [true , true] " from both parser generated by COCO/R as well as ANTLR. Figure 7.5 shows a more detailed analysis of the learning process for the JsonParser generated by COCO/R. We calculated $precision(L(G'_p), L(G_p))$ and $recall(L(G'_p), L(G_p))$ every time an equivalence query was performed, where G'_p is the current state of the inferred grammar. Again, we have used 1 000 randomly sampled words each. In terms of recall, we see a consistent increase over time as the learned grammar is expanded, and as a result, the learned language grows significantly. When a rule that consumes terminals is learned, the boost in recall is often greater. For example, the final spike in recall occurs while learning the rules for parsing digits and mathematical symbols.

- **URLParser:** This language accepts a subset of the URL[3] language. To correctly identify the grammar of the URLParser, we require more than one input. The reason is that the parser covers HTTP-, FTP-, Telnet- as well as Gopher-addresses. If we only use a single input word, which is, for example, a HTTP-address, then only the production rule `HTTPaddress` and its subelements are covered. Extracting

[2] `https://www.json.org/json-en.html`
[3] `https://www.ietf.org/rfc/rfc1738.txt`

```
JsonParser → Element
   Element → Ws Value Ws
        Ws → " "
     Value → Object
     Value → Array
     Value → Number
     Value → "true"
    Member → Ws String Ws ":" Element
    String → """ Characters """
Characters → ""
Characters → Character Characters
 Character → "a"
    Number → Integer Fraction Exponent
   Integer → Onenine
    Digits → Digit
     Digit → "0"
   Onenine → "1"
  Fraction → ""
  Exponent → "e" Sign Digits
      Sign → "-"
```

Listing 7.10: JsonParser Extracted Grammar Fraction.

a seed grammar by AST analysis gives us an incorrect grammar. The resulting seed grammar lacks the rule for `Selector`. This is due to an ambiguity in the rule `GOPHERaddress`. In this case `Gtype Selector` is equivalent to the rule `Search` and as such the rule is incorrectly removed, because all input words are still accepted if a rule containing `Path` is used instead. Unfortunately, this grammar cannot be used as a seed for the learning algorithm because the parse trees from the program and the syntax trees do not match.

However, to avoid incorrectly removing a rule, we can inspect the parse tree by executing the target parser on the input word and verifying that the rules used to parse the word match those under consideration. If not, we know that this is an ambiguous path that should be handled appropriately. We could, for example, track how a word w is consumed by a grammar g using productions p. If we remove a

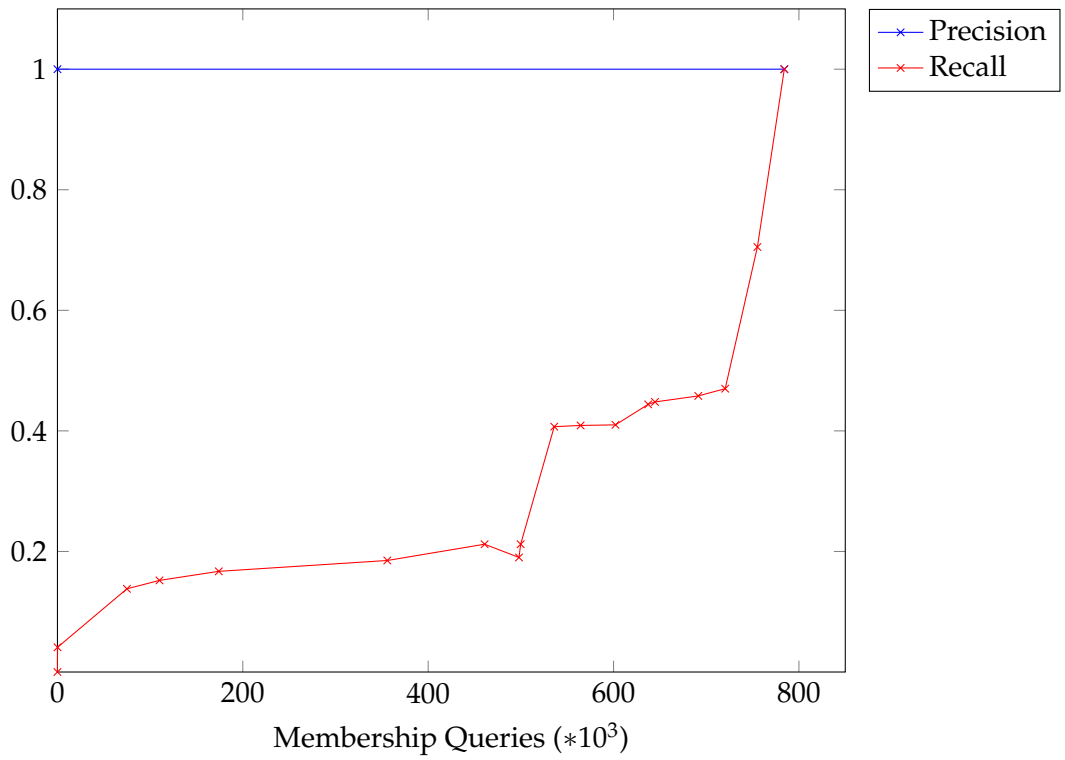


Figure 7.5: Json Learning process

production p, the conditions for consuming w change, but w can still be created by g, we know that we have identified an ambiguous path. Finally, by running the target program with w, we can determine the correct parsing path by inspecting the resulting parse tree. This allows us to identify and remove the corresponding rule accurately. In our experiment, we use the seed grammar also used in our work in [13] to fully recover the target grammar from both the parser generated by COCO/R as well as ANTLR.

Figure 7.6 shows the learning process while learning a grammar from the parser generated by COCO/R. Note that recall stays pretty low during most of the learning process, followed by a steep increase at the end. The reason behind this behavior lies in the order in which the rules are learned. Early on the algorithm learns rules that expand the grammar by mostly adding non-terminals. This does not lead to an increase in recall; however, lays the foundation for being able to correctly recognize the language later. It is only at the very end that rules which define how a string or

```
JsonParser → Element.
   Element → Ws Value Ws.
        Ws → " ".
     Value → Object | Array | String | Number |
             "true" | "false" | "null".
    Object → "{" (Ws "}" | Ws ("}" | String Ws
             ":" (Element "}" | Element ("}" |
             "," Members "}")))).
   Members → Member | Member ("," Members | ϵ).
    Member → Ws String Ws ":" Element.
     Array → "[" (Ws "]" | Ws ("]" | Value (Ws "]"
             | Ws ("]" | "," Elements "]")))).
  Elements → Element | Element ("," Elements | ϵ).
    String → "'" Characters "'" .
Characters → ϵ | Character Characters.
 Character → "a" | "b" | "c".
    Number → Integer Fraction Exponent.
   Integer → "0" | Onenine | Onenine (Digits | ϵ) |
             "-" (Onenine | Onenine (Digits | ϵ) | "0").
    Digits → Digit | Digit (Digits | ϵ).
     Digit → "0" | Onenine.
   Onenine → "1" | "2" | "3".
  Fraction → ϵ | "." Digits.
  Exponent → ϵ | "E" Sign Digits | "e" Sign Digits.
      Sign → ϵ | "+" | "-".
```

Listing 7.11: JsonParser Grammar inferred by the learning based approach.

numbers should look like are learned and the associated terminals are added. Until then, the learned grammar only accepts strings and numbers, which are given as an initial seed input and nothing more.

We must remark that we have rarely used optimizations in our implementation, which leaves much room for improvement in future work. Possible performance improvements include (i) using hash-tables to store previously seen membership-queries instead of a plain-text list, (ii) replacing the early-parser used to determine whether a grammar

```
    URLParser → HTTPaddress
    URLParser → FTPaddress
    URLParser → TELNETaddress
    URLParser → GOPHERaddress
  HTTPaddress → "http://" Hostport "/" Path "?" Search
   FTPaddress → "ftp://" Login "/" Path
TELNETaddress → "telnet://" Login
GOPHERaddress → "gopher://" Hostport "/" Search
        Login → User ":" Password "@" Hostport
     Hostport → Host ":" Port
         Host → Hostname
         Host → Hostnumber
     Hostname → IAlpha
   Hostnumber → Digits "." Digits "." Digits "." Digits
         Port → Digits
         Path → XAlphas
       Search → XAlphas
       Search → XAlphas Search
         User → XAlphas
     Password → XAlphas
       XAlpha → Safe
       XAlpha → Extra
      XAlphas → XAlpha
       IAlpha → Alpha XAlphas
        Alpha → "a"
        Digit → "1"
         Safe → "&"
         Safe → ""
        Extra → "!"
       Digits → Digit
```

Listing 7.12: URLParser Extracted Grammar Fraction

produces a given word with something more efficient, (iii) using paralellization to speed up fuzzing, and (iv) simultaneously learning the different rules of the seed grammar.

```
    URLParser → ( HTTPaddress "" | FTPaddress "" |
                TELNETaddress "" | GOPHERaddress "").
  HTTPaddress → "h" "t" "t" "p" ":" "/" "/" (Hostport ""
                | Hostport ("?" Search "" | "") | Hostport
                ("?" Search "" | "/" (Path "" | Path ("?"
                Search "" | "") ) | "") ) .
   FTPaddress → "f" "t" "p" ":" "/" "/" Login "/" Path "".
TELNETaddress → "t" "e" "l" "n" "e" "t" ":" "/" "/" Login "".
GOPHERaddress → "g" "o" "p" "h" "e" "r" ":" "/" "/"
                (Hostport "" | Hostport ("?" Search "" |
                "") | Hostport ("?" Search "" | "/" (Gtype
                "" | Gtype ("?" Search "" | "") | Gtype
                (Selector "" | Selector ("?" Search "" |
                "") | "?" Search "" | "")) | "")) .
        Login → (User "@" Hostport ""  | User ("@" Hostport ""
                | ":" Password "@" Hostport "" ) ).
     Hostport → (Host ""  | Host (":" Port ""  | "") ).
         Host → (Hostnumber ""  | Hostname "" ).
     Hostname → (IAlpha ""  | IAlpha ("." Hostname ""
                | "") ).
   Hostnumber → Digits "." Digits "." Digits "." Digits "" .
         Port → Digits "" .
     Selector → (Path ""  | "").
         Path → (XAlphas ""  | XAlphas ("/" Path ""  | "") ).
       Search → (XAlphas ""  | XAlphas ("+" Search ""
                | "") ).
         User → XAlphas "" .
     Password → XAlphas "" .
        Gtype → XAlpha "" .
       XAlpha → (Safe ""  | Extra ""  | Digit ""
                | Alpha "" ).
      XAlphas → (XAlpha ""  | XAlpha (XAlphas ""  | "") ).
       IAlpha → (Alpha ""  | Alpha (XAlphas ""  | "") ).
        Alpha → ("c" ""  | "b" ""  | "a" ""  | "C" ""
                | "B" ""  | "A" "" ).
        Digit → ("3" ""  | "2" ""  | "1" ""  | "0" "" ).
         Safe → ("_" ""  | "-" ""  | "&" ""  | "$" "" ).
        Extra → (";" ""  | "," ""  | "*" ""  | ")" ""
                | "(" ""  | "!" "" ).
       Digits → (Digit ""  | Digit (Digits ""  | "") ).
```

Listing 7.13: URLParser Grammar inferred by the learning based approach.

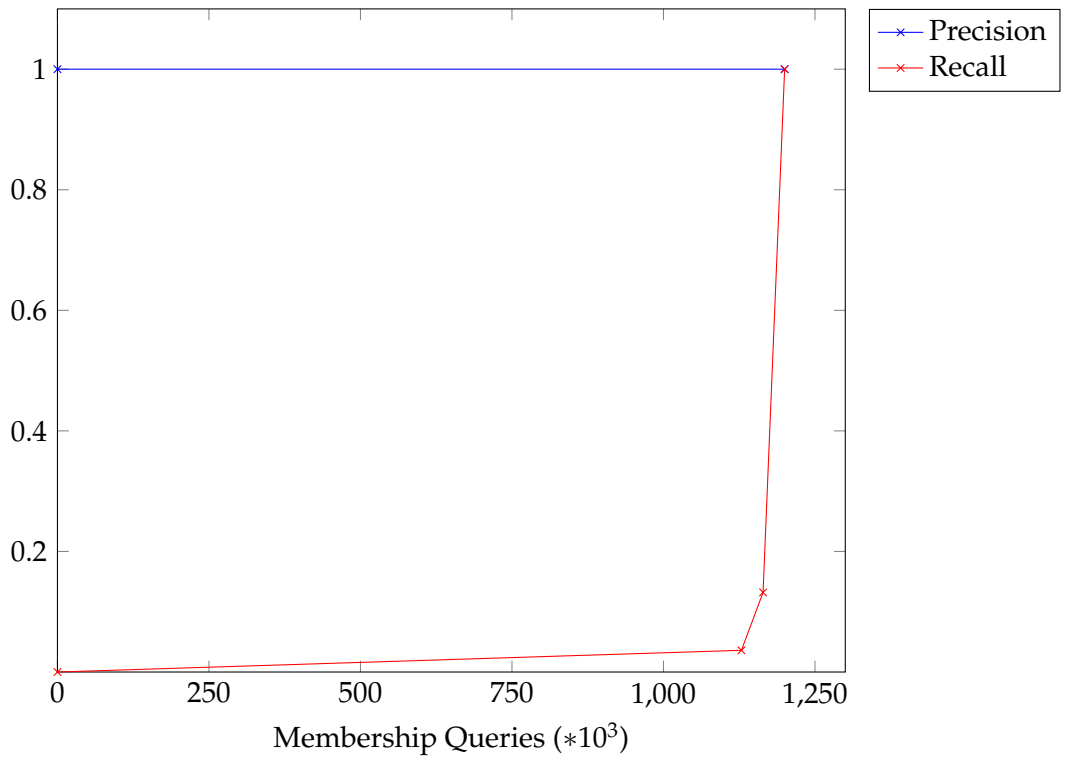


Figure 7.6: URL Learning process

We have shown through extensive experimentation that our method can learn grammars with consistently high levels of recall and precision from recursive descendent parsers, even for sophisticated languages such as JSON. Notably, these results were obtained starting with a couple of arbitrary seed inputs. By contrasts, other state of the art methods need much higher numbers of seed inputs.

8 Conclusion and Outlook

To conclude this thesis, we begin by presenting related work and a critical discussion of our findings, including an analysis of the limitations inherent to the concept of *Fuzzing-based Grammar Inference*. We then summarize our conclusions and highlight potential directions for future research through a set of open questions. The thesis closes with a brief epilogue.

8.1 Related Work

Here we aim to provide an extensive introduction to related work. We start by presenting the state of the art in the field of *grammar-based fuzzing* as enabling this field is the main motivation of this thesis. Next, we discuss the topic of *grammar inference*, where we start with the fundamental results on which this field of research is based upon, followed by a presentation of the most relevant related work with respect to this thesis.

First, let us give a quick introduction to the most prominent grammar-based fuzzers currently available:

- *LangFuzz* [7] parses a set of seed programs into abstract syntax trees (ASTs) using a given context-free grammar. It introduces mutations by replacing subtrees within the ASTs with subtrees from other programs' ASTs, while ensuring the resulting program remains syntactically correct.
- *NAUTILUS* [8] takes a grammar as input and generates new inputs by applying mutations that preserve syntactic validity. A feedback loop that measures code coverage is used to drive input generation to produce new inputs that increase code coverage.

- *Skyfire* [9] combines the two approaches described above. It first constructs a grammar based on a given set of input samples. It then uses a coverage-based feedback loop to guide input generation toward inputs that increase code coverage.
- *EvoGFuzz* [10] takes as input a grammar and assigns probabilities to the production rules. Applying an evolutionary algorithm, it drives input generation to produce inputs with a higher chance of exposing defects by dynamically adapting the probabilities of the production rules.
- *Inputs from Hell* [32, 33] is a technique that takes a grammar and a set of commonly observed samples as input. These samples are then parsed, and probabilities are assigned to the grammar that reflects these samples. Now, the authors propose to invert these probabilities to drive input generation to produce new *uncommon* inputs.

As we can see above, all prominent tools and methods in the field of *grammar-based fuzzing* depend on a given grammar. In most cases, this grammar is context-free. In the remainder of this section we want to discuss advancements made in the field of *grammar inference* that aims to produce such a grammar by various methods.

The fundamentals of learning languages have been established by Gold's theorem on learning languages in the limit [4] in 1967. This work addresses the general capabilities and limitations of learning languages solely based on observed samples. It establishes that identifying a grammar that correctly produces some context-free language is not possible solely based on positive samples. Specifically, Gold's theorem demonstrates that:

Theorem 1 (Gold, 1967)**.** *No algorithm can identify the correct grammar of a superfinite class of languages in the limit using only positive examples. A superfinite class of languages is any class of formal languages that includes all finite languages and at least one infinite language, for example, the class of context-free languages. However, identification becomes possible when both positive and negative examples (counterexamples) are available.*

In 1984 Valiant proposed a *Probably Approximately Correct (PAC) Learning* algorithm [34] to infer context-free languages. In a PAC scenario, we randomly draw new positive and negative examples from the language and try to infer a grammar that correctly produces this language. Valiant established that, due to the complexity and ambiguity of context-free grammars, converging to the correct grammar is very challenging. It is important

to note that Valiants observations are primarily relevant to a black-box learning setting, where no program analysis or internal structure of the language-generating system is considered. This is especially relevant to our work as we aim to infer our grammar from random inputs produced by a grammar-based fuzzer which closely resembles the PAC scenario, but unlike the pure PAC setting, we also have the ability to leverage additional information such as program analysis.

Next, Angluin explored the usage of a teacher that supports a learner in his task of inferring regular and context-free languages in [2]. Angluin established that the inference of regular languages based upon a teacher that can answer membership-queries (i.e. wheter a given word belongs to a language or not) and equivalence-queries (i.e. wheter a given automata correctly accepts the language and if not provide a counterexample) is possible. Note that this distinguishes Angluins work from Valiants by explicitly providing counterexamples to the current hypothesis about the language, whereas Valiants framework only uses negative examples without explicit counterexamples. However, this does not apply to context-free languages. In such a setting, Angluin established that the teacher has to be able to answer a different kind of membership-query. This time, the teacher has to be able to determine if, given some nonterminal and some word, this word can be produced by the grammar we want to learn starting from the given non-terminal symbol.

It is not feasible to cover every publication that has introduced new or improved methods for grammar inference. Instead, we focus on the most recent contributions directly related to our work. For a comprehensive overview of the field, we refer the reader to Wieczoreks book on Grammatical Inference [35], which surveys relevant foundational and contemporary contributions. Let us start with *black-box* approaches that aim to infer a grammar purely from positive and negative samples. Usually, these algorithms use the target program as an oracle to determine whether some input belongs to the language they aim to learn or not. We then see that more advanced tools incorporate some sort of additional program analysis to guide grammar inference, hence *white-box* approaches.

The V-Star tool [36] infers a context-free language solely from sample program inputs. V-Star deduces the production rules of the grammar based on nested patterns. However, the work is limited to Visibly PushDown Languages (VPLs), a subclass of context-free languages where the input symbols dictate stack operations. This constraint gives VPLs stronger structural guarantees and makes certain analysis tasks more tractable. Nevertheless, it restricts the approachs applicability to languages with strictly input-driven nested

patterns, excluding more complex context-free languages. The ARVADA algorithm [37] infers a context-free grammar from input samples. ARVADA initially constructs parse trees from positive samples only, which results in a minimal hierarchical structure. These structures are then generalized by adding new layers to approximate recursive patterns and are validated against an oracle.

GLADE [18] synthesizes a grammar that precisely captures the given set of input samples. They apply a step-by-step generalization approach in which the generalization is validated in each step by randomly generating and executing new inputs from the generalized grammar. The generalization is only retained if these inputs are accepted by the program. Based on their findings while developing GLADE [18], Bastani et al. developed REINAM [38] which in addition uses a symbolic execution engine to find a good initial set of inputs to reduce dependence on a high quality set of seed inputs. Furthermore, they extend the synthesized grammar to a probabilistic context-free grammar that considers and adapts the assigned probabilities during generalization.

AUTOGRAM [21] takes a set of positive samples as input and uses dynamic data flow analysis to track how a program processes these inputs. Based on identified patterns and structures, a context-free grammar is then constructed. This grammar is further enhanced by methods to identify optional elements or repetitions among other syntactical patterns. These generalizations are validated against the program. PYGMALION [39] generates its own set of seed inputs. It does so by using dynamic tainting to track all comparisons against an initially empty string. Iteratively, the string is filled with expected values until it is accepted by the program. These inputs are then used by a method similar to AUTOGRAM [21] to generalize a grammar. MIMID [17] uses dynamic control flow analysis to track how a program handles certain positive input samples. The results of this analysis are then combined to form a general context-free grammar. The approach is specifically tailored to infer grammars from programs that use a recursive-descent parsing implementation. The downside of this approach is that it is highly dependent on a good set of seed inputs. Elements of the language that are not represented in the initial set of seed inputs cannot be identified.

Finally, Bettscheider et al. have recently proposed STALAGMITE [40] that uses *symbolic parsing* to synthesize a set of seed inputs from the recursive descent parser and subsequently generalize them to a grammar. The symbolic parsing method offers a significant advantage over the MIMID method [17]. This leads to overall better results with regard to

precision and recall compared to related work. A comparison to the work presented in this thesis is not available.

8.2 Discussion

Before presenting our conclusions, we first place the results obtained in this thesis within a broader context. Additionally, we provide a direct comparison with related work to highlight the significance and originality of our contributions. A summary of the experimental results is provided in Table 8.1. In addition, Table 8.1 shows experiments of related work where a direct comparison is feasible, namely Autogram [21] and Mimid [17]. We have ordered Table 8.1 by the different languages that have been targeted by our inference algorithms. Each language has been clearly defined earlier in Section 4. For each language, we present the results of inferring an approximate version of the original language using various methods, including those from related work. The first column indicates the method used and its source publication. The second and third columns show results in terms of recall and precision, respectively. The fourth column reports the number of seed inputs required to approximate the target language. Note that for Autogram [21] and Mimid [17] we do not have exact numbers for the required number of seed inputs, but we know that this number has to be greater than zero. The fifth column shows the total number of membership queries made while inferring the target language. Each membership query requires a single execution of the target program. Finally, column six shows the time needed to infer the grammar.

In summary, the grammar inference approaches presented in Chapters 5 to 7 each yield promising results. However, their relative suitability depends on the specific task requirements and the resources available. Here, we provide an informed assessment of which methods are most appropriate for different scenarios. This assessment is based on insights derived from the experimental results, rather than on a formal comparative analysis. To structure the discussion, we identify and examine the following key factors individually:

- Availability of prior knowledge
- Availability of computational resources

MathExpr					
Method	*Recall*	*Precision*	*seeds*	*queries*	*time*
Autogram [21]	0.0	0.303	*n*	n/a	n/a
Mimid [17]	0.927	0.875	*n*	n/a	n/a
Grammar Refinement [12]	1.0	1.0	0	n/a	2.7s
FbGL [14]	1.0	1.0	1	15 361	7s
Grammar Forge [15]	1.0	1.0	0	7 760	n/a

AdvMathExpr					
Method	*Recall*	*Precision*	*seeds*	*queries*	*time*
FbGL [14]	1.0	1.0	1	24 100	9s

Mail					
Method	*Recall*	*Precision*	*seeds*	*queries*	*time*
Grammar Refinement [12]	1.0	1.0	0	n/a	34.7s
FbGL [14]	1.0	1.0	1	28 227	11s
Grammar Forge [15]	1.0	1.0	0	258 354	n/a

Hello					
Method	*Recall*	*Precision*	*seeds*	*queries*	*time*
FbGL [14]	1.0	1.0	1	9 927	58s
Grammar Forge [15]	1.0	1.0	0	56 979	n/a

JSON					
Method	*Recall*	*Precision*	*seeds*	*queries*	*time*
Autogram [21]	0.0	0.538	*n*	n/a	n/a
Mimid [17]	0.93	0.987	*n*	n/a	n/a
Grammar Refinement [12]	0.926	1.0	0	n/a	30m 18.8s
FbGL [14]	1.0	1.0	1	841 514	22m 40s
Grammar Forge [15]	1.0	1.0	0	1 011 157	n/a

URL					
Method	*Recall*	*Precision*	*seeds*	*queries*	*time*
Autogram [21]	1.0	1.0	*n*	n/a	n/a
Mimid [17]	0.964	1.0	*n*	n/a	n/a
FbGL [14]	1.0	1.0	3	1 270 165	40m 49s
Grammar Forge [15]	1.0	1.0	0	2 719 994	n/a

Table 8.1: Experiment Results (blue) in comparison to related work (red).

- Availability of code analysis methods
- Empirical effectiveness of results

Assume that we have good knowledge about the target language, including a good set of seed inputs that also give a good coverage of corner cases. In such a scenario, Autogram and Mimid are best used as they are able to provide a good approximation based on the set of seed inputs fast. If the given set of seed inputs is not sufficient to get a good approximation, we can further refine the results of Mimid and Autogram by applying our own learning-based method described in [14] to automatically identify missing parts of the grammar. This would also work if only a very small set of seed inputs is known, but will lead to an increase in runtime of our learning-based method presented in Section 7.2.2. Finally, if no prior knowledge is available, we have to apply our grammar approximation method [15] or our refinement-based method [12] as they do not depend on providing any seed input at all. The trade-off, however, is increased runtime.

This leads us directly to the next factor that we have to consider: The availability of computational power. Seed grammar extraction for our approximation-based method is linearly bounded by the size of the AST or CFG being analyzed, as detailed in Section 5.3. However, in the worst case, the approximation-based grammar inference algorithm has superexponential runtime complexity as established in Section 5.2.5. Next, extracting a seed grammar for the refinement-based method yields, in the worst case, a grammar with an exponential number of rules, as described in Section 6.3. As shown in Section 6.2.1, the runtime complexity of the refinement algorithm is exponential in the nesting depth of the seed grammar. Note that this nesting depth is not increased by a higher number of rules added during seed grammar extraction. Finally, seed grammar extraction for our learning-based grammar inference method can be done in linear time using the proposed dynamic symbolic execution method as established in Section 7.3.3. As shown in Section 7.2.3, the learning-based grammar inference method finishes in polynomial time, making it the most efficient of our proposed methods. However, all the methods discussed are well-suited for parallelization. With Mimid and Autogram, we can calculate the grammar fractions for each given seed input simultaneously. In our refinement-based approach, the testing of words until a certain length may also be executed simultaneously. With our learning-based method, we are able to calculate the resulting expressions for each rule independently of the others. Finally, our approximation-based approach enables us to follow separate branches that have been discovered in parallel. This is supported by our experiments,

which demonstrate that grammar inference is feasible for real-world inputs such as URLs and JSON, despite the theoretically high runtime complexity of our proposed methods.

Next, we look at the available code analysis methods. For our grammar inference algorithms, that does not make a difference, as all of them rely on the ability to instrument the source code of the target program with the exception of the refinement-based method, which does not rely on any additional program analysis at all. The methods used for seed grammar extraction have a strong dependence on code analysis techniques. Let us start with the two methods for extracting a core grammar structure that is used in our approximation-based grammar inference method. As described in Section 5.3, both methods extract a seed grammar based on a given AST or CFG, respectively. The AST or CFG used is extracted by program analysis, in our case in the form of the eKnows tool [26]. Constructing the expanded core grammar, required by our refinement-based inference method, does not rely on code analysis. Instead, it depends on one of the core grammar extraction methods, such as CFG or AST analysis, as the expanded core grammar is derived directly from a given core grammar. Finally, extracting a grammar fraction used by our learning-based grammar inference method can be done directly via dynamic symbolic execution or via a detour from an expanded core grammar, where the detour involves reducing the expanded core grammar so that it accepts only a given set of inputs. In this case, since the grammar fraction is extracted from an expanded core grammar, which in turn is derived from a core grammar, the requirements for program analysis needed to extract a core grammar also apply here. Specifically, this involves program analysis that produces the programs CFG or AST as described above for extracting a core grammar.

Finally, we discuss the empirical effectiveness of our results, defining it as the extent to which inferred grammars achieve high recall and precision (as described in Section 2.3.1) relative to the target grammar. As we can see in Table 8.1, all of our proposed methods provide nearly perfect results with respect to the precision and recall of the inferred grammar. However, our refinement-based method may incorrectly remove a rule which would be needed to fully produce the target language, as it was the case in the JSON experiment described in Section 6.4. Apart from this issue, the results demonstrate the effectiveness of our method in following the structure of the target program to produce a good approximation of the target grammar. Next, our learning-based method works as expected and, given some good starting point in the form of a seed input, it is capable

of approximating the target language, yielding perfect results in regard to precision and recall. However, as shown in Section 7.4, the learned grammar may contain unexpected structures. Specifically, some rules may be unnecessarily nested due to the transformation from a nondeterministic finite state automaton to a regular expression, as described in Section 7.2.2. Our approximation-based method yields perfect results in regard to precision and recall while directly following the structure of the target grammar to produce the target language.

Next, we compare our methods to related work. To some extent, that has already been discussed. Here, however, we want to summarize our findings. First, in regard to runtime cost, the related work [21, 17] offers much more efficient methods. However, this comes at the cost of having to provide a good set of seed inputs. In addition, we argue that runtime efficiency is not the primary concern. It is far more important that the resulting grammar is of high quality. Since grammar inference is performed only once, and the resulting grammar is used in subsequent processing, runtime performance is of secondary importance. We have found that the methods we provide in this work offer much better results with regard to precision and recall (see Table 8.1) wherever a direct comparison is applicable. We achieve these results using significantly fewer seed inputs, and, in the case of the approximation-based and refinement-based methods, no seed input is required at all. In summary, our methods deliver higher-quality results in terms of precision and recall, at the cost of increased runtime, while remaining independent of the need for a good set of seed inputs.

8.2.1 Limitations

To conclude this section, we briefly discuss the main limitations of our methods beyond those already noted in our evaluation and experiments. Ideally, one would like to take an arbitrary program and then to be able to infer a grammar for its input language using fully automated procedures. However, the parsing subroutine of such a program that actually determines whether a given input is valid has to be known to be able to apply our methods. In all of our experiments, we have assumed that this parsing subroutine is isolated from the rest of the program. However, in a real world setting, such an assumption will most likely not hold. Identifying and isolating the parsing subroutine of some program is not trivial for large and complex systems. In practice, the usage of dynamic taint analysis

may be applied to identify parts of the target program associated with handling the given input [16].

All of our methods for extracting a seed grammar from source code rely on tracking function calls within the source code of the parser, be it via analyzing the abstract syntax tree, control flow analysis, or dynamic symbolic execution. The same applies to the methods for approximation-based and learning-based grammar inference. This limits our approach to how the parser is implemented. Let us highlight our reasoning by providing an example. Given some context-free grammar with rule `S` $\rightarrow$ `( S )`. A parser can now implement this rule by recursive calls or by utilizing some loop that remembers the opening brackets and counts if the same number of closing brackets is present. For our methods, the latter case of using loops within a function is not considered. Such a case will be treated as a black-box, as it does not include a call to itself, and as such will not yield good results. However, the former case of utilizing recursive calls gives good results. Solving this issue would mean that the core of our proposed analysis methods needs to operate at the code branch level, accounting for loops among other structures rather than being based solely on simple function calls, as is currently the case. Thus, we can conclude that *Fuzzing-based Grammar Inference* currently is ***limited to a recursive descending parser***.

This leads us to the final limitation of our approaches: All of them have high runtime complexity. However, as we have shown in our experiments, we can apply our methods successfully to real-world examples nonetheless. The (theoretically) high runtime complexity implies it is impossible to approximate *all* arbitrarily complex context-free languages, even if their parsing subroutine is implemented in a recursive-descending way. This is applicable for the refinement-based and the approximation-based methods. The learning-based approach, on the other hand, has a polynomial runtime. However, this does not take into account that we have to provide a teacher that is capable of answering equivalence queries (see Section 7.2), or, in other words, is capable of providing counterexamples if there are any to a given automaton. In theory, we are unable to answer such a query for infinite languages, as this would mean that we have to test an infinite number of words. As described in Section 7.2, we address this problem by stopping after a finite number of steps, and assume that a counterexample does not exist if we have not found it up to that point.

8.3 Conclusion

The research presented in this thesis introduces a novel approach to grammar inference, namely *fuzzing-based grammar inference*. By leveraging fuzzing techniques to generate positive and negative examples and employing program analysis to guide grammar inference, the proposed approaches have demonstrated significant improvements in both precision and recall over existing methods. The three core methods - approximation, refinement, and learning-based grammar inference - each provide distinct strategies for grammar inference.

In summary, this thesis, as demonstrated by our evaluation, proposes grammar inference methods that each achieve high precision and recall across diverse language benchmarks. The choice of method is context-dependent, balancing trade-offs between seed input availability, computational resources, and robustness. Although related work offers a more efficient runtime in cases with substantial seed inputs, our methods provide superior precision and recall without such prerequisites. However, since grammar inference is typically performed only once and the resulting grammar is used for subsequent applications, such as grammar-based fuzzing, runtime performance is not a major concern.

This thesis presents several key contributions in the field of grammar inference, including the integration of fuzzing techniques to facilitate grammar learning, a systematic framework for utilizing seed grammars, and the identification of methods that make use of less sophisticated tools for program analysis, such as instrumentation. Additionally, we have developed methods that significantly reduce or eliminate dependence on prior input data, employing adapted learning algorithms to extract program input grammars even in the absence of seed inputs. Finally, our proposed approaches consistently achieve higher accuracy in grammar inference compared to existing methods, underscoring their effectiveness in diverse scenarios.

Finally, let us address the objectives set in the Introduction in Section 1.1. Objective *O1* - Develop methods to extract seed grammars, as well as objective *O2* - Develop methods that infer a context-free grammar for the input language of a program from its source code and seed grammar, have been clearly achieved by establishing methods for approximation-, refinement- and learning-based grammar inference in Chapter 5 to Chapter 7, respectively. Each of the proposed methods for fuzzing-based grammar inference comes with an appropriate method for seed grammar extraction. The experiment setup used in the

evaluation of these methods satisfies Objective *O3* - Design and implement a reusable evaluation framework to assess the performance of the proposed methods. To fulfill objective *O2.1* - Ensure the applicability of developed methods to real world applications, we have used a broad list of languages in our experiments, including widely adopted languages such as JSON and URL. As we achieve good results in terms of precision and recall, as supported by our experiments, we can conclude that both Objective *O2.1* and Objective *O2.2* - The inferred grammar should be of high quality, are achieved as well.

Before concluding this thesis, let us briefly outline open questions. At this point, we do not want to focus on obvious technical improvements for individual methods, such as further improving the robustness of the method xy. Instead, we would like to highlight an important limitation of our current approach: its reliance on Recursive-Descent Parsing. While our evaluation demonstrates that the proposed methods already yield good and robust results, their broader impact depends on extending support beyond a single parsing technique. As such, this work serves as a foundation for future research aimed at building upon and extending the ideas presented in this thesis.

Next, let us discuss seed grammar extraction. Our method for extracting a grammar fraction using dynamic symbolic execution is not dependent on recursive-descent parsing. However, extraction of a core grammar by abstract syntax tree or control flow analysis is. The issue could be addressed by executing the same analysis in a more fine-grained way that, besides function calls, considers other control statements as well. This may seem trivial, but we could run into other problems that arise due to an increase in the total number of elements in the extracted grammars, such as decreased runtime performance. However, since we are addressing open questions, this should be recognized as an interesting challenge for future research.

Regarding grammar inference itself, the issue lies in how we use code instrumentation to identify and address ambiguous paths in the inferred grammar. This does not apply to our refinement-based approach as it is a black-box approach, and only partially applies to the approximation-based approach. There, instrumentation is used to find such ambiguous paths and discard them early. This serves only to omit the need to fully explore such paths, which speeds up the approximation algorithm substantially. In summary, the instrumentation here is (theoretically) optional. However, our learning-based approach is highly dependent on this information. If ambiguous paths are not identified, the learning algorithm cannot correctly identify the target language. One solution could be, again,

to treat individual code branches as the core element of the method instead of whole functions. Now, our learning algorithm could be applied to learn the language accepted within such code branches (e.g. the language inside a loop). Again, this poses a significant challenge and may open up other issues that have not been explored yet.

On a final note, an interesting open question is the potential impact of Large Language Models (LLMs) on the field of grammar inference. Early work, such as HyGenar [41], uses an LLM to extract an initial grammar from a set of given examples, followed by a genetic algorithm used for fine-tuning. Due to their increasing capabilities in analyzing and understanding source code, it remains to be seen to what extent they can infer grammars directly from source code, rather than generalizing grammars solely from input examples.

8.4 Epilogue

This thesis documents the results of a five-year long research endeavor that started with a simple question: Can we reproduce the results achieved by Andreas Zeller and his team in [21, 17] using our internal dynamic symbolic execution framework integrated into eKnonws [26]. Based on some initial success, and several years later, we now have developed several new methods that try to achieve the same goal: Inferring a context-free grammar from source code.

The decision that I wanted to do my Ph.D. was taken years earlier when I started working as a researcher. However, after some initial publications in the field of automated security testing, I have still not been able to decide on the exact topic. It all started with in-depth discussions with my colleague Flavio Ferrarotti that lasted days on the topic of the learnability of context-free languages and how modern program analysis methods may be applied. Based on these discussions, I was able to produce my first thesis result in this subject, which is covered in Chapter 6. Although this refinement-based approach works well, I was not very satisfied with the exponential runtime complexity and with the fact that we have made some strong assumptions about our capabilities. To counteract this and due to a personal fascination with learning algorithms, the idea of using some learning approaches to solve the remaining issues grew. At this point, my supervisor Robert Wille greatly assisted me as a sparring partner to refine that idea down to a practical

implementation. After a very difficult time implementing and testing and failing with several algorithms, we finally came up with the method described in [13]. The main contribution lies in adapting the learning algorithm L^* to our practical scenario, and integrating instrumentation into the learning process to guide it. Being rewarded with the Best Paper Award for this work was a great motivation boost. With the Award came the invitation of the Journal of Computer Languages to publish our approach there. After further improving and adding another method to extract the seed grammar we need for our learning-based approach led to [14]. All our findings on learning-based grammar inference are presented in Chapter 7. Finally, after overcoming the initial struggle to find my exact research field, I reached a point where I struggled with the question which of the various ideas in my head I want to follow next. After several experiments, we finally implemented the approach described in [15] which led to our final publication. This idea of approximation-based grammar inference is presented in Chapter 5. At this point, we decided that it is time to wrap everything up neatly in this thesis. Fortunately, as a full-time researcher, I hope to have the opportunity to follow up on my ideas further and actually work on the open questions described earlier.

Bibliography

[1] Matthew Young-Lai. "Grammar Inference". In: *Encyclopedia of Database Systems*. Ed. by LING LIU and M. TAMER ÖZSU. Boston, MA: Springer US, 2009, pp. 1256–1260. ISBN: 978-0-387-39940-9. DOI: 10.1007/978-0-387-39940-9_182. URL: https://doi.org/10.1007/978-0-387-39940-9_182 (cit. on p. 1).

[2] Dana Angluin. "Learning Regular Sets from Queries and Counterexamples". In: *Inf. Comput.* 75.2 (1987), pp. 87–106 (cit. on pp. 1, 2, 68, 77, 78, 80, 98).

[3] Benedikt Bollig et al. "Angluin-Style Learning of NFA". In: *IJCAI*. 2009, pp. 1004–1009 (cit. on pp. 1, 66, 69, 71, 76).

[4] E.Gold. "Language Identification in the Limit". In: *Inf. Control.* 10.5 (1967), pp. 447–474 (cit. on pp. 1, 4, 16, 97).

[5] Xiaoqi Zhao et al. "A systematic review of fuzzing". In: *Soft Computing* 28.6 (2024), pp. 5493–5522 (cit. on p. 2).

[6] Patrice Godefroid, Michael Y. Levin, and David Molnar. "SAGE: Whitebox Fuzzing for Security Testing: SAGE has had a remarkable impact at Microsoft." In: *Queue* 10.1 (Jan. 2012), pp. 20–27. ISSN: 1542-7730. DOI: 10.1145/2090147.2094081. URL: https://doi.org/10.1145/2090147.2094081 (cit. on p. 3).

[7] Christian Holler, Kim Herzig, and Andreas Zeller. "Fuzzing with code fragments". In: *Proceedings of the 21st USENIX Conference on Security Symposium*. Security'12. Bellevue, WA: USENIX Association, 2012, p. 38 (cit. on pp. 3, 96).

[8] Cornelius Aschermann et al. "NAUTILUS: Fishing for Deep Bugs with Grammars". In: *26th Annual Network and Distributed System Security Symposium, NDSS 2019, San Diego, California, USA, February 24-27, 2019*. The Internet Society, 2019. URL: https://www.ndss-symposium.org/ndss-paper/nautilus-fishing-for-deep-bugs-with-grammars/ (cit. on pp. 3, 96).

[9] Junjie Wang et al. "Skyfire: Data-Driven Seed Generation for Fuzzing". In: *2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22-26, 2017*. IEEE Computer Society, 2017, pp. 579–594. DOI: 10.1109/SP.2017.23. URL: https://doi.org/10.1109/SP.2017.23 (cit. on pp. 3, 97).

[10] Martin Eberlein et al. "Evolutionary Grammar-Based Fuzzing". In: *Search-Based Software Engineering - 12th International Symposium, SSBSE 2020, Bari, Italy, October 7-8, 2020, Proceedings*. Ed. by Aldeida Aleti and Annibale Panichella. Vol. 12420. Lecture Notes in Computer Science. Springer, 2020, pp. 105–120. DOI: 10.1007/978-3-030-59762-7_8. URL: https://doi.org/10.1007/978-3-030-59762-7%5C_8 (cit. on pp. 3, 97).

[11] Bachir Bendrissou, Cristian Cadar, and Alastair F. Donaldson. "Grammar Mutation for Testing Input Parsers (Registered Report)". In: *Proceedings of the 2nd International Fuzzing Workshop, FUZZING 2023, Seattle, WA, USA, 17 July 2023*. Ed. by Marcel Böhme et al. ACM, 2023, pp. 3–11. DOI: 10.1145/3605157.3605170. URL: https://doi.org/10.1145/3605157.3605170 (cit. on p. 3).

[12] Hannes Sochor and Flavio Ferrarotti. "A Refinement Based Algorithm forăLearning Program Input Grammars". In: *From Data to Models and Back*. Ed. by Juliana Bowles, Giovanna Broccia, and Roberto Pellungrini. Cham: Springer International Publishing, 2022, pp. 138–156. ISBN: 978-3-031-16011-0 (cit. on pp. 6, 7, 21, 23, 39, 50, 101, 102).

[13] Hannes Sochor, Flavio Ferrarotti, and Daniela Kaufmann. "Fuzzing-Based Grammar Inference". In: *Model and Data Engineering*. Ed. by Philippe Fournier-Viger, Ahmed Hassan, and Ladjel Bellatreche. Cham: Springer Nature Switzerland, 2023, pp. 72–86. ISBN: 978-3-031-21595-7 (cit. on pp. 6, 7, 21, 23, 66, 91, 109).

[14] Hannes Sochor, Flavio Ferrarotti, and Daniela Kaufmann. "Fuzzing-based grammar learning from a minimal set of seed inputs". In: *J. Comput. Lang.* 78 (2024), p. 101252. URL: https://doi.org/10.1016/j.cola.2023.101252 (cit. on pp. 6, 7, 8, 21, 23, 39, 43, 66, 101, 102, 109).

[15] Hannes Sochor, Flavio Ferrarotti, and Robert Wille. "GrammarForge: Learning Program Input Grammars forăFuzz Testing". In: *Software Engineering and Formal Methods*. Ed. by Alexandre Madeira and Alexander Knapp. Cham: Springer Nature Switzerland, 2025, pp. 272–289. ISBN: 978-3-031-77382-2 (cit. on pp. 6, 7, 21, 23, 31, 39, 43, 101, 102, 109).

[16] Matthias Höschele and Andreas Zeller. "Mining input grammars from dynamic taints". In: *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*. ASE '16. Singapore, Singapore: Association for Computing Machinery, 2016, pp. 720–725. ISBN: 9781450338455. DOI: 10.1145/2970276.2970321. URL: https://doi.org/10.1145/2970276.2970321 (cit. on pp. 6, 105).

[17] Rahul Gopinath, Björn Mathis, and Andreas Zeller. "Inferring Input Grammars from Dynamic Control Flow". In: *CoRR* abs/1912.05937 (2019) (cit. on pp. 6, 81, 82, 99, 100, 101, 104, 108).

[18] Osbert Bastani et al. "Synthesizing program input grammars". In: *PLDI*. ACM, 2017, pp. 95–110 (cit. on pp. 6, 16, 99).

[19] Margaret Aycinena, Mykel J Kochenderfer, and David Carl Mulford. "An evolutionary approach to natural language grammar induction". In: *Final Paper Stanford CS224N June* (2003) (cit. on p. 6).

[20] Hari Mohan Pandey, Ankit Chaudhary, and Deepti Mehrotra. "Grammar induction using bit masking oriented genetic algorithm and comparative analysis". In: *Appl. Soft Comput.* 38 (2016), pp. 453–468. DOI: 10.1016/J.ASOC.2015.09.044. URL: https://doi.org/10.1016/j.asoc.2015.09.044 (cit. on p. 6).

[21] Matthias Höschele and Andreas Zeller. "Mining input grammars with AUTOGRAM". In: *ICSE (Companion Volume)*. IEEE Computer Society, 2017, pp. 31–34 (cit. on pp. 6, 81, 82, 99, 100, 101, 104, 108).

[22] *Programmable controllers - Part 3: Programming languages*. Standard. Geneva, CH: International Electrotechnical Commision, 2013 (cit. on p. 7).

[23] John E. Hopcroft and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages and Computation*. Addison-Wesley, 1979. ISBN: 0-201-02988-X (cit. on pp. 10, 14, 69).

[24] Peter Linz. *An introduction to formal languages and automata, 4th Edition*. Jones and Bartlett Publishers, 2006. ISBN: 978-0-7637-3798-6 (cit. on p. 12).

[25] International Organization for Standardization. *ISO/IEC 14977:1996(E) Information technology Syntactic metalanguage Extended BNF*. https://www.iso.org/standard/26153.html. 1996 (cit. on p. 14).

[26] Michael Moser and Josef Pichler. "eknows: Platform for Multi-Language Reverse Engineering and Documentation Generation". In: *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 2021, pp. 559–568 (cit. on pp. 41, 43, 81, 103, 108).

[27] Frances E. Allen. "Control flow analysis". In: *Proceedings of a Symposium on Compiler Optimization, Urbana-Champaign, Illinois, USA, July 27-28, 1970*. Ed. by Robert S. Northcote. ACM, 1970, pp. 1–19. URL: https://doi.org/10.1145/800028.808479 (cit. on p. 43).

[28] Julien Romero. "Pyformlang: An Educational Library for Formal Language Manipulation". In: *SIGCSE '21: The 52nd ACM Technical Symposium on Computer Science Education*. Ed. by Mark Sherriff et al. ACM, 2021, pp. 576–582. DOI: 10.1145/3408877.3432464 (cit. on p. 60).

[29] Jacques Sakarovitch. *Elements of Automata Theory*. Cambridge University Press, 2009. ISBN: 978-0-521-84425-3 (cit. on p. 71).

[30] Dean N. Arden. "Delayed-logic and finite-state machines". In: *2nd Annual Symposium on Switching Circuit Theory and Logical Design, Detroit, Michigan, USA, October 17-20, 1961*. IEEE Computer Society, 1961, pp. 133–151. URL: https://doi.org/10.1109/FOCS.1961.13 (cit. on p. 71).

[31] J.H. Conway. *Regular Algebra and Finite Machines*. Chapman and Hall mathematics series. Dover Publications, Incorporated, 2012. ISBN: 9780486485836. URL: https://books.google.at/books?id=1KAXc5TpEV8C (cit. on p. 71).

[32] Ezekiel O. Soremekun et al. "Inputs From Hell". In: *IEEE Trans. Software Eng.* 48.4 (2022), pp. 1138–1153. DOI: 10.1109/TSE.2020.3013716. URL: https://doi.org/10.1109/TSE.2020.3013716 (cit. on p. 97).

[33] Esteban Pavese et al. "Inputs from Hell: Generating Uncommon Inputs from Common Samples". In: *CoRR* abs/1812.07525 (2018). arXiv: 1812.07525. URL: http://arxiv.org/abs/1812.07525 (cit. on p. 97).

[34] Leslie G. Valiant. "A Theory of the Learnable". In: *Commun. ACM* 27.11 (1984), pp. 1134–1142. DOI: 10.1145/1968.1972. URL: https://doi.org/10.1145/1968.1972 (cit. on p. 97).

[35] Wojciech Wieczorek. *Grammatical Inference - Algorithms, Routines and Applications*. Vol. 673. Studies in Computational Intelligence. Springer, 2017. ISBN: 978-3-319-46800-6. DOI: 10.1007/978-3-319-46801-3. URL: https://doi.org/10.1007/978-3-319-46801-3 (cit. on p. 98).

[36] Xiaodong Jia and Gang Tan. "V-Star: Learning Visibly Pushdown Grammars from Program Inputs". In: *Proc. ACM Program. Lang.* 8.PLDI (2024), pp. 2003–2026. DOI: 10.1145/3656458. URL: https://doi.org/10.1145/3656458 (cit. on p. 98).

[37] Neil Kulkarni, Caroline Lemieux, and Koushik Sen. "Learning Highly Recursive Input Grammars". In: *36th IEEE/ACM International Conference on Automated Software Engineering, ASE 2021, Melbourne, Australia, November 15-19, 2021*. IEEE, 2021, pp. 456–467. DOI: 10.1109/ASE51524.2021.9678879. URL: https://doi.org/10.1109/ASE51524.2021.9678879 (cit. on p. 99).

[38] Zhengkai Wu et al. "REINAM: reinforcement learning for input-grammar inference". In: *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2019, Tallinn, Estonia, August 26-30, 2019*. Ed. by Marlon Dumas et al. ACM, 2019, pp. 488–498. DOI: 10.1145/3338906.3338958. URL: https://doi.org/10.1145/3338906.3338958 (cit. on p. 99).

[39] Rahul Gopinath et al. *Sample-Free Learning of Input Grammars for Comprehensive Software Fuzzing*. 2018. arXiv: 1810.08289 [cs.SE]. URL: https://arxiv.org/abs/1810.08289 (cit. on p. 99).

[40] Leon Bettscheider and Andreas Zeller. "Look Ma, No Input Samples! Mining Input Grammars from Code with Symbolic Parsing". In: *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. FSE 2024. Porto de Galinhas, Brazil: Association for Computing Machinery, 2024, pp. 522–526. ISBN: 9798400706585. DOI: 10.1145/3663529.3663790. URL: https://doi.org/10.1145/3663529.3663790 (cit. on p. 99).

[41] Weizhi Tang et al. *HyGenar: An LLM-Driven Hybrid Genetic Algorithm for Few-Shot Grammar Generation*. 2025. arXiv: 2505.16978 [cs.AI]. URL: https://arxiv.org/abs/2505.16978 (cit. on p. 108).